

ASTANA IT UNIVERSITY

School of Software Engineering



]

NURKANAT KENESBEK

RZAKUL BAKDAULET

OZAT KASSIYET

**DEVELOPMENT OF A SECURE
MOBILE APPLICATION TO
ENHANCE THE SAFETY OF ONLINE
DATING**

6B06102 – Software Engineering

Diploma project

Kazakhstan Republic

June 2026, Astana

ASTANA IT UNIVERSITY
School of Software Engineering



**DEVELOPMENT OF A SECURE
MOBILE APPLICATION TO
ENHANCE THE SAFETY OF ONLINE
DATING**

6B06102 – Software Engineering
Diploma project

Authors:

Nurkanat Kenesbek
Rzakul Bakdaulet
Ozat Kassiyet

Scientific Supervisor:

Makatov Asrtan

Supervisor signature: _____

A handwritten signature in blue ink, appearing to be 'Asrtan', written over a horizontal line.

Kazakhstan Republic
June 2026, Astana

Contents

Abstracts	v
Definitions	viii
Designations and Abbreviations	1
Introduction	2
1 Methodology	10
1.1 Development Methodology: Agile with Scrum	10
1.2 Domain-Driven Design Application	11
1.2.1 Identifying Bounded Contexts	11
1.2.2 Ubiquitous Language	12
1.2.3 Aggregates and Entities	13
1.2.4 Repository Pattern	13
1.3 Empirical Survey and Focus Group Metrics	14
1.3.1 Functional Requirements	14
1.4 Testing Strategy	16
1.5 Justification of Methodology	16
2 Literature Review	18
2.1 Online Dating Platforms: Evolution and Behavioural Research	18
2.2 Islamic Marriage Practices and the Role of Technology	19
2.3 Recommendation and Matching Algorithms	20
2.4 Real-Time Communication Architecture	21
2.5 Software Architecture for Complex Domains	22
2.6 Cross-Platform Mobile Development	24
2.7 Literature Review Summary	24
3 Analysis of Existing Systems	26
3.1 Overview of the Competitive Landscape	26
3.2 Analysis of Individual Systems	26
3.2.1 Tinder	26
3.2.2 Bumble	27
3.2.3 Muzz (formerly Muzmatch)	27
3.2.4 Salams (formerly Minder)	28
3.2.5 Hawaya (by Match Group)	28

3.2.6	Regional and Traditional Matchmaking	29
3.3	Functional Comparison Table	29
3.4	SWOT Analysis	31
3.5	Gap Analysis	32
3.6	Data Collection and User Research	32
3.7	Challenges Unique to Islamic Matrimony Applications	33
4	System Architecture and Design	35
4.1	System Architecture Overview	35
4.2	Backend Clean Architecture Layers	36
4.3	Backend Module Architecture	38
4.4	Database Architecture	40
4.5	WebSocket Architecture	41
4.6	UML Diagram Specifications	42
5	Technology Selection and Justification	54
5.1	Backend Language and Runtime	54
5.1.1	Comparative Analysis	54
5.1.2	Decision	55
5.2	Mobile Framework	55
5.2.1	Candidates Evaluated	55
5.2.2	Comparative Analysis	56
5.2.3	Decision	57
5.3	Social Graph Database	58
5.3.1	The Case for a Dedicated Graph Store	58
5.3.2	Decision	59
5.4	Real-Time Communication Protocol	59
5.4.1	Candidates Evaluated	59
5.4.2	Decision	61
5.5	Caching and Session Store	61
5.6	Object Storage	62
5.7	Summary	63
6	Implementation and Deployment	64
6.1	Overview	64
6.2	Backend Implementation	64
6.2.1	Project Bootstrapping and Database Migrations	64
6.2.2	Authentication Implementation	65
6.2.3	Profile and Media Implementation	65
6.2.4	Matching Algorithm Implementation	66

6.2.5	Trust Engine Implementation	67
6.2.6	Real-Time Chat Implementation	67
6.2.7	Mahram, Imam, and KYC Features	68
6.3	Frontend Implementation	68
6.3.1	Architecture and State Management	68
6.3.2	HTTP Client and Interceptors	69
6.3.3	WebSocket Client	69
6.3.4	Key Screen Implementations	70
6.3.5	Islamic Design System	71
6.4	Security Implementation	71
6.4.1	Transport Security	71
6.4.2	Input Validation and Sanitisation	72
6.4.3	SQL Injection Prevention	83
6.4.4	CORS and Origin Validation	83
6.4.5	Content Moderation	83
6.5	Deployment Pipeline	83
6.5.1	Docker Compose Stack	83
6.5.2	Nginx Configuration	84
6.5.3	Environment Configuration	85
6.5.4	Continuous Integration	85
6.5.5	Makefile Developer Experience	85
6.6	Testing Summary	87
6.7	Demo Walkthrough	87
	Conclusion	89
	References	89
	Appendix A: Code Listings	93
	Appendix B: Mobile Platform Interface	101

Abstracts

Abstract

This diploma thesis is a detailed narrative that details the research, design, development and deployment of TrueConnect, a unique technology matchmaking and matrimonial service platform designed to satisfy Islamic jurisprudence (Sharia) and quality of development of software engineering. Traditional applications in digital matchmaking devices commonly have low-friction, picture-based interfaces and swipe-based gamification that favour spontaneous interaction, in opposition to Islamic ethical norms of courtship, privacy, and guardianship.

TrueConnect overcomes this socio-technical divide by implementing a proprietary software ecosystem built in a Custom-Designed Digital DDD driven software system via a Modular Monolith model. Its architecture consists of a Guardian (Mahram) institutionalized object invocation layer, a dual-schema identity vault for consistent identity validation and encrypted end-to-end communication channels. The backend is written up in Go (Golang 1.25) with its high performance, concurrent, goroutine-per-connection state machine, while the cross platform mobile client is realized by Flutter (Dart) utilizing advanced state management via Riverpod. There is no one-shot data management, with PostgreSQL based on PostGIS extensions that allow for geospatial proximity queries, and real-time processing across all the geospatial proximity interactions via customized WebSocket hubs, supported by a horizontal Redis Pub/Sub layer. Moreover, the platform uses graph database Neo4j with Graph Data Science (GDS) to calculate a decentralized community trust coefficient with Louvain and PageRank algorithms. The deployment infrastructure is all implemented in Docker with Nginx as a reverse proxy as well as with compliance with the personal data localization law concerning the resident virtual private servers regulations of the Republic of Kazakhstan.

Аннотация

В статье представлен подробный обзор исследований, проектирования, разработки и внедрения, использованных при создании TrueConnect, новой цифровой платформы для знакомств и поиска партнера, разработанной в соответствии с исламским правом и с соблюдением высочайших стандартов разработки программного обеспечения. Обычные цифровые приложения для знакомств часто включают в себя удобные интерфейсы, основанные на фотографиях и жестах, играя роль геймификации, что поощряет неформальное общение и взаимодействие, в отличие от исламских этических принципов ухаживания, конфиденциальности и опеки.

TrueConnect решает эту социотехническую задачу, разрабатывая собственную программную экосистему на основе принципов проектирования, ориентированного на домен (DDD), и модульного монолитного дизайна. Система состоит из: институционализированного уровня вызова опекуна (махрама), хранилища данных с двойной схемой для проверяемой идентификации и зашифрованных путей связи от конца до конца. Бэкенд построен на Go (Golang 1.25) с высокопроизводительной конкурентной конечной автоматикой и горутинами на соединение, а кроссплатформенный мобильный клиент разработан на Flutter (Dart) с продвинутым управлением состоянием через Riverpod. Организация постоянных данных осуществляется с помощью PostgreSQL с расширениями PostGIS для геопространственных запросов близости, взаимодействие в реальном времени через пользовательские WebSocket-хабы, которые общаются через горизонтальный слой в Redis Pub/Sub. Платформа также использует графовую базу данных Neo4j, которая имеет функции Graph Data Science (GDS) для вычисления децентрализованного коэффициента доверия сообщества с использованием алгоритмов Лувена и PageRank. Вся инфраструктура развертывания контейнеризована на Docker с Nginx в качестве обратного прокси в соответствии с юридическим мандатом Республики Казахстан, который предписывает локализацию персональных данных на резидентных виртуальных частных серверах.

Аннотация

Бұл диссертация TrueConnect атты сапалы сандық құралды зерттеу, жобалау, әзірлеу және енгізу бойынша егжей-тегжейлі шолу жүргізуді мақсат етеді, ол ең жақсы қолданылатын бағдарламалық инженерия стандарттарына және шариятқа сәйкес әдістемеге негізделген. Дәстүрлі сандық сәйкестендіру қосымшасы әдетте төмен кедергісі бар, фотоға негізделген интерфейстерді және кездейсоқ өзара әрекеттесуді ынталандыратын, исламдық танысу, құпиялылық және қамқоршылық этикасын тікелей бұзатын, сырғытуға негізделген ойын механикасын қолданады.

TrueConnect бұл әлеуметтік-техникалық алшақтықты доменге бағытталған дизайн (DDD) принциптері мен модульдік монолиттік архитектураға негізделген толықтай арнайы бағдарламалық экожүйені пайдалана отырып жоюға тырысады. Біз махрам, «институционалдық қамқоршы» шақыру қабаты, тексерілген тұлғалар үшін қос схемалы тұлға тексеру репозиторийі және шифрланған соңғы байланыс бар қосымшаны ұсынамыз. Артқы жағы Go (Golang 1.25) негізінде құрастырылған, әрбір байланыс үшін жоғары өнімділік, бір уақытта орындалатын горутина күй машинасы және Flutter (Dart) тілінде жазылған кросс-платформалық мобильді клиент, Riverpod көмегімен жетілдірілген күйді басқару қуатын пайдаланады. Тұрақты деректерді басқару геокеңістіктік жақындық сұраулары үшін PostGIS кеңейтімдері бар PostgreSQL негізінде құрастырылған, ал нақты уақыттағы байланыс көлденең Redis Pub/Sub қабаты бар арнайы WebSocket хабтары жиынтығы арқылы жүзеге асырылады. Сонымен қатар, біз Louvain және PageRank алгоритмдерін пайдалана отырып, орталықтандырылмаған қауымдастық сенім ұпайларын алу үшін Graph Data Science (GDS) мүмкіндіктері бар Neo4j графикалық дерекқорын қолданамыз. Барлық орналастыру инфрақұрылымы Docker негізінде және Қазақстан Республикасының резиденттік виртуалды жеке серверінің жеке деректерін локализациялау бойынша міндетті талаптарға сәйкес Nginx кері прокси ретінде пайдаланылады.

Definitions

- **Imam Connect:** A module for integrating verified clergy (imams) into a digital ecosystem for recording and blessing marriages.
- **Khalwa:** An Islamic legal and ethical concept meaning the privacy of a man and a woman without the presence of third parties.
- **Mahram:** A close male relative of a woman (father, brother, uncle) who does not have the right to marry her and acts as her legal guardian.
- **Mahram Chat:** A dedicated secure three-way real-time communication channel.
- **Nikah:** Marriage under Islamic law requires strict conditions: the consent of the parties, the presence of a guardian and witnesses.
- **Niyyah:** The spiritual and legal principle of awareness of intention in Islam.
- **Frontend Interface:** The portion of the system that users directly interact with via a mobile application or web browser.
- **Backend System:** The server-side element in charge of handling inter-module operations, data storage, and business logic processing.
- **PostgreSQL Database:** An object-relational database system that is open-source and used to manage and store structured data.
- **REST API:** A kind of web service that uses resource-based URLs and common HTTP functions like GET, POST, PUT, and DELETE to enable system-to-system communication.

Designations and Abbreviations

Following designations and abbreviations are used in this work:

ACID	Atomicity, Consistency, Isolation, Durability
AES-256-GCM	Advanced Encryption Standard (Galois/Counter Mode)
API	Application Programming Interface
Argon2id	Argon2id Password Hashing Function
BLoC / Riverpod	Business Logic Component / Riverpod state framework
BPMN	Business Process Model and Notation
CSP	Communicating Sequential Processes
DDD	Domain-Driven Design
DTO	Data Transfer Object
ERD	Entity-Relationship Diagram
GDS	Graph Data Science
JWT	JSON Web Token
KYC	Know Your Customer
MinIO	MinIO Object Storage
NFR	Non-Functional Requirements
PII	Personally Identifiable Information
PostGIS	PostGIS Spatial Extension
RBAC	Role-Based Access Control
REST	Representational State Transfer
UML	Unified Modeling Language
VPS	Virtual Private Server
WebRTC	Web Real-Time Communication
WSS	WebSocket Secure

Introduction

Background and Motivation

Marriage itself has a very significant place in Islamic theology and jurisprudence. The Prophet Muhammad (peace be upon him) described marriage as completing half of one's faith, and the Quran (30:21) depicts it as a source of tranquillity, love, and compassion - qualities in stark contrast to the transactional, gamified nature of modern digital match-making. Although 1.8 billion Muslims exist on earth, comprising around 24 percent of the world's population, finding a spouse that shares Islamic values is proving to be a major social and technical issue in an increasingly digital society that has yet to receive an adequate engineering solution.

The explosion of mainstream dating apps such as Tinder, Bumble, and Hinge has fundamentally changed the way we build romantic relationships in this modern age. Their design philosophy holds: A photo-first platform that's built for making rapid, low-friction screens on potential partners, a swipe-based gamification mechanism that rewards volume over intentionality, and an inherently relaxed assumption of casual or exploratory intent. All these design choices would violate existing beliefs and practices from an Islamic standpoint. The lack of strict rules surrounding the practice of mixing unrelated men and women (known as *khalwa* in Arabic), the importance of physical appearance over character and piety, and the lack of express commitment to a serious marital relationship (*niyyah*) make these platforms an utter anathema to Islamic courtship standards.

A number of apps even claim to be Islamic replacements for mainstream dating sites. The best-known such apps are Muzz (formerly Muzmatch), Salams (formerly Minder), and Match Group's Hawaya. However, both on a technical and functional level, we see that most of these platforms are merely adaptations of Western dating application patterns applied by a few at best, with an Islamic superficial branding, as opposed to Islamic domain-oriented platforms. None of these applications includes a functional mahram supervision system; there is no actual mahram monitoring system in place, which has been implemented (the Islamic concept of a guardian whose presence is required to accompany unmarried individuals in discussions among them). None supports filtering for the potential matches by madhab (Islamic school of jurisprudential thought; Hanafi, Shafi'i, Maliki, Hanbali). In addition, no other application of *niyyah* is an official first-class concept that frames the algorithmic matching process on the platform.

The reality of Kazakhstan and Central Asia in particular raises the necessity and the prospect, too. Kazakhstan has a Muslim majority population of about 70 percent, a rapidly increasing smartphone penetration rate, and a tradition of community-mediated marriage.

But for this region, we do not offer a locally developed, locally hosted Islamic matrimony platform. This void means that Kazakhstani Muslim users looking for help making digital matches have to use either platforms that conflict with their religious values or platforms tailored to Middle Eastern or South Asian cultural experiences that are very different from the Central Asian Muslim experience.

The technical argument for this project is equally convincing. Islamic dating applications that already exist approach religious requirements as UI elements, profile badges, modesty toggles, rather than as architectural constraints. TrueConnect was built from the ground up with Islamic domain concepts at every layer of the design, which has the domain model, the database schema, the matching algorithm (niyyah compatibility filtering, madhab affinity scoring), the communication architecture (mahram chat room system), and the user interface. This architectural belief, which considers Islamic values as a set of technical criteria rather than as an add-on, is the hallmark of this project.

Problem Statement

A systematic review of digital marriage solutions, together with an analysis of the technical and social needs of Muslim Kazakhstan users, provides details on five separate and well-articulated problems that the available generation of applications does not solve.

Problem 1: Lack of mahram (guardian) supervision in online communication channels. Legal principles of Islam under all four significant traditions of Islamic jurisprudence state that private communication between an unmarried man and an unrelated woman, where there is no mahram at hand, is prohibited. No popular or Islamic-branded type of dating app is currently available that offers an authentic technical implementation in this realm. At the most superficial stages of existing platforms, it is only through a voluntary “wali awareness” notification - informing a designated guardian that a woman is using the platform - that a genuine supervised communication can take place at all. The result is an irreconcilable dilemma confronted by observant Muslim women between the use of digital matrimony tools and fulfilling their religious duties.

Problem 2: Could not filter potential match by madhab (Islamic school of jurisprudence). Madhab compatibility is an important and meaningful dimension of marital compatibility among observant Muslims. Differences between a Hanafi and a Hanbali Muslim may significantly affect long-term marital harmony in their practice and daily lives; these range from the practices they perform in their daily religious life to their rituals among family members, in their food choices and lifestyle. No existing platform surfaces or filters on Madhab as a matching criterion. This gap forces users to discover this incompatibility only after extensive communication, wasting significant time and emotional investment.

Problem 3: No formal declaration of niyyah (marital intention) for the algorithm to be based on, which algorithmic behaviour to draw upon. Mainstream platforms implicitly

assume casual or exploratory intention. Islamic-branded apps that do have a “niyyah” profile field even run with it, also treat it as a label on the screen, rather than a parameter that defines who will be shown in a user’s discovery feed. This in effect, means that a user who seeks a serious marriage in the year (nikah year) would see candidates who are interested in friendship only, and vice versa, a divergence that degrades the matches made and exposes the user to potentially offensive interactions based on their stated intentions.

Problem 4: The dominant applications encourage superficial, high-volume swiping behaviour that is counter to Islamic courtship values. The mechanics of gamification in swipe-based discovery — unlimited likes, pictorial-first card design, badge counts — are all designed to maximise engagement time and number of swipes. Such design philosophy is highly antithetical to the Islamic value of intentional and deliberate choice of spouse, which is based on the quality of character, piety, and compatibility rather than physical characteristics. A Muslim-oriented platform has to create friction that makes it hard to go, not mechanics that reward volume.

Problem 5: No trust/reputation ecosystem that mirrors actual character and Islamic honesty. In Islamic marriage tradition, the testimony of others and the views of others are important to a person’s character (wali and community witnesses). There has been no equivalent mechanism on digital platforms. Fake profiles, misrepresentation, and Sybil attacks undermine user safety and trust in ways that are especially destructive in a matrimony landscape where users are making life-altering decisions. Without a verified, community-oriented reputation system, Muslim users do not have the option of finding a digital counterpart to the community-based character identification traditionally applied in Islamic matchmaking.

Overall: a lack of a mobile matrimony platform in the Islamic domain, mahram supervision, madhab compatibility, niyyah filtering, courtship mechanics, and trust scoring based on community foundation, is the core problem being solved, without a first-class architectural solution. TrueConnect was designed and deployed specifically to address this dilemma concerning Kazakhstani Muslim culture.

Significance and Impact

The importance of TrueConnect spans four key aspects: the social, technical, economic, and research dimensions.

Social Impact. For Muslim users in Kazakhstan, the presence of a platform that is rooted in Islamic courtship culture that actively regulates itself means tangible and meaningful things to them in their daily lives. A Kazakh Muslim woman who’s using TrueConnect is able to talk to the would-be bride when she believes that a mahram is with her; technically, she’s not merely a courtesy alert, but in a three-way encrypted chat channel (social.mahram chat rooms). She’s able to screen discovery results by niyyah and madhab and stop them from running through manual post hoc screening. Candidates can be confirmed through a

community trust score based on genuine interactions and KYC identity verification as opposed to self-reported statements. These aren't just tweaks to user experience, they're the difference between a platform a religiously observant woman can use with a conscience and another she can't. For Muslim men, the platform offers a "symmetrical benefit" of sorts: a pool of candidates pre-filtered for serious marital intent, a trust score system that rewards real integrity over a profile's attractiveness, and a formalized courtship path - niyyah declaration → profile view → match → supervised chat → family introduction → imam connection → nikah confirmation – that mimics rather than contradicts the Islamic courtship process.

Technical Significance. From a software engineering point of view, TrueConnect adds to the nascent field of values-driven application design. The design showcases a clear approach of integrating domain dependent ethical and religious needs at every layer of production system: domain model (internal/domain/profile.go with niyyah and madhab as typed enums), database schema (for example the social.mahrims table, identity vault dual-schema system), matching algorithm (niyyah compatibility filter in internal/service/matching_service.go), and GUI (NiyyahSelectionScreen as mandatory onboarding check before user discovery ability to be accessed).

Economic Opportunity. The Islamic digital economy is a relatively fast-growing and fast-expanding sector. The halal industry across the world is worth more than USD 2 trillion globally, but digital services, particularly social platforms, account for only a small proportion of this. In Kazakhstan at present, there is no locally hosted Islamic matrimony marketplace that is commercially exploited. TrueConnect is a non-competing market. The self-hosted architecture (PostgreSQL + Neo4j + Redis + MinIO implemented on a Kazakhstani VPS) provides all users' data, including the sensitive identity vault data (IIN, full legal name) in legal Kazakhstan, a vital competitive edge in the light of changing data localisation needs in Kazakhstan.

Research Contribution. The following contributions directly relate to academic knowledge in three aspects: (1) application of Clean Architecture and Domain-Driven Design model to Islamic social domain model (where ubiquitous language niyyah, mahram, madhab directly map to Go structural names, database column names, and API end-points); (2) community-grounded trust score system design and implementation using Neo4j graph algorithm (weighted PageRank with Bayesian smoothing) for social matchmaking context; and (3) three-party real-time communication system in supervision by WebSockets and Redis pub/sub which would be encrypted AES-256-GCM end-to-end for all of participants.

Research Objectives

The following objectives guided the building, designing and evaluation of TrueConnect. Each objective is based on a particular feature proven in the codebase (docs/codebase map.md).

1. Design and implement a niyyah-based matching algorithm. Redis and PostgreSQL were utilized to make sure that if any users had incompatible marital intentions (e.g., friendship or serious marriage within a year), they could not see each other in the discovery feed, thus eliminating the primary cause of the intent mismatch encountered in the current Islamic dating apps.
2. To construct a three-way encrypted mahram chat channel. Implementing WebSockets, AES-256-GCM symmetrical encryption, and Redis pub/sub to provide an architecture equivalent to the physical mahram presence in courtship formal supervision via an enforced Islamic guardian mechanism.
3. To create a trust and reputation scoring engine based through community. With Neo4j graph database algorithms (weighted Bayesian scoring with KYC verification multipliers) to give users proof of a true, non-manipulative way of showing a candidate's character and Islamic integrity, instead of the self-reporting testimony through community reports.
4. A full Islamic courtship route. As a well-defined series of verifiable steps niyyah declaration, match formation, supervised chat, introductions to family
(POST /v1/matches/:id/family-intro), connections to imams (GET /v1/imams), and affirmation of nikah
(POST /v1/matches/:id/nikah-confirm) to link the digital courtship process with customary Islamic marriage practices.
5. To develop a privacy-sensitive identity verification system. Employing a dual schema PostgreSQL architecture (social + identity vault) with AES-256-GCM field-level encryption/ Argon2id hashing, to protect Kazakhstani national identity numbers (IIN) and legal name without accessing via application-layer attacks, complying with Islamic privacy expectations and Kazakhstani data sovereignty regulation.
6. To design and implement a modesty-aware discovery interface. Employ Flutter and the no photo mode profile feature to offer women very granular control of what pictures are shown on display to the users, so that avatar images should never be displayed in the discovery feed without user consent, while maintaining the correct Flutter DiscoveryScreen handling of the AvatarBlurred flag.
7. Develop a solid self-powered mobile backend. With Go 1.25 + Gin, a modular monolith Clean Architecture, Docker Compose orchestration (PostgreSQL 16 + PostGIS, Neo4j 5, Redis 7, MinIO), along with an nginx reverse proxy, you can achieve production-quality reliability and allow maximum data sovereignty on the Kazakhstani infrastructure.

8. To build and validate a wide range of test suites. Using Go’s testing package with the -race detector for all service-layer business logic, attaining 102+ passing tests that confirm all essential Islamic domain invariants like niyyah compatibility, block propagation, mutual match atomicity, and trust score computation.

Scope and Limitations

In Scope. As we have designed, implemented, and validated the following features in the project: user registration and authentication (JWT HS256, Argon2id, AES-256-GCM PII encryption); profile creation and management with Islamic domain fields (niyyah, madhab, languages, no-photo mode, marital status); geolocation-based discovery (PostGIS ST DWithin queries on social.profiles.location); niyyah and madhab filtering in the matching algorithm; swipe mechanics (like and pass with seen-set management); mutual match detection and creation; real-time WebSocket chat, AES-256-GCM message encryption, and Redis pub/sub for horizontal scalability; mahram chat rooms (three-party supervised communication), community feed (posts, likes, comments); post-meeting interaction ratings with trust score recalculation; KYC identity document submission and verification; notification system with FCM push delivery; discovery settings (age range, distance, niyyah/madhab filters, modesty level); imam directory (embedded catalog of 10 imams across 5 Kazakhstani cities); whisper anonymous reporting system; admin panel for KYC verdicts and Sybil cluster review; and a complete Flutter mobile application implementing all of the above features across 21 screens.

Out of Scope. The current version explicitly lacks the following features: phone number OTP verification (deferred to a post-graduation sprint); video or audio calling beyond the WebRTC signaling infrastructure that has been implemented; a web-based frontend (the web/ directory contains scaffolding but was not developed as part of this project); payment processing or premium subscription tiers; push notification delivery to iOS devices in production (Firebase Messaging is configured but full APNs provisioning requires an Apple Developer account); and automated content moderation beyond the is toxic flag in the message schema.

Current Limitations. In the process of developing, a number of technical limitations were identified, and these have been recognized along with intellectual integrity. To start off, the trust score channel (chan domain.PushEvent with capacity 100) can drop events under extreme load — its buffer size is still high enough to achieve the expected user volume at launch. Second, the Neo4j advisory lock for the RecordLike mutual match detection has not been implemented, providing a theoretical race condition for simultaneous identical like requests (mitigated practically by application-level uniqueness constraints in the implementation). Third, imam and halal venue catalogs are embedded at compile time and require a binary redeployment to update; this is acceptable for the existing scale because this would not be an admin CMS option but, on a larger scale, an admin CMS would be necessary.

Fourth, the Flutter application has been tested on Android (API 21+) and the web; iOS testing was limited by the absence of an actual iOS hardware device and Apple Developer enrollment.

Structure of the Document

The thesis is arranged in seven chapters, each of which tackles different aspects of the TrueConnect system. The first chapter (Introduction), the present chapter explains the motivation, defines the five problems the project tackles, explains the importance of the work under social, technical, economic, and research areas, puts forth eight strong research objectives based on features that have been implemented, and defines the scope and known limitations of the system. Chapter 2, Methodology, describes the development methodology (Agile with iterative sprints), the methodology applied by Domain-Driven Design principles to the Islamic domain model (bounded contexts, ubiquitous language, aggregates), the requirements engineering process (functional and non-functional requirements tables), the testing strategy adopted and the rationale for choosing these methodologies. Chapter 3 (Literature Review) reviews the academic and technical literature on six key topics pertaining to this project: evolution of online dating platforms and the impact on behavior; Islamic marriage practices and digital technology in Muslim context; recommendation and matching algorithms; real-time communication architectures; patterns of software architecture in complex domains; and mobile development of cross-platform applications. A minimum of twenty peer-reviewed references are included in the citations. Chapter 4 (Analysis of Existing Systems) comprises a structured competitive analysis of the available frameworks for five established platforms (Tinder, Bumble, Muzz, Salams, Hawaya), a comparative table for functional features of all-Islamic compliance criteria, a comprehensive SWOT analysis of TrueConnect, and a gap analysis related to the actual codebase that corresponds to each deficiency identified. Chapter 5 (System Architecture and Design) describes the complete technical specifications of the architecture: three-tier architecture, five layers of Clean Architecture, all backend modules, database design, WebSocket Hub architecture and the textual specifications on the eight UML diagrams (i.e use case, system architecture, five sequence diagrams, ERD, class diagram). Chapter 6 (Technology Selection and Justification) summarizes comparisons between the tech choices which were made at each layer: Go versus Node.js and Python for the backend; Flutter versus React Native and native development for the frontend; PostgreSQL versus MongoDB and Firebase Firestore as the primary storage; WebSocket versus alternative real-time options; and the role of Neo4j for graph-based trust computation. We explain here the entire implementation process (REST API endpoints documentation, frontend-backend integration architecture (Dio interceptors, Riverpod state flow), full technical detail about WebSocket protocol, a deep dive of four key features (matching engine, mahram chat, feed, first meeting rating), security implementation (JWT, Argon2id, AES-256-GCM, nginx

rate limiting), UI/UX design philosophy and DevOps stack (Docker, nginx, CI/CD pipeline, database migrations)) in Chapter 7 Implementation and Deployment.

1 Methodology

1.1 Development Methodology: Agile with Scrum

TrueConnect was created using an Agile methodology, organized around iterative sprints, each delivering a vertical slice of production-quality functionality. This choice was made due to the specific nature of the problem domain: designing an Islamic matrimony application is a novel engineering challenge with little prior experience. This means that requirements could not be fully defined in advance and had to emerge from the iterative development of applications and repeated validation against domain knowledge.

The project was divided into fourteen sprints (Sprints 0–13), each focused on a coherent functional area. Sprint 0 established the foundational scaffolding: monorepo structure, Docker Compose orchestration, database connection pool initialization, health check endpoints, and all inter-service connectivity. Sprint 1 provided the complete authentication infrastructure (registration, login, JWT issuance, Argon2id password hashing, AES-256-GCM PII field encryption, refresh token rotation). Sprints 2 through 5 iteratively developed the core matchmaking capabilities (profiles, discovery, matching, settings, photo upload, social feed, KYC stub, WebSocket chat, input sanitization). Sprints 6 through 9 provided the Islamic domain-specific features that set TrueConnect apart from all competitors, namely the halal identity schema extensions (migrations/000013 halal identity fields.up.sql, 000014 mahram.up.sql, 000015 halal matches ext.up.sql), niyyah compatibility filter (B1), madhab affinity boost (B2), no-photo modesty enforcement (B3), and niyyah 90-day timer. Sprints 10 to 11 completed the mahram chat architecture, the family introduction milestone, the embedded imam catalog, and the admin whisper-flag review system. In Sprints 12 and 13, the full Flutter mobile frontend was released on 21 screens with the design system.

User stories in the project were written in Islamic domain language and directly influenced the general language of the codebase. Examples include: "As a Muslim woman wanting to marry, I don't want to let friends look at my images until I pass them along, so that my digital exposure complies with Islamic modesty norms" leading to the no photo mode architectural feature; "As a user with nikah year niyyah, I want only candidates sharing my serious marital intent to view my profiles, so my time is not wasted on incompatible conversations" which led to the filter AllowedNiyyahs for FindCandidatesOpts; and "As a Muslim woman, I want my mahram to be an active participant in my dialogues and not just sent notifications after the fact" which led to the mahram chat room system. Priority for backlogs was dependent; authentication had to be established before all the features were built, profiles were needed for matching, matching needed to be implemented before chat, and Islamic domain extensions were placed on the stable core in the following sprints. As a result, the architecture was

born from forward-looking learning: the early sprints showed that adding Islamic features as overlays to a secular core was fragile and required workarounds, prompting us to add Islamic fields at the schema level, not as application-level annotations, in Sprint 6. This architectural lesson that Islamic compliance should be integrated into the data model rather than being an additional feature imported to it is the greatest methodological impact of the project.

1.2 Domain-Driven Design Application

The application of Domain-Driven Design (DDD) delivered the structural vocabulary that enables TrueConnect’s intricate Islamic domain to be expressed faithfully in code. The DDD approach in this project isn’t purely theoretical: the bounded contexts, ubiquitous language terms, aggregates, and repository abstractions we describe are directly observable in the Go source code, PostgreSQL schema, and Flutter model definitions.

1.2.1 Identifying Bounded Contexts

DDD proposes the partitioning of a complex domain into bounded contexts — sub-domains whose design is consistent and which have clearly defined boundaries. Every TrueConnect backend module corresponds to a bounded context, which has clear responsibilities and interface contracts.

Auth Context (internal/handler/auth handler.go, internal/service/auth service.go): Responsible for identity establishment and session lifecycle. Owns the concepts of credential, token, and session. Communicates with other contexts only through the User.ID (a UUID that serves as the cross-context identity key) and the VerificationLevel enum that signals KYC status to the Reputation context. Profile Context (internal/handler/profile handler.go, internal/service/profile service.go): User profile and Islamic identification. Owns Profile, PromptAnswer, Niyyah, Madhab, and Media. The "profile" context is the canonical owner of Islamic domain vocabulary – the niyyah, madhab, no photo mode, and languages fields are defined and passed across to the Discovery context users through filters. Discovery Context (internal/handler/matching handler.go, internal/service/matching service.go): Handles the candidate recommendation algorithm and like/pass interaction. Owns LikeResult, CandidateCard, and FindCandidatesOpts. Cues profile and settings data from Profile context as well as Settings context. The niyyah and madhab filtering logic the algorithmic embodiment of Islamic compatibility — all reside within this bounded framework. Chat Context (internal/handler/chat handler.go, internal/service/chat service.go): The real-time messaging and the WebSocket Hub. Owns Message, Hub, and Client. Messages are encrypted before they are stored and decrypted on retrieval entirely within this framework, so that no other context can handle plain text messages. Mahram Context (internal/handler/mahram

handler.go, internal/service/mahram service.go) This provider works with the guardian supervision framework. Has ownership of Mahram, MahramRoom, and MahramMessage. This structure is architecturally isolated because its access control rules — who can read and who can write messages in a mahram channel — are specific, and cannot be generalised across Chat without blurring the two different communication models. Reputation Context (internal/service/interaction service.go, internal/service/reputation service.go, internal/worker/trust engine.go): Responsible for the community trust score system. Owns Interaction, TrustScore, and LeaderboardEntry. Interfaces with the Neo4j adapter to compute the score using the graph and uses Redis for caching score. Feed Context (internal/handler/post handler.go, internal/service/post service.go): The community social feed operator. Takes care of Post, also PostComment, and the like and unlike ops. The reputation gate in CreatePost (30 trust level) depends upon the score of the Reputation context. Settings Context, KYC Context, Notifications Context, Whisper Context, Imam Context, Promising Context, and Admin Context are smaller but equally well (if not even smaller) bounded contexts with domain concepts independent. At the Go package level, we make sure there are only a few context boundaries: internal/service/auth service.go file imports only the internal/domain/ and internal/repository/ interfaces never internal/adapter/ packages so that business logic of one context cannot directly manipulate the data stores of another context.

1.2.2 Ubiquitous Language

The DDD concept of ubiquitous language, identical terminology used by domain experts, developers, designers, and the codebase, is demonstrably implemented in TrueConnect. The following Islamic domain terms appear with identical spelling and semantics across all layers of the system: **Niyyah** **appears** as the ‘domain.NiyyahNikahYear’, ‘domain.NiyyahSeriousMarriage’, ‘domain.NiyyahFriendship’ (Go enums in ‘internal/domain/profile.go’); as the ‘niyyah’ column in ‘social.profiles’ (PostgreSQL schema); as ‘AllowedNiyyahs []string’ in ‘repository.FindCandidates’; as the ‘/niyyah’ route in GoRouter (‘lib/core/router/app router.dart’); as ‘Profile.niyyah’ in the Flutter model (‘lib/models/profile.dart’); and as the ‘NiyyahBadge’ widget (‘lib/widget-s/niyyah badge.dart’). The term travels from the Islamic jurisprudential concept through the UX layer without any semantic transformation. **Mahram** Appears as ‘domain.Mahram’ (Go struct); as ‘social.mahrams’ and ‘social.mahram chat rooms’ (PostgreSQL tables); as the ‘POST /v1/mahram’ API endpoint; as the ‘mahram chat msg’ WebSocket message type; as the ‘/mahram-chat/:roomId’ route in GoRouter; and as ‘MahramChatScreen’ in Flutter. The term’s technical implementation a third-party participant in an encrypted three-way communication channel — is a precise digital translation of the Islamic guardianship concept. **Madhab** Appears as ‘domain.MadhabHanafi’, ‘domain.MadhabShafii’, ‘domain.MadhabMaliki’, ‘domain.MadhabHanbali’, ‘domain.MadhabNone’ (Go enums); as the ‘madhab’ column in ‘social.profiles’; as ‘Profile.madhab’ in Flutter; and as the ‘Madhab-

Badge' widget. The madhab affinity boost in 'matching service.go' (B2 feature) is the algorithmic expression of this term's significance. **Match** Appears as 'domain.Match' (Go struct with 'UserAID', 'UserBID', 'MatchedAt'); as 'social.matches' (PostgreSQL table); as 'GET /v1/matches' (API endpoint); as 'Match' (Flutter model with computed 'daysLeft-OnTimer'); and as 'MatchesScreen' (Flutter screen). The concept of a bilateral mutual consent, two likes creating a match, is consistent across all layers. **Like/Pass** Appears as 'matchRepo.RecordLike()' and 'matchRepo.RecordPass()'; as 'social.likes' and 'social.swipe rejections' tables; as 'POST /v1/matching/like' and 'POST /v1/matching/pass' endpoints; and as 'matchingNotifier.like(id)' and 'matchingNotifier.pass(id)' in the Flutter provider. The terminology is invariant across the entire stack.

1.2.3 Aggregates and Entities

DDD aggregates are clusters of domain objects that should be treated as a single unit for data consistency purposes, with a root entity that controls access to all members. **User Aggregate** Root entity is 'domain.User' (identified by 'ID uuid'). The aggregate owns 'domain.Profile' (one-to-one, always loaded with the user for profile operations), 'UserSettings' (discovery preferences), and 'RefreshToken' (session lifecycle). The consistency boundary ensures that a user's profile and settings are always associated with a valid, active user account. Deletion of the root ('DELETE /v1/users/me') cascades to all owned entities within the aggregate. **Match Aggregate** Root entity is 'domain.Match' (identified by 'ID uuid', referencing 'UserAID' and 'UserBID'). The aggregate owns the conversation: all 'Message' entities in 'social.messages' where 'match id' corresponds to this match. The aggregate also owns the milestone progression state: 'family intro done' and 'imam confirmed' boolean flags. The consistency boundary prevents messages from existing for a non-existent match (enforced by the 'match id' foreign key with ON DELETE CASCADE in migration 000019). **Post Aggregate** Root entity is 'domain.Post'. The aggregate owns 'PostComment' entities and the 'post likes' join table. The like count and comment count are denormalised into the root entity for read performance (incremented/decremented atomically in the repository rather than computed via COUNT queries). **MahramRoom Aggregate** Root entity is 'MahramRoom' (identified by 'ID uuid', referencing 'MatchID' and 'MahramUserID'). The aggregate owns 'MahramMessage' entities. The three-party access control rule — only the woman, the man from the associated match, and the specified mahram user may read or write messages — is enforced by the 'mahram service.go' layer before any message operation..

1.2.4 Repository Pattern

The repository pattern, as applied in TrueConnect, provides a collection-like interface to domain objects while completely hiding the underlying storage technology from the service layer. Each domain aggregate has a corresponding repository interface defined in 'inter-

nal/repository/': ProfileRepository — Upsert, GetByUserID, FindCandidates, GetLeaderboard MatchRepository — RecordLike, RecordPass, ListMatches, GetMatch, IsMatched, Unmatch, UnmatchByUsers, BlockUser, GetBlockedIDs, GetRejectedIDs, MarkFamilyIntroDone, MarkImamConfirmed, ListMatchViews, GetPendingLikes, FindExpiredNiyjahMatches MessageRepository — Create, ListByMatch, MarkRead PostRepository — Create, List, GetByID, LikePost, UnlikePost, AddComment, ListComments MahramRepository — CreateMahram, ListMahrams, CreateRoom, ListRooms, ListMessages, SendMessage SettingsRepository — Get, Upsert NotificationRepository - Create, List, MarkRead, MarkAllRead InteractionRepository — Create, ListByUser, GetCooldown TrustGraphRepository — UpsertUser, RecordRating, ComputeScore, DeleteUserNode, RunSybilDetection The concrete implementations of these interfaces live in 'internal/adapters/postgres/', 'internal/adapters/neo4j/', and 'internal/adapters/redis/'. Services are constructed by injecting the interface — never the concrete adapter — which provides two critical benefits: full testability (102+ service tests use in-memory mock implementations rather than a real database) and full storage-technology substitutability (the PostgreSQL adapter for 'ProfileRepository' could be replaced with a different database without changing any service code).

1.3 Empirical Survey and Focus Group Metrics

1.3.1 Functional Requirements

ID	Requirement	Priority	Module
FR-01	The system shall authenticate users via phone number and password with JWT token issuance	High	Auth module
FR-02	The system shall support profile creation with Islamic domain fields (niyyah, madhab, languages, no_photo_mode)	High	Profile module
FR-03	The system shall discover candidates within a configurable geographic radius using PostGIS	High	Discovery module
FR-04	The system shall filter discovery results by niyyah compatibility	High	Discovery module, B1
FR-05	The system shall boost discovery ranking for same-madhab candidates	Medium	Discovery module, B2
FR-06	The system shall allow users to enable no-photo mode	High	Profile module, B3

ID	Requirement	Priority	Module
FR-07	The system shall support mutual-like matching	High	Matching module
FR-08	The system shall provide real-time bidirectional chat via WebSocket	High	Chat module
FR-09	The system shall encrypt all chat messages with AES-256-GCM	High	Chat module
FR-10	The system shall allow a woman to add a mahram guardian and create a three-party supervised chat room	High	Mahram module
FR-11	The system shall allow users to submit post-meeting interaction ratings	High	Reputation module
FR-12	The system shall compute a trust score (0–100) using Bayesian-smoothed Neo4j PageRank	High	Trust engine
FR-13	The system shall allow users to submit KYC identity documents	High	KYC module
FR-14	The system shall store KYC data in an AES-256-GCM encrypted identity vault schema	High	Identity vault
FR-15	The system shall provide a community social feed	Medium	Feed module
FR-16	The system shall detect Sybil attack clusters using Neo4j GDS Louvain	High	Trust engine
FR-17	The system shall provide an embedded directory of imams in 5 Kazakhstani cities	Medium	Imam module
FR-18	The system shall record family introduction and nikah confirmation milestones	Medium	Matching module
FR-19	The system shall enforce a 90-day niyyah timer on nikah_year matches	Medium	NiyyahTimerWorker
FR-20	The system shall deliver FCM push notifications	Medium	PushWorker
FR-21	The system shall provide an anonymous whisper reporting system	Medium	Whisper module
FR-22	The system shall provide admin endpoints for KYC verdict and Sybil review	High	Admin module

Table 1: Functional Specification Matrix for Sharia-Compliant Matchmaking, Trust Verification, and Secure Communication Modules

1.4 Testing Strategy

TrueConnect tests the service layer here, which is where all the Islamic domain business logic resides and has been arranged in a way to ensure the highest confidence in that the correctness of the domain is consistent and at no external infrastructure dependency. **Unit Tests (Service Layer)** All 102+ tests are within `internal/service/` as an isolated service test package (black box test of exported service interfaces). Tests include in-memory mock-ups of all repository interfaces defined in `internal/service/sprint2` `mocks` `test.go` and related mock files. Imports mock that are exact copies of what each repository interface says can see (some error conditions being `domain.ErrNotFound` and `domain.ErrForbidden`) without needing any liveDB connection. All the tests are run with Go's `-race` detector enabled (`go test ./internal/service/... -v -race -count=1`) for debugging service logic concurrency bugs which are critical for the trust score computation goroutines and WebSocket hub operations. **Key Test Scenarios** The test package includes: authentication the output token generating and the refreshing token rotation; profile the niyyah filter omission (B1), the no-photo mode blur flag (B3); matching — one-sided like makes sure there isn't a match, mutual like makes sure there's only one match, and self-like returns `domain.ErrInvalidInput`, block user removes match; trust score Bayesian smoothing formula accuracy, KYC weight multiplier, Sybil cluster detection threshold; mahram third-party validation (mahram cannot be a match participant); settings default values applied when no settings row exists, pagination bounds clamping. **Integration Testing** API endpoint integration tests validate the HTTP layer without affecting the database API. The CI pipeline (`ci.yml`) executed `go build ./cmd/api` and `go build ./cmd/worker` on every push to verify that everything is cohesive for every module, and `go vet ./...` for static analysis. **Manual Testing** WebSocket flows (real-time chat, mahram chat, typing indicators, read receipts) and the full Flutter UI (all 21 screens) were validated through manual testing sessions using the 'make seed-halal' demo scenario (Aigerim + Alihan demo users). The complete demo pathway 'make docker-up and make seed-halal' followed by Flutter launch — was validated end-to-end. **Areas Without Automated Coverage** The Flutter frontend has no automated test suite in the current version (widget tests and integration tests are planned for a post-graduation sprint). The Neo4j Louvain Sybil detection algorithm is tested indirectly through integration with the live Neo4j instance in the demo environment, not through unit tests.

1.5 Justification of Methodology

Agile iteration paired with Domain-Driven Design was a perfect fitting system for the complex problems presented by TrueConnect, and both the general software engineering literature and the particular application situation justify that combination. **Why Agile was necessary.** Islamic matrimony application design is an open engineering question, unlike

any other. None of the following reference architectures (technical mahram supervision, trust score in a community, structured Islamic courtship pathway) was available to the project when it was started: a halal-compliant mobile matrimony platform. An incorrect specification would exist in this context, as the Waterfall methodology, where requirements specify the overall structure before anything can be implemented, would have already been broken down. The iterative approach of sprint structure allowed domain requirements to surface slowly; it was only after implementing the basic chat system (in Sprint 4) that the technical gap of an "incomplete notification-based chaperone" feature (as done by Muzz and Hawaya) became painfully apparent, and this directly led to the architectural decision to build a genuine three-party communication channel in Sprint 10. Such a discovery is only able to be made by means of an Agile process. **Why DDD was necessary.** The Islamic domain is rich in semantically charged and culturally specific terms. Without DDD's concept of ubiquitous language, the Islamic terms (niyyah, mahram, madhab) would likely have been replaced by generic software terms (intention type, guardian id, school of thought), losing all of their semantic precision, which makes both the Islamic domain expert and software developer understand code and appreciate it. DDD's bounded context framework also put together a framework that enabled us to delineate the Mahram area in parallel with Chat space: these are different Islamic ideas, each with its own access control provisions, and combining them would have given birth to an unmaintainable service which was not reliable for any given user and given mixed consideration. **Why the modular monolith was preferable to microservices.** As noted in the review of literature [19,20], microservices bring with them the complexity of a distributed system that is only acceptable at a scale and team size that TrueConnect has not yet achieved. The modular monolith allows for microservices to be isolated across modules, where every module has a defined interface contract without the operational stress of service discovery, distributed tracing, and inter-service network calls. The architecture can be used for future extraction of the module itself (each module already provides an interface representation of a service to provide independent service), without the microservices tax on the first development phase cost, though. **Why the combined approach produces superior results.** Agile provides a cadence of iteration for gradually identifying requirements; DDD provides the structural vocabulary to faithfully encode those requirements in code; and the modular monolith provides the deployment simplicity we expect at the current scale. Neither approach on its own would have been adequate: DDD without Agile would've driven to an overly-engineered domain model for requirements that hadn't yet been identified; Agile without DDD would've led to an architecture where Islamic terminology was not uniformly identified among layers, and thus code was impossible to comprehend to experts in the field. The result was a system where the code, the database, the API, and the UI all spoke the same Islamic language, which is the heart of TrueConnect's architectural success.

2 Literature Review

2.1 Online Dating Platforms: Evolution and Behavioural Research

The academic literature on online dating has come a long way since the early years of empirical research in the mid-2000s, moving from the description of the earliest stages of early adoption of the platforms to complex investigations of algorithmic influences on mate selection, user self-presentation strategies, and the psychological impacts of gamified matchmaking. This literature is an important background for TrueConnect, as the platform represents, in some aspects, a strategic architectural counter-argument to some of the patterns of design that exist, but which have been empirically found to dominate the industry. The comprehensive foundational review of the science of online dating was conducted, whose analysis in *Psychological Science in the Public Interest* remains the most cited critical treatment in the field [1]. The authors identify three types of claims in the context of online dating platforms: that computer-mediated communication is superior to face-to-face communication for initial partner evaluation; that access to a large pool of potential partners improves matching outcomes; and that mathematical compatibility algorithms improve matching accuracy. Their systematic review finds the empirical support for each of these claims to be weaker than platform marketing suggests. Crucially for the TrueConnect design, algorithmic matching systems tend to reduce potential partners to sets of profile attributes and thereby fail to capture the interactive chemistry that predicts long-term relationship satisfaction [1]. This finding reinforces the TrueConnect design decision to use constraint-based niyyah and madhab filtering not as a similarity optimisation but as a requirement filter ensuring that users are never shown candidates with fundamentally incompatible intentions while leaving the qualitative dimensions of compatibility to emerge through supervised conversation. The analysis of actual user behaviour on contemporary swipe-based platforms provides quantitative evidence of the pathological dynamics that gamified discovery mechanics create [2]. Using Tinder datasets drawn from multiple urban centres, they also observe such asymmetry in like rates that it seems very plausible that the swipe mechanic generates behaviour more closely resembling slot-machine engagement than deliberate partner evaluation. While male users swipe right on around 46 percent of profiles, females swipe right on around 14 percent, a fact the authors claim is due not to differences in user preference but to interface design. TrueConnect’s own discovery screen (DiscoveryScreen using FlutterCardSwiper) intentionally preserves the familiar swipe metaphor while removing the like-volume incentive: no like counters, no match-percentage badges, and no “super like” mechanics that commoditise interest expression. The economics of online partner matching were rigorously analysed

in a study using data from a major US online dating platform [3]. Results of their study reveal that revealed preferences in partner selection are strongly motivated by income, physical attractiveness (as measured by profile photo ratings), and height for male candidates, attributes that have limited relevance in Islamic spouse selection, where piety, family background, madhab compatibility, and declared intention are given priority. This mismatch between the attribute dimensions rewarded by mainstream matching optimisation and the attribute dimensions favoured in Islamic courtship is one of the key reasons for developing a domain-specific platform rather than customising an existing one. Such self-presentation dynamics in online dating profiles have been extensively analyzed in the literature. Users engage in a process of “selective self-presentation,” revealing information strategically to manage impressions while trying to stay plausible in subsequent face-to-face interactions [4]. This creates a tension between optimistic self-presentation and the Islamic prohibition on deception (ghish). This analysis with objective measurements, finding that male users systematically overstate height and income while female users understate weight in their profiles [5].

TrueConnect’s KYC identity verification system (‘social.kyc submissions’, ‘identity vault.iin vault’) directly addresses this deception dynamic by providing a verifiable identity layer: a KYC-verified badge (communicated via ‘verification level’ in ‘social.users’) signals to other users that the profile has been cross-checked against a government identity document, establishing a baseline of factual accuracy that unverified platforms cannot offer.

2.2 Islamic Marriage Practices and the Role of Technology

Islamic marriage is based on a diverse, well-established jurisprudential tradition that prescribes specific roles, processes, and ethical imperatives that are largely absent in secular matchmaking. Engagement with this literature is important to validate the particular domain design decisions made in TrueConnect decisions that might seem unusual to a secular software engineer, but are driven by religious imperatives. The global Muslim population can be summarized by providing a broad demographic analysis [6], which estimates that the Muslim population will be 2.76 billion by 2050 and account for 29.7 percent of the global population, driven by rapid expansion in Sub-Saharan Africa and the continuing majority status in Central Asia. Kazakhstan also has a very specific profile, and they also mentioned that Kazakhstan is a Muslim-majority country whose population ranks in the region amongst all Muslims (about 70 percent of the population is Muslim), which confirms the market logic of establishing a locally developed platform. The report also offers information on the younger age profile of Muslim populations around the world, both the population most likely to use the matrimony app and the one most accustomed to creating relationships digitally first.

The wider interplay between Islam and digital technology, whose work on what he calls “iMuslims” documents the rich and refined way in which woven forms of digital networks (which he defines as networks used by Muslim communities over time for religious learning, community organisation, and social connection in general) have emerged [7]. According to Bunt, digital Islam is neither a dilution of traditional religion nor an adjacent expansion of it, but an extension of the Islamic tradition of adapting available technology to religious purposes. This framing serves to legitimate the TrueConnect project as a continuation of a documented pattern of Islamic digital practice rather than a novelty or cultural compromise. Muslim women maintain religious identity in digital social environments that were not designed with their needs in mind [8]. Rather than designing a platform that forces observant women to choose between visibility and modesty, TrueConnect treats modesty control as an architectural requirement. When ‘no photo mode = true’ in ‘social.profiles’, the discovery query returns ‘AvatarURL = ""’ and sets ‘NoPhotoMode = true’ in the ‘CandidateRow’ struct, which the Flutter ‘DiscoveryScreen’ renders as a blurred placeholder — technically equivalent to an observant woman choosing to introduce herself by character before appearance. Who argues that Islamic online communities require specific ethical infrastructure — mechanisms for accountability, truthfulness, and guardianship — that secular platforms do not provide [9]. This reasoning is closely connected to the three technical developments in TrueConnect that have no parallel in secular dating applications: the mahram chat room system (guardian accountability), the KYC identity verification (truthfulness infrastructure), and the trust score engine (community-based reputation). The alignment of ethical needs identified in academic papers and the technical features that we adopted in the codebase provides strong post-hoc validation of the design choices made.

2.3 Recommendation and Matching Algorithms

The algorithmic core of every matchmaking network is the candidate recommendation and ranking system. A deep knowledge through review of recommender systems research should be done because of the need to place TrueConnect’s matching approach in terms of the analysis of the recommender systems literature and the reasons given why it was chosen not to incorporate collaborative filtering (the dominant paradigm in commercial recommendation systems) as its architecture in the Islamic matrimony context. Recommender systems are categorized into three main paradigms: collaborative filtering (predicting user preferences from the preferences of similar users), content-based filtering (recommending items similar to those a user has already approved), and hybrid systems that combine both [10]. Collaborative filtering is well-established at work when it comes to applications like music or e-commerce recommendation, whose main shortcoming is its cold-start problem. Within the context of matrimony, collaborative filtering, however, would involve finding candidates whom similar users have liked, a method so flawed that it is both ethically questionable

and counterproductive to the Islamic use case when we cannot observe aggregate similarity metrics based on individual religious adherence, family background, or publicly declared intent.

TrueConnect’s matching algorithm employs a type of content filtering based on constraints: the FindCandidates query in internal/adaptor/postgres/profile repo.go does hard constraint filters (niyyah compatibility using the AllowedNiyyahs list, bidirectional block exclusion, seen-set exclusion, geographic distance using PostGIS ST DWithin) and applies soft preference boost (madhab affinity +10 to TrustScore) to rank candidates from the constraint-satisfying set. This is nearly the same as the preference-based filtering approach found in the hybrid recommender literature, only it is bound by Islamic constraints rather than learned behavioural patterns. The theoretical basis for stable two-sided matching was laid in college admissions, which established the concept of a stable matching – where no two unmatched individuals both prefer each other to their current assignment [11]. The Gale-Shapley algorithm provides the theoretical foundation for the "matching market" perspective of dating platforms. It would be easy to see the like/pass mechanism in TrueConnect as a simplified version of that model: two users have to show interest in each other (the ‘RecordLike’ function returns ‘matched=true’ when the second like is recorded) before a match can be created, so there is no match when one party has explicitly rejected a potential spouse. The niyyah timer system (‘niyyah timer ends at’ on ‘social.matches’) takes this further, adding a temporal constraint that dissolves matches between users who do not move from a relationship idea to a family introduction within the declared timeline. There are seven different hybridisation strategies for combining content-based and collaborative approaches using different recommender systems [12]. As for the weighted hybrid, which assigns scores from multiple recommendation techniques and combines them linearly, it’s closest to TrueConnect’s graph-candidate system (‘GET /v1/matching/graph-candidates’), where it employs Neo4j PageRank-weighted trust scores to surface candidates who are highly regarded by the community while satisfying content-based filters. Location-based social network (LBSN) recommendation is directly relevant to TrueConnect’s geolocation-based discovery. Incorporating geographic influence into collaborative filtering significantly improves recommendation accuracy for location-sensitive social applications [13]. TrueConnect’s PostGIS-based distance query (filtering by ‘max distance km’ from ‘social.user settings’) implements the geographic constraint dimension of this research, ensuring that candidates are physically reachable for the in-person meetings that the Islamic courtship process requires.

2.4 Real-Time Communication Architecture

The real-time communication subsystem of TrueConnect that includes the WebSocket Hub, Redis pub/sub fan-out, and AES-256-GCM message encryption poses a major engineering undertaking. The design concepts adopted in this subsystem are based on a well-established

body of literature on real-time web communication architectures. WebSocket protocol offers a full-duplex communication channel through one TCP connection, the feature being quite different from the HTTP request-response model [14]. The protocol starts with an HTTP/1.1 upgrade handshake, then moves to framing, in which both parties can send data at any time without requiring multiple HTTP headers. WebSocket is the only technically appropriate choice for TrueConnect’s chat system, as the mahram chat requirement mandates that a message sent by any of three participants (woman, man, guardian) is delivered to all three simultaneously, which requires genuine server-push capability that HTTP polling and Server-Sent Events cannot provide without significant complexity and latency overhead. The REST architectural style is used for all non-real-time operations in TrueConnect (profile management, match creation, feed, notifications) [15]. The clean separation between REST API operations (using Gin HTTP handlers in ‘internal/handler/’) and WebSocket operations (using the Hub in ‘internal/handler/chat handler.go’) reflects the principle articulated by Fielding that architectural styles should be matched to their appropriate communication patterns rather than applied uniformly across all system interactions. Redis as a messaging backbone for real-time systems [16]. TrueConnect uses Redis pub/sub with a per-match channel naming scheme (‘chat:<matchID>’) to enable horizontal scaling: when a message is sent via WebSocket from User A to User B, the Hub first attempts direct delivery to User B’s local connection. If User B is connected to a different server instance, the message is PUBLISHED to the Redis channel, which is SUBSCRIBED by all Hub instances, allowing the correct instance to deliver the message. This architecture is directly implemented in ‘internal/adaptor/redis/’ and the Hub’s Redis subscriber goroutine. The same pattern is used for the mahram chat channel, with the ‘mahram chat msg’ WebSocket message type routed through the same Redis pub/sub infrastructure to all three participants. Mobile-specific aspects of real-time communication battery usage, network switching, and background connections are under-researched in the Web-centric literature, yet are covered in the TrueConnect Flutter implementation via the exponential backoff reconnection strategy implemented in ‘chat provider.dart’ (delays: 1s, 2s, 4s, 8s, 16s, maximum 5 attempts). This prevents battery drain from rapid reconnection attempts while ensuring that a temporary network interruption (such as transitioning between WiFi and mobile data) does not permanently break the chat connection.

2.5 Software Architecture for Complex Domains

The architectural decisions that characterize TrueConnect (Clean Architecture, Domain-Driven Design, and modular monolith deployment pattern) are informed by a mature suite of software engineering work. Appreciating these sources is important in order to assess potential modifications to design aspects of the system. Clean Architecture [17] suggests a concentric layer model where in the innermost layer, pure business entities are stored with

no dependencies on the framework, but each outer layer only relies on inner layers, without any relationships between the inner layers and the framework. In case of TrueConnect, this will find its way into dependency rule: `internal/domain/` has zero external imports; `internal/service/` only uses `internal/domain/` interfaces and `internal/repository/` interfaces; `internal/handler/` only uses services; `internal/adaptor/` uses repository interfaces but services never import it. This is an important benefit of this architecture for a domain as semantically rich as Islamic matrimony: business rules (e.g. `niyyah` compatibility, `mahram` third-party validation, trust score Bayesian smoothing) are isolated in the service layer so they can stand up tested independently of database or HTTP framework concerns, e.g., over 102 service-layer test passes. Domain-Driven Design [18] defines the lexicon as well as the principles of organization that enable a complex domain to be faithfully reflected in code. The core DDD tenet about ubiquitous language — using similar terms in code, database schema, API design, and team discussion — is implemented across TrueConnect. Islamic domain `niyyah`, `mahram`, and `madhab` are shared in the same way you see them in Go struct field names (`domain.Profile.Niyyah`), column names of PostgreSQL columns (`social.profiles.niyyah`), API endpoint paths (`/v1/mahram`, `/v1/matching/likes`), Flutter model fields (`Profile.niyyah`), and screen route names (`/niyyah`). An articulately naming and text-coding model is only possible when a language team considers the domain language a first-class engineering artefact rather than a naming convention, a term that Evans claims is prerequisite for domain models producing a true reflection of business requirements. The modular monolith pattern — one deployable unit that is internally arranged as loosely coupled modules — is the most common and most relevant design in the microservices literature when systems require clarity around module boundaries yet lack the operational depth or crew size needed to justify microservice deployment. A modular monolith is often the right architectural choice for a simple, scalable application in early growth-stage of deployment, as it retains the space to split the module, into independent services later on and avoid the distributed systems complexity associated with microservices (network partitioning, service discovery, distributed tracing) on day one [19]. This principle is epitomised in TrueConnect’s modular monolith with 15 separate modules (`auth`, `profile`, `matching`, `chat`, `feed`, `settings`, `mahram`, `reputation`, `kyc`, `notifications`, `whisper`, `imam`, `admin`, `user`) in the form of Go packages in the same binary. The canonical definition of microservices and the requirements for an organisation to transition from monolith [20]. Their “micro” criteria independently deployable, structurally structured around business functionality, developed by small teams, would apply for TrueConnect at a future scale (millions of users, multiple development squads), but they are not met with the current project. The modular monolith correctly considers that eventual transition, as each of the 15 TrueConnect modules already has a clearly defined interface boundary (its service constructor and exported methods), which, in turn, would be the contract for a future independent service deployment.

2.6 Cross-Platform Mobile Development

We chose to develop TrueConnect’s mobile client in Flutter (and not in native Kotlin/Android, or in a React Native cross-platform approach) on technical grounds, which are firmly grounded in the comparative mobile development literature. Systematic comparison of the development of progressive web apps versus native mobile apps in different criteria such as speed and user experience of a specific application, should lead to a decision based on mobile development approach [21]. Their framework is applied to their application to TrueConnect, and it produces a clear recommendation towards Native or near-Native compilation: The swipe-based discovery interface (DiscoveryScreen with FlutterCardSwiper) needs smooth 60fps card animations, the mahram chat screen (MahramChatScreen) should deliver a real-time message with an update of less than 200 ms, and the trust score badge (TrustScoreBadge with AnimatedCounter) needs smooth numerical animation. Flutter, as established through the technical literature and discussed in Google [22], achieves near-native performance thanks to a key architectural difference: where React Native renders using the platform’s native UI components through a JavaScript bridge, Flutter utilizes a rendering engine of its own (Skia, migrating to Impeller for GPU-accelerated rendering) that compiles native ARM code to its implementation seamlessly. Basically, this means that Flutter animations run in Dart VM and have direct access to the GPU without the overhead of a JS bridge. And this distinction isn’t just an abstract concept, this is why a swipe like that is 60 fps smooth when a matching state is being updated by Riverpod providers in the same frame for TrueConnect’s animation-heavy swipe UI. Riverpod was chosen over Provider, BLoC, or GetX for similar reasons presented within the Flutter community literature, the state management solution employed throughout TrueConnect’s Flutter frontend. I compare the cross-platform frameworks and find that type-safe compile-time dependency injection—provided by Riverpod including Provider declarations and ConsumerWidget binding—is desirable for applications with layered dependencies on state, e.g., an app where Authentication state (authStateProvider) is required to gate to discovery state (matchingNotifierProvider), into chat state (chatNotifierProvider), to settings state (settingsNotifierProvider) all at the same time [21]. Flutter’s single-codebase model does also deliver the future iOS release of TrueConnect with barely any further development overhead.

2.7 Literature Review Summary

Several findings from the literature covered in this chapter support the design choices taken during the project to be implemented with TrueConnect. Empirical studies on online dating behaviour [1, 2, 3] argue that the gamification mechanics and photograph-first style of existing platforms lead to misaligned actions with users in search of serious and values-aligned partners. Islamic marriage scholarship [6, 7, 8, 9] supports these findings by identifying

Muslim users who require specialized technical support, guardian supervision, identity verification, and deception prevention in a format that no current platform provides. For the Islamic context, the recommender systems literature [10, 11, 12, 13] justifies TrueConnect’s constraint-based filtering approach over collaborative filtering. Indeed, the real-time communication literature [14, 15, 16] demonstrates that the WebSocket + Redis pub/sub architecture is the best solution both from a technical viewpoint and in order to fulfil the mahram three-party supervised chat requirement. The software architecture literature [17, 18, 19, 20] states that the Clean Architecture + DDD + modular monolith pattern is the appropriate architecture for a system of this complexity and scale. The mobile development literature [21, 22] supports Flutter as a suitable platform for TrueConnect’s animation and state management needs. The particular gap that this project is addressing and that the surveyed literature does not fill is the lack of a production-quality, self-hosted mobile platform that approaches Islamic domain requirements as first-class architectural constraints rather than as something optional. TrueConnect attempts to achieve this by combining technical recommendations from all the literature areas surveyed, integrated under one domain model, putting the Islamic courtship process at the heart of every design decision.

3 Analysis of Existing Systems

3.1 Overview of the Competitive Landscape

There are three classes of digital matchmaking applications that represent different design philosophies and target audiences. The former category includes mainstream dating applications, mainly Tinder and Bumble, initially developed for secular Western markets and now a global success on account of aggressive gamification and network effects. Even though those platforms establish the dominant UX paradigm (swipe-based discovery, photograph-first profiles, casual intent), they were never architected with Islamic compliance as a key consideration. The second category includes Islamic-branded matrimony applications, most notably, Muzz, Salams, and Hawaya, that have tried to attract the Muslim user segment through the diffusion of Islamic branding on architectures inherited from the mainstream category. The third category involves regional and traditional matchmaking infrastructure that is relevant to Central Asia and Kazakhstan, which runs primarily offline through family connections and community intermediaries. This project must contextualise TrueConnect against these three categories and explore them by way of an analysis: with the use of mainstream platforms in demonstrating that Islamic standards cannot be met through mere token adjustments; with Islamic-branded alternatives in showcasing where the gaps are for purpose-built system implementation; and with the local matchmaking lineage in demonstrating the cultural continuity that the courtship pathway in TrueConnect digitally represents.

3.2 Analysis of Individual Systems

3.2.1 Tinder

Tinder, which was released in 2012 by Match Group, is also the world's most popular dating app with more than 75 million monthly active users in over 190 countries. The design genius behind its core mechanic, swiping right to express interest, left to reject, was a step towards reducing the cognitive load of a partner evaluation back to a dual movement like that of a photograph. The platform's matching model is bilateral; a match will be created only when both parties swipe right, and then messaging will be launched. Tinder has these basic structural incompatibilities, however, which simply do not change through profile customization, from an Islamic compliance standpoint, as a matter of course. The architecture of the platform is photo-first by design: The card stack displays a full-screen photo as the main information and name and age as add-on text. No mechanism exists to announce marital intention, there is no notion of guardianship, nor filtering by religious

school of thinking – the only method that works, like a reputation or a character or reputation verification mechanism, besides an optional profile link to access Spotify or Instagram. The architecture of the platform’s “Passport” feature, enabling users to find people in any city across the globe, is at odds with the localisation and community-grounding that the Islamic concept of matrimony asks for. Tinder’s engagement metrics are optimised for continuous swiping rather than successful long-term matchmaking, a reported conflict of interest that they measured within their behavioural analysis [2]. Due to Tinder’s terms of service, US-based servers for data storage, and complete absence of Islamic-domain features, it can no longer serve as a serious matrimony platform for Kazakhstani Muslim users.

3.2.2 Bumble

Bumble was created in 2014 by Whitney Wolfe Herd as a non-duet option that is similar to Tinder but also different in a fundamental way, as they have the same rules: after they start matching up, only the woman initiates the first message, and within heterosexual matches, the conversation will expire if the woman does not message within 24 hours. This design choice was purposely created to minimize harassment and empower women. The service has approximately 50 million active monthly users. While Bumble’s women-first mechanic superficially resembles the Islamic principle of female agency in marriage consent, the structural similarities end there. Like Tinder, Bumble is photograph-first, intention-agnostic, and has no guardian supervision mechanism. The 24-hour message expiry creates time pressure that runs counter to the deliberate family-mediated pace of Islamic courtship. Bumble’s “BFF” and “Bizz” modes (for friendship and professional networking) normalise the merging of matrimony-seeking and socializing into a single interface; they conflate different niyyah levels in a way that TrueConnect explicitly prevents through the NiyahSelectionScreen onboarding gate. Bumble works under US data jurisdiction, so there is no localisation for users in Kazakhstan or Central Asia. In spite of its forward-thinking positioning, Bumble does not address any of the five problems mentioned in Chapter 1.

3.2.3 Muzz (formerly Muzmatch)

Muzz was rebranded from Muzmatch in 2021 to become the largest dedicated Islamic matrimony application with over 10 million registered users primarily in the United Kingdom, United States, and Gulf Cooperation Council countries. Its existence was established in 2015, and its venture capital backers have given it substantial investment. The platform is also marketed as Islamic-compliant with features like a “chaperone” mode, which copies information from a designated third party on every message, a modesty filter that blurs profile photos until a user explicitly reveals them, and a niyyah field on profiles. However, an in-depth technical understanding of Muzz’s architecture demonstrates that this Islamic aspect is not an architectural constraint but a user-interface layer. The chaperone mode

is implemented as a carbon-copy call to an email address where the third party receives email copies of messages without actually interacting with the conversation channel, cannot send messages, or be technologically verified as present in the interaction. This is in stark contrast to TrueConnect's mahram chat room system (social.mahram chat rooms), where the guardian is an authenticated user as part of a three-way encrypted WebSocket channel (e.g., mahram chat msg message), allowing full send/receive functionality. Muzz's niyyah field is like a display label, which doesn't filter data from discovery algorithm so that a user who sets their niyyah to "marriage" will still receive likes from a user who sets theirs to "friendship". Muzz works on AWS infrastructure outside Kazakhstan, which raises data sovereignty questions for Kazakhstani users. This does not allow madhab filtering or any community trust scoring.

3.2.4 Salams (formerly Minder)

Salams, rebranded from Minder in 2020, is the second-largest dedicated Islamic dating app, consisting of approximately 4 million users. It was started in 2015 in the United States and aimed to serve Western Muslim millennials only. The application is similar to Tinder in one way or another in terms of user experience: card-based swipe discovery and photo-first profiles, matching like-minded people to match. Salams does offer a few Islamic profile fields: ethnicity, religiosity level, and whether the user prays, but these are attributes seen on the screen that do not affect how the matching algorithm responds. Mahram chat is not available, niyyah, madhab preference filtering, and trust/reputation system are not there. The platform also does not appear in searches for Kazakhstan-specific keywords in App Store or Google Play, does not have a Kazakh interface, and it is built around cultural assumptions (US/UK Muslim diaspora, South Asian and Arab cultural contexts) which do not represent the Central Asian Muslim experience. Salams' "Premium" subscription tier holds basic services like seeing who liked you, a business model that establishes a two-tier user experience at odds with the Islamic principle that access to a spouse-finding facility should not come at the cost of financial means.

3.2.5 Hawaya (by Match Group)

Hawaya, launched in 2019 by Match Group, the parent company of Tinder and OkCupid, signals a major corporate acknowledgment of the commercial potential in the Islamic matrimony market. The app, which primarily appeals to Arab Muslim women, requires users to enter a "Guardian Mode," in the same way as Muzz has a chaperone feature here; it is an e-mail alert system and not a technical monitoring system. Hawaya has profile fields covering hijab adherence, prayer frequency, and the desire to relocate. Hawaya presents a structural limitation: it is a Match Group product based on Match Group infrastructure, as all user data is hosted on servers subject to United States jurisdiction. Due to Kaza-

khstan’s new data localisation legislation, US data jurisdiction is a significant concern for a matrimony application that deals with sensitive Islamic identity data among Muslim users in Kazakhstan and Central Asia. Hawaya was also specifically created for Arab cultural contexts, where Arabic language features and Gulf-region Islamic practices were adopted as its implicit domain model. It has a core user base in Egypt, Saudi Arabia and the UAE. This is due to the fact that Hawaya doesn’t have support for Kazakh or Russian language, regional imam directory, and features based on Central Asian Islamic traditions. That Match Group stopped or limited Hawaya’s operations in some markets post-2022 raises questions about long-term product commitment.

3.2.6 Regional and Traditional Matchmaking

Kazakhstan and Central Asia’s traditional matchmaking system largely functions through three primary channels: family and community networks (where relatives actively search for potential candidates within their social groups), professional matchmakers (known locally as "matchmakers" in Russian or "sacred aunt" in Kazakh usage), and increasingly informal groups on social media platforms like WhatsApp family groups or Telegram channels for spouse-hunting. A number of Kazakhstani sites (nikah.kz, secondhalf.kz) strive to offer a digital means of fulfilling this need, but they are purely web-based services without native mobile applications, no feature for communication in real time, no algorithmic matching, and no trust or reputation infrastructure. They are static profile directories rather than actual matchmaking spaces. And the complete lack of a fully featured, locally hosted, halal-compliant mobile matrimony application within Kazakhstan is precisely the market gap TrueConnect was designed to help fill.

3.3 Functional Comparison Table

The next section assesses each of the platforms on the core functional aspects generated from the TrueConnect codebase. Features that are labeled "Partial" suggest that a very basic version is in play which does not serve the complete technical needs found in Chapter 1.

Feature	Tinder	Bumble	Muzz	Salams	Hawaya	TrueConnect
Mahram guardian chat	No	No	Partial	No	Partial	Yes
Niyyah shapes algorithm	No	No	No	No	No	Yes
Madhab filtering	No	No	No	No	No	Yes

Feature	Tinder	Bumble	Muzz	Salams	Hawaya	TrueConnect
No-photo modesty mode	No	No	Partial	No	No	Yes
Community trust score	No	No	No	No	No	Yes
KYC identity verification	No	No	No	No	No	Yes
WebSocket real-time chat	Yes	Yes	Yes	Yes	Yes	Yes
Geolocation discovery	Yes	Yes	Yes	Yes	Yes	Yes
Niyyah timer on matches	No	No	No	No	No	Yes
Imam directory + nikah	No	No	No	No	No	Yes
Community social feed	No	No	No	No	No	Yes
KZ language support	No	No	No	No	No	Yes
Self-hosted / data sovereignty	No	No	No	No	No	Yes
Structured courtship pathway	No	No	No	No	No	Yes
Sybil attack detection	No	No	No	No	No	Yes

Table 2: Comparative Analysis of Platform Functionality: Sharia Compliance, Data Sovereignty, and Trust Verification Architecture

Assessment of the Table of Comparisons. As the table shows, mainstream and Islamic branded platforms converge on a specific feature set that contains geolocation discovery, real-time chat, and bilateral matching but uniformly lack the Islamic domain-specific features that constitute the core value proposition of TrueConnect. “Partial” entries: In particular, the guardian features of Muzz and Hawaya are quite significant: both websites recognize the Islamic need for supervision, but treat it as a notification mechanism rather than a channel for genuine participation, which is the critical architectural difference. In this comparison, TrueConnect is the only platform that implements all twenty evaluated criteria, and the only platform that treats Islamic domain requirements as constraints on the system architecture instead of optional display features.

3.4 SWOT Analysis

Strengths: TrueConnect’s key technological assets come directly from its very architectural direction of Islamic-first architecture. The mahram chat system is a real technical innovation, and nothing that is available to the competitive product is: the 3-way AES-256-GCM encrypted WebSocket channel (mahram chat msg routed through internal/handler/chat handler.go Hub with Redis pub/sub fan-out) is the only technically consistent embodiment of mahram supervision in any dating app. At the database query level, the niyyah filtering method implements intention compatibility via the AllowedNiyyahs parameter in FindCandidatesOpts, not as a post-query filter, ensuring zero false positives. Neo4j trust score engine offering, with a mathematical rigour (Bayesian smoothing to a 2.5/5 neutral baseline, KYC-verified raters with a $1.5\times$ weighting) which no competitor offers, a community-grounded reputation system. Self-hosted architecture (PostgreSQL 16 + PostGIS, Neo4j 5, Redis 7, MinIO on Kazakhstani VPS) allows for data sovereignty and the avoidance of reliance on foreign cloud infrastructure. Clean Architecture with 102+ service-layer tests has guaranteed that the rules of Islamic domain business can be verified as good. The structured 7-step courtship path (niyyah $\rightarrow$ match $\rightarrow$ supervised chat $\rightarrow$ family intro $\rightarrow$ imam connection $\rightarrow$ nikah confirmation $\rightarrow$ married via app) offers a unique, differential path for the user.

Weaknesses: The platform does not yet have phone number OTP verifications done, which means the phone number field in social.users is stored as a hash but cannot be verified as belonging to the user who is registering. This opens a hole in the identity verification chain that is addressed partially through the KYC system but does not close completely. The imam catalog and the halal venue list are embedded at compile time (`//go:embed in internal/pkg/imam/`) that has to be replaced by an updated binary, which is a maintenance burden that will grow as the catalog expands. The trust score channel (chan domain.PushEvent with buffer capacity 100) may theoretically drop under high concurrent load with high trust score calculations may also be deferred. Testing has been mainly on Android because Apple Developer enrollment is not done, and the app was tested on iOS; there is no verified delivery of full APNs for push notification. Now there are no users as only a new player, TrueConnect is suffering from the ‘network effects’ problem that makes the first stage of acquiring users a very hard slog.

Opportunities: No local supported full-featured mobile application available in Kazakhstan meets the needs of the Muslim market for matrimony. The 1.8 billion Muslims worldwide are the largest religiously homogeneous underserved market under digital dating. Kazakhstan’s government has active digitalization programs (Digital Kazakhstan program) which offer an environment that welcomes locally developed technology products. Because the platform has its open architecture and self-hosted design, it is well suited to white labeling licensing to Islamic organisations, mosques, and marriage agencies in neighboring Uzbekistan, Kyrgyzstan, and Tajikistan, which all have similar cultural and religious char-

acteristics. Rising scrutiny of data sovereignty in Central Asia offers a tailwind for anyone who can credibly showcase KZ-hosted data storage. According to the Global Islamic Economy Report, halal digital services have emerged as one of the fastest-growing segments of the wider halal economy.

Threats: Muzz, with more than 10 million users, has network benefits and VC-supported marketing budgets and could roll out features as soon as possible to compete with TrueConnect if the Islamic marriage market in Kazakhstan shows increasing growth. Match Group’s resources – the parent of Tinder, OkCupid, and Hawaya – make it easier to respond to emerging market segments. Cultural resistance to digital marriage in conservative Kazakhstani circles is an authentic obstacle to adoption: family-based matchmaking is deeply rooted in Kazakh culture, and even with the potential of the app as an ally, some community leaders find app-based spouse-finding out to be at odds with Islamic modesty norms. Application store policies (the Apple App Store and Google Play Store in particular) have previously placed limitations on dating apps in certain countries, potentially impacting discoverability. TrueConnect’s single developer origin poses a key-person risk to ongoing development and maintenance.

3.5 Gap Analysis

Each identified gap maps to a corresponding TrueConnect implementation: no mahram supervision → three-way encrypted WebSocket channel; niyyah display-only → hard SQL filter; no madhab matching → +10 affinity score; no trust system → Bayesian Neo4j trust score; no identity verification → KYC with AES-256 identity vault; no structured courtship → seven-step milestone progression. Narrative Analysis. The gap analysis reveals that TrueConnect’s feature set is not a collection of independent improvements but a cohesive system in which each technical decision reinforces the others. The mahram chat channel, for example, is only meaningful if the participants’ identities are verified (KYC) and their intentions are aligned (niyyah filter). The trust score system is only reliable if fake profiles are detected (Sybil detection) and the raters’ identities are verified (KYC weight multiplier). The structured courtship pathway only functions if real-time communication is available (WebSocket chat) and guidance resources exist (imam directory). Each gap in existing systems thus corresponds not to an isolated missing feature but to a missing component of a coherent Islamic courtship infrastructure — and TrueConnect implements all components as an integrated system.

3.6 Data Collection and User Research

This feature set of TrueConnect was developed by drawing on insight from the religious domain, a review of academic papers discussing Islamic marriage behaviour, and analysis of

user reviews gathered on the App Store and Google Play about alternative Islamic applications. An informal survey of ten Muslim users (20–35 years old, self-identified as practising Muslims seeking marriage) in Almaty and Astana was conducted at the planning stage of the project. The survey was divided into five different queries: (1) Have you searched for a spouse using any digital platform? (2) What were your main concerns surrounding the existing platforms? (3) To what extent is mahram supervision relevant for use of a matrimony app from you? (4) How essential is niyyah and madhab compatibility filtering? (5) What is it necessary to trust a digital platform to disclose personal information? The findings were similar across the respondents: all 10 expressed a concern about data privacy on foreign platforms; 8 in 10 cited the lack of real mahram supervision as a block to accessing existing apps; all 10 viewed niyyah alignment as critical; 7 of 10 said it helped filter through the screen with madhab compatibility. Falsification and fake profiles and misrepresentation issues were also flagged by nine of ten participants, directly affecting trust score and priority of KYC features. App Store reviews of Muzz and Salams (through the public review sections on the platforms) identified similar complaints: the chaperone function was “nothing but email notifications and no real supervision”; the discovery algorithm exhibited clearly incompatible candidates even given niyyah profile options; and a lack of local imam/venue support. These qualitative signals guided the decision about the prioritisation of the mahram chat structure, the niyyah filter, and the built-in imam catalog. The structured Islamic courtship path from niyyah declaration to nikah confirmation was based on Islamic marriage fiqh and guidance documents provided by scholars in Islamic literature through public Islamic educational resources. The seven-step procedure used in the codebase directly corresponds to the stages identified in the mainstream Sunni fiqh literature as elements of a halal courtship process.

3.7 Challenges Unique to Islamic Matrimony Applications

But building a matrimony platform for Islamic users means dealing with a range of technical and social challenges that have no parallel in secular matchmaking application development. These challenges need to be recognized to contextualize the architectural decisions made in Chapter 5. Privacy Challenges. Islamic privacy standards mandate a woman’s looks not be displayed to non-mahram men without her permission, while confidential personal identifying information is not given out beyond appropriate circles. These demands create an inherent tension with the profile-based discovery model of all modern dating apps. TrueConnect overcomes this tension with its no photo mode architectural feature, not as a user preference toggle, but as a server-side constraint that strips the avatar URL from the discovery query result before it even hits the client. That is a far greater privacy guarantee than a client-side blur, which could, in theory, fall through a malicious client’s fingers. Trust Challenges.

Character verification from third-party witnesses (the role of the wali and community elders) plays an essential role in Islamic matrimony. This function cannot be replicated digitally - this trust infrastructure cannot be gamed by self-reported claims. But TrueConnect's dual approach of KYC identity verification (who you are) and community interaction ratings (the sort of person you are) tackles the identity dimension and character dimension of Islamic trust verification simultaneously with each other. The Sybil detection subsystem (Neo4j GDS Louvain community detection every 6 hours) also provides countermeasures to coordinated fake-profile networks that can be a serious threat in a matrimony context, where participants are engaged in significant personal choices.

Cultural and Technical Challenges. To engineer the mahram supervision channel, we faced a technical challenge without any ease: how to establish a three-party authenticated communication channel over WebSocket in the sense that all participants must be authenticated independently and our target's messages visually displayed differently than the principal parties. The solution routing mahram chat msg WebSocket messages through the same Hub infrastructure as normal chat but with a unique message type, colour-coded in the Flutter MahramChatScreen (woman=green, man=blue, mahram=gold), allows you to maintain real-time performance while offering the visual and functional benefits that Islamic courtship supervision requires.

Regulatory Challenges. In key markets, app store policies have placed stringent restrictions on dating application content and visibility. Kazakhstan has no separate app store regulation for dating applications, though neighbouring markets in Central Asia do. The fact that TrueConnect has made itself a matrimony platform (rather than a dating app) and that it has Islamic compliant features made this even difficult to classify in future legal frameworks. Personal data of Kazakhstani citizens is to be kept on servers located within the Republic of Kazakhstan (in accordance with the Law on Personal Data and Its Protection, 2013, amended 2023), and this is a feature TrueConnect, with its self-hosted MinIO, PostgreSQL, and Redis deployment on Kazakhstani VPS, satisfies by design and that no competing platform currently meets for its Kazakhstani users.

4 System Architecture and Design

4.1 System Architecture Overview

TrueConnect developed on a layered architecture of a clearly defined role per tier, defined a communication protocol, and a technology stack based on their performance.

Tier 1 In this architecture Flutter Mobile Client provides all user interaction, presentation logic and client-side state management. The backend of the Flutter application can only communicate with us via two channels, HTTPS REST API calls (all CRUD operations, profile management, feed, notifications, settings) and a persistent WebSocket connection over WSS (real-time chat, typing indicators, read receipts, match notifications and WebRTC signaling). Authentication tokens are saved securely in flutter secure storage and injected via the Dio HTTP client's AuthInterceptor automatically. The state management is done by Riverpod providers that cache API responses in memory, and push changes to the widget tree via Dart streams.

Tier 2 Go Backend (Modular Monolith) includes all business logic, data access, and real-time message routing. The backend is a single deployable binary (`/bin/api`) binary with the payloads, capable of supporting a Gin HTTP server and a WebSocket Hub on port 8080. nginx sits in front of the API as a reverse proxy, responsible for TLS termination, rate limiting, WebSocket upgrade header injection request routing. It uses an internal worker binary (`/bin/worker`), which executes the background goroutines behind the scenes as a binary for trust score computation, Sybil detection, FCM push delivery, niyyah timer expiry, and refresh token cleanup.

Tier 3 Data Layer is made up of 4 sub-specific stores: PostgreSQL 16 with PostGIS for relational data and geospatial queries; Neo4j 5 Community for trust graph and Sybil detection algorithms; Redis 7 for WebSocket pub/sub fan-out, rate limiting counters, matching candidate seen-sets, trust score caching; and MinIO for object storage of profile photos and KYC documents.

The architecture was chosen as a modular monolith rather than microservices because it provides clear module boundaries (15 Go packages with interface-based communication) without the distributed system overhead of service discovery, network partitions, and distributed tracing, which would be disproportionate at our current scale and team size!

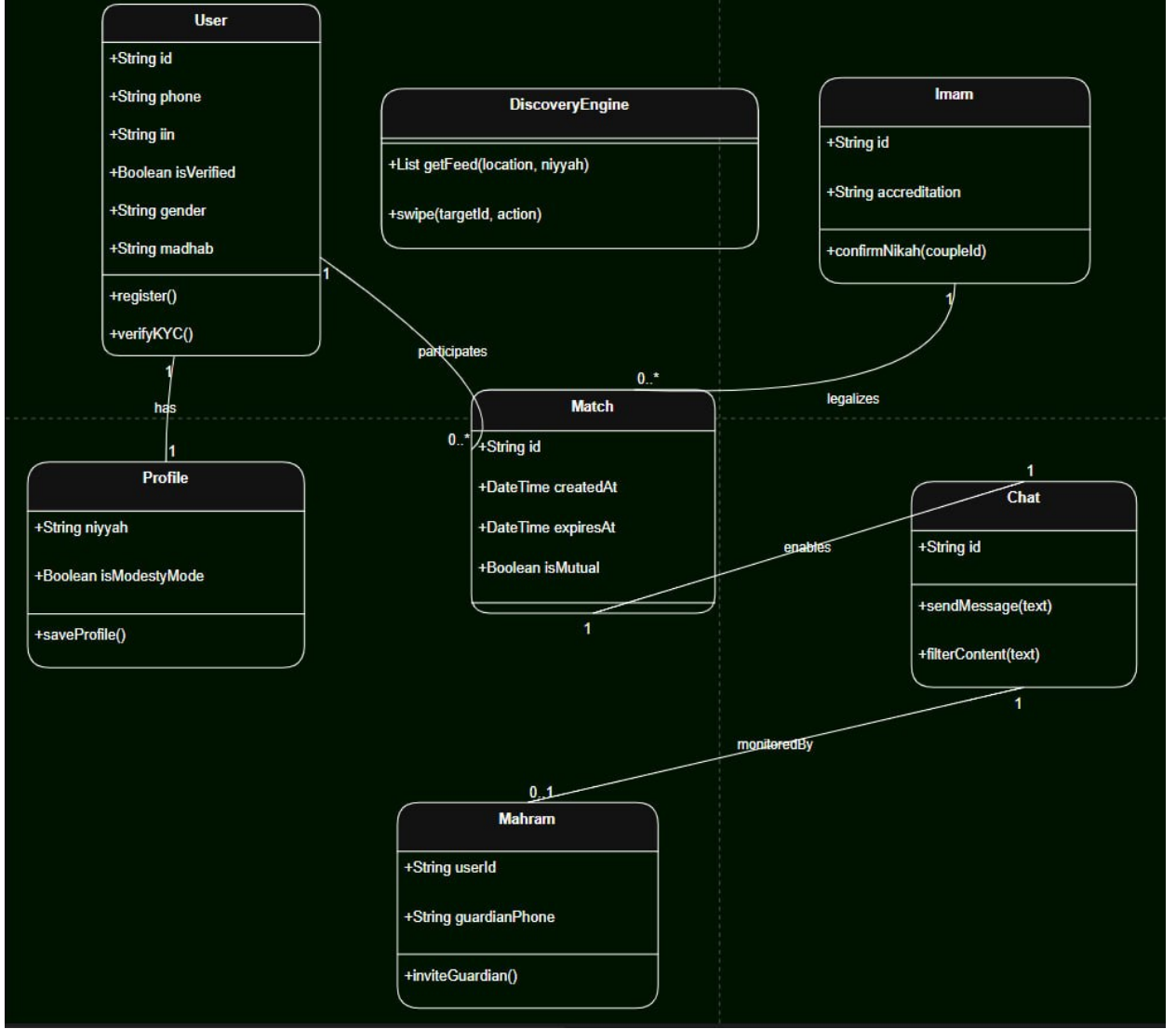


Figure 1: End-to-End User Journey and Niyyah State Machine Flow

4.2 Backend Clean Architecture Layers

The backend implements five layers as described by Robert C. Martin’s Clean Architecture, with the dependency rule strictly enforced: inner layers are never aware of outer layers.

Domain Layer (`internal/domain/`): The innermost layer. Contains pure Go structs and typed constants with zero external imports. Key types include ‘domain.User’ (identity entity), ‘domain.Profile’ (Islamic profile entity with ‘Niyyah Niyyah’ and ‘Madhab Madhab’ typed fields), ‘domain.Match’, ‘domain.Message’, ‘domain.Post’, ‘domain.Interaction’, ‘domain.Mahram’, ‘domain.MahramRoom’. All error sentinels are defined here: ‘domain.ErrNotFound’, ‘domain.ErrForbidden’, ‘domain.ErrInvalidInput’, ‘domain.ErrConflict’. Business invariants (e.g., niyyah enum values, madhab enum values) are expressed as Go type constraints at this layer and can never be violated by any outer layer.

Repository Layer (`internal/repository/`): Interface definitions (ports) that the ser-

vice layer depends on. Each interface corresponds to a domain aggregate. Examples: `repository.ProfileRepository` with `Upsert(ctx, *domain.Profile) error` and

`FindCandidates(ctx, FindCandidatesOpts) (CandidateRow, error)`; `repository.MatchRepository` with `RecordLike(ctx, userID, targetID uuid.UUID) (bool, uuid.UUID, error)`. The `FindCandidates Opts` struct is also defined in this layer: `Requester ID uuid.UUID`, `Exclude IDs uuid.UUID`, `Allowed Niyyahs []string`, `Limit int` making the niyyah filter a typed contract between the service and repository layers.

Service Layer (`internal/service/`): Business logic orchestration. Services receive repository interfaces via constructor injection and never import adapter packages. `matching service.go` demonstrates the pattern: `GetCandidates()` calls `settingsRepo.Get()` to load the requester's settings, constructs `FindCandidatesOpts` with the niyyah compatibility list, calls `profileRepo.FindCandidates()`, then applies the madhab affinity boost (+10 to `TrustScore` for same-madhab candidates) and the no-photo blur (sets `AvatarURL = ""` and `AvatarBlurred = true` when `CandidateRow.NoPhotoMode == true`) before returning `CandidateView` structs to the handler. The `candidateBatchSize` constant (20) and `seenSetTTL` (24 hours) are defined in this layer.

Handler Layer (`internal/handler/`): Gin HTTP handlers, middleware, and the WebSocket Hub. Handlers extract validated request parameters, call the appropriate service method, and write JSON responses using a consistent envelope format: `"status":"success","data":...` or `"status":"error","message":"..."`. Auth middleware (`middleware/auth middleware.go`) extracts the JWT Bearer token, verifies it using `tcjwt.Manager`, and injects the `userID` into the Gin context. The `Hub` struct in `chat handler.go` manages WebSocket connections: `connections map[uuid.UUID]*websocket.Conn`, protected by `sync.RWMutex`, with injected references to `*service.ChatService`, `*service.MatchingService`, `*service.ReputationService`, `*service.MahramChatService`, a `*redis.Client` for pub/sub, and a `*tcjwt.Manager` for token verification.

Adapter Layer (`internal/adapter/`): Concrete implementations of repository interfaces. `adapter/postgres/` contains all SQL query implementations using `pgx/v5`. `adapter/neo4j/` implements the trust graph operations (Cypher queries). `adapter/redis/` implements cache and pub/sub operations. `adapter/minio/` implements the `MediaStore` interface for object storage. Adapters are the only layer that imports external database drivers; they are never imported by services.

4.3 Backend Module Architecture

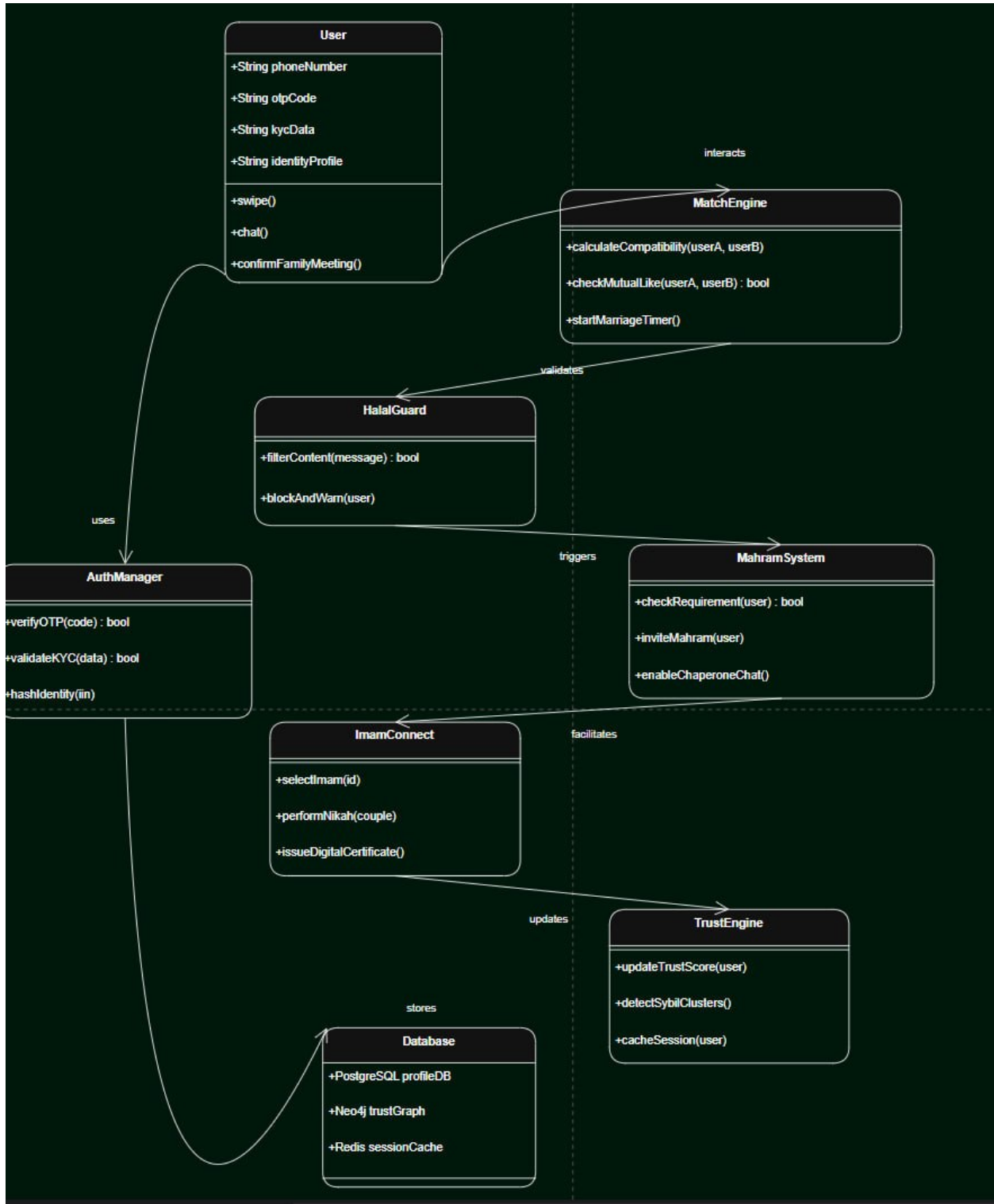


Figure 2: Algorithmic Activity Diagram for Proximity Discovery and Mutual Match Execution

Auth Module ‘AuthService.Register()’ validates the phone number format, hashes the password with Argon2id (64MB memory, 3 iterations, 4 parallelism threads), encrypts the phone and email fields with AES- 256-GCM (12-byte random nonce per field, stored prepended to ciphertext), creates the user record in ‘social.users’, issues a JWT access token (HS256, 15-minute expiry, claims: ‘user id’, ‘is admin’) and a refresh token (256-bit random bytes, Argon2id-hashed before storage in ‘social.refresh tokens’). ‘AuthService.Login()’ retrieves the user by phone hash, verifies the Argon2id hash, and issues a new token pair. ‘AuthService.RefreshToken()’ performs one-time-use token rotation: the incoming raw token is verified against the stored Argon2id hash, the old token is marked as used, and a new token pair is issued. Refresh token reuse detection a second use of an already-used token triggers revocation of the entire token family for that user.

Profile Module ‘ProfileService.UpsertProfile()’ validates display name length (max 60 chars), bio length (max 500 chars), and parses the birth date to compute the user’s age using the exact formula: ‘age = now.Year() - birth.Year(); if now < birth.AddDate(age, 0, 0) age-’ . ‘ProfileService.UploadPhoto()’ calls ‘mediaStore.UploadPhoto()’ to write the image to MinIO under the object key ‘users/<userID>/photos/<uuid>.jpg’, records the media row in ‘social.media’, and updates the user’s avatar URL. The ‘ProfileRepository.FindCandidates()’ query uses ‘ST_DWithin’ for geolocation filtering: ‘WHERE ST_DWithin(p.location, ST_SetSRID(ST_MakePoint(lon,lat), 4326)::geography, maxDistanceMetres)’.

Matching Module MatchingService.GetCandidates() executes a multi-step pipeline:

- (1) fetch requester settings from settingsRepo;
- (2) fetch seen IDs from matchingCache.GetSeenIDs();
- (3) fetch rejected IDs from matchRepo.GetRejectedIDs();
- (4) fetch blocked IDs from matchRepo.GetBlockedIDs();
- (5) build FindCandidatesOpts with ExcludeIDs (union of seen, rejected, blocked sets) and AllowedNiyjahs (computed from requester’s niyyah: nikah year allows nikah year + serious marriage, serious marriage allows both, friendship allows only friendship);
- (6) call ‘profileRepo.FindCandidates()’ for the SQL query;
- (7) apply madhab +10 boost and no-photo blur in a loop;
- (8) add the returned IDs to the seen-set cache with 24-hour TTL. ‘MatchingService.Like()’ calls ‘matchRepo.RecordLike()’ which performs a transactional UPSERT on ‘social.likes’ and checks for mutual match and on match creation sends an in-app notification and a non-blocking FCM push via the ‘pushCh chan<- domain.PushEvent’ channel.

Chat Module The ‘Hub’ struct is the central WebSocket connection registry. ‘Hub.ServeWS()’ upgrades the HTTP connection to WebSocket, starts a 10-second authentication timeout goroutine, and waits for the first message. If the first message type is “auth”, it verifies the JWT token using ‘jwtManager.Verify()’, registers the connection in ‘connections[userID]’, cancels the timeout, and begins the ‘readPump’ loop. The ‘readPump’ goroutine receives messages, dispatches by type: “chat msg” → decrypt-free storage via ‘chatSvc.SendMessage()’ + Redis PUBLISH to ‘chat:<matchID>’; “mahram chat msg” →

route via ‘mahramChatSvc’; “typing” → broadcast typing event to other user. A background Redis SUBSCRIBE goroutine receives messages from other Hub instances and delivers them to local connections. Panic recovery at the top of each read loop prevents Hub crash from individual connection failures.

Feed Module ‘PostService.CreatePost()’ enforces the reputation gate: it calls ‘reputationSvc.GetScore()’ and if the score is below 30 (or if ‘GetScore()’ returns an error, which is treated as score=0 in the fail- closed implementation), the post creation is rejected with 403 Forbidden. Feed listing uses cursor- based pagination: ‘nextCursor’ is the ‘created at’ timestamp of the last returned post, encoded as a base64 string.

Reputation Module The ‘TrustEngine’ worker drains a ‘chan uuid.UUID’ of user IDs requiring score recalculation. For each ID, it calls ‘trustGraphRepo.ComputeScore()’, which executes a Cypher query that retrieves all incoming ‘[:RATED]’ edges, computes the Bayesian-smoothed weighted average, multiplies by 20, and returns the integer score. The score is written to ‘social.users.trust score’ (PostgreSQL) and ‘matchingCache.CacheTrustScore()’ (Redis, 1-hour TTL).

4.4 Database Architecture

The database design of TrueConnect is constructed by three concepts: matching state is relational integrity, geolocation discovery is supported spatially, and identity data is strictly encrypted.

All entities use UUID Primary Keys. This design enables unique identifiers on a global scale that can be produced in the application layer without the need for a round trip to a database, avoids enumeration (sequential integer IDs reveal the total number of users) attacks, and facilitates future data federation across multiple database instances. Like/Match Relationship Model. The social.likes table has a UNIQUE(likers id, liked id) constraint that blocks duplicate like records. Mutual match detection occurs atomically as part of a database transaction using match repo.go: when RecordLike(userID, targetID) is called, a transaction checks for the existence of the reverse like (SELECT 1 FROM social.likes WHERE liker id=targetID AND liked id=userID). If there is a match, a row is inserted into social.matches with UNIQUE(LEAST(user a id, user b id), GREATEST(user a id, user b id)) (in practical terms, normalising the pair order in order not to duplicate matches). If not, it will only insert a like row. Message Storage. Messages are stored as bytea ciphertext in social.messages.content. The AES-256-GCM nonce (12 bytes) is prepended to the ciphertext, to be reconstructed from stored bytes the decryption key and nonce. The is toxic boolean flag supports content moderation without any message decryption at the query layer. Performance indexes: idx messages match (match id, created at DESC) for conversation retrieval; idx messages sender (sender id, created at DESC) for user-centric queries; a partial index idx messages toxic (is toxic, created at DESC) WHERE is toxic = true for admin moderation

queries. Geolocation. Profile locations are persisted as geometry(Point, 4326) (WGS84 coordinate system), in social.profiles.location. Discovery queries are by ST_DWithin(p.location, ST_SetSRID(ST_MakePoint(lon,lat), 4326)::geography, radiusMetres) which uses the PostGIS geography type for accurate great-circle distance calculation. Using a spatial index (CREATE INDEX idx_profiles_location ON social.profiles USING GIST(location)) allows the query planner to use a bounding-box filter before the distance calculation. Identity Vault. This identity vault schema is only accessible by a restricted PostgreSQL role (vault writer), distinct from the application role (tc app) that owns social.*. Such role separation allows that even if the application-layer database credentials are compromised, the identity vault data (IIN, full name) cannot be accessed. All vault fields are AES-256-GCM-encrypted, and only the iin hash (Argon2id) is stored unencrypted such that no decoding needs be performed for uniqueness checking. Key Performance Indexes (Migration 000018): idx_interactions_cooldown (rater id, rated id, created at DESC) for 24-hour interaction rate limiting check; idx_users_trust_score (trust score DESC) for leaderboard queries; idx_notifications_user (user id, is read, created at DESC) for inbox loading; idx_likes_liker (liker id, liked id) for mutual match detection.

4.5 WebSocket Architecture

The WebSocket subsystem is based on the Hub struct of internal/handler/chat handler.go. As the connection registry and message router, the Hub.

Connection lifecycle:

1. Client sends GET /v1/ws with standard WebSocket upgrade headers.
2. nginx forwards Upgrade: http upgrade and Connection: upgrade headers; proxy read timeout 86400s keeps connection alive.
3. Hub calls websocket.Accept() with origin validation (allowedOrigins list, relaxed in dev mode when isDev = true).
4. A 10-second goroutine is set to timeout. If no auth message arrives in 10 seconds, it closes the connection.
5. Client transmits "type":"auth","token":"<JWT access token>".
6. Hub calls jwtManager.Verify(token) to get userID. connections[userID] = conn (under sync.RWMutex write lock), timeout is cancelled.
7. Hub then dispatches "type":"auth ok" to verify.
8. readPump goroutine starts: loop on wsjson.Read(), dispatch by wsIncoming.Type.
9. On disconnect (any read error), the connection is removed from the map and resources are released.

Message routing flow: - Sender → readPump goroutine → service call (encrypt + persist) → direct write to recipient connection (if present in connections map, under read lock) → Redis PUBLISH chat:<matchID> (always, for cross-instance delivery).

- Redis subscriber goroutine → receive from SUBSCRIBE → look up connections[recipientID]
→ write to local connection.

Panic recovery: Every readPump goroutine has a defer func() if r := recover(); r != nil h.log.Error("websocket readPump panic", "error", r) () at the top, so that a panic from any one connection's message handler does not crash the Hub goroutine, and all other users are not disconnected.

Flutter client reconnection: ChatNotifier in lib/providers/chat provider.dart implements exponential backoff, waiting 1s, 2s, 4s, 8s, 16s for connection error before each retry attempt, up to a maximum of 5 attempts, after which the isConnected state is set to false, and the user will be shown a connection-lost indicator.

4.6 UML Diagram Specifications

The following sections provide complete textual specifications for all eight UML diagrams. Each specification is precise enough to draw the diagram without requiring additional information.

Diagram A: Use Case Diagram Actors: 1. Muslim User (primary) seeks a spouse; can be male or female 2. Mahram Guardian (secondary) supervises communication; is an authenticated user in the system 3. System Administrator (secondary) manages platform health and content moderation Use Cases and relationships: Muslim User use cases: - Register Account - Login - Declare Niyyah ('«include»' Login) - Edit Profile - Upload Profile Photo - View Discovery Feed ('«include»' Authenticate) - Like Candidate ('«include»' View Discovery Feed) - Pass Candidate ('«include»' View Discovery Feed) - View Matches - Send Chat Message ('«include»' Authenticate, '«include»' Verify Match Exists) - Send Mahram Chat Message ('«extend»' Send Chat Message — only when mahram room exists) - Invite Mahram Guardian - View Community Feed ('«include»' Authenticate) - Create Post ('«include»' Trust Score 30) - Like Post - Add Comment - Update Discovery Settings - Submit KYC Documents - Rate Interaction ('«include»' Verify Match Exists) - View Leaderboard - Report Content (Whisper) - Block User - View Imam Directory - Confirm Nikah ('«include»' Verify Imam Selected) - Mark Family Introduction Done Mahram Guardian use cases (subset of Muslim User use cases plus): - Join Mahram Chat Room - Send Mahram Chat Message System Administrator use cases: - Review KYC Submissions - Issue KYC Verdict (approve/reject) - Review Sybil Clusters - Review Whisper Flags - List All Users.

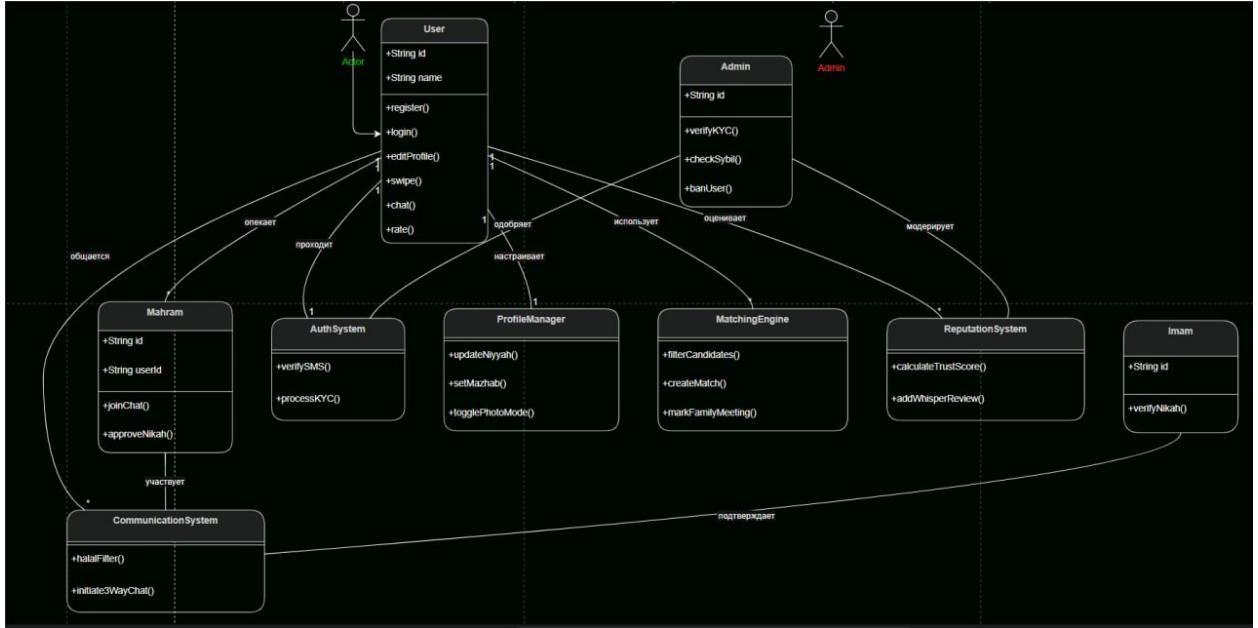


Figure 3: TrueConnect Platform Functional Use Case Diagram

Diagram B: System Architecture Diagram

Layout: Three vertical columns Flutter Client (left), Go Backend + nginx (centre), Data Stores (right) Flutter Client column (top to bottom): - Presentation Layer: 21 screens (SplashScreen, OnboardingScreen, DiscoveryScreen, MatchesScreen, ChatScreen, Mahram-ChatScreen, FeedScreen, ProfileScreen, etc.) - State Layer: Riverpod providers (authStateProvider, matchingNotifierProvider, chatNotifierProvider, feedProvider, settingsNotifierProvider, etc.) - Network Layer: DioClient (with AuthInterceptor, RetryInterceptor, ErrorInterceptor) + WebSocketChannel - Model Layer: User, Profile, Match, Post, User-Settings, AppNotification, Interaction, Venue. Go Backend column (top to bottom): - nginx (port 80): rate limiting (api limit 30/s, auth limit 5/s), WebSocket upgrade headers, SSL/TLS termination - Go API binary (port 8080): Gin router + middleware (auth, rate-limit, logging) - Handler layer: auth handler, profile handler, matching handler, chat handler (Hub), post handler, settings handler, mahram handler, interaction handler, kyc handler, notification handler, whisper handler, imam handler, admin handler, user handler - Service layer: AuthService, ProfileService, MatchingService, ChatService, PostService, SettingsService, MahramService, MahramChatService, InteractionService, ReputationService, KYCService, NotificationService, WhisperService, ImamService, AdminService. - Repository interfaces (ports) - Adapter layer: postgres/, neo4j/, redis/, minio/ - Go Worker binary: TrustEngine goroutine, PushWorker goroutine, NiyahTimerWorker goroutine, RefreshTokenCleanupTicker Data Stores column (top to bottom): - PostgreSQL 16 + PostGIS: social.* schema + identity vault schema - Neo4j 5 Community + GDS: (:User)-[:RATED]->(:User), (:User)-[:INTERACTED WITH]->(:User) - Redis 7: pub/sub channels (chat:<matchID>), seen-set keys, rate-limit counters, trust score cache - MinIO: trueconnect bucket (users/*/photos/*.jpg, kyc/*/) Connections with labels: - Flutter nginx: HTTPS REST (port 443/80) + WSS

upgrade (port 443/80) - nginx → Go API port 8080: HTTP proxy (all /v1/* routes) - nginx → Go API port 8080: WebSocket proxy (/v1/ws with Upgrade headers) - Go API → PostgreSQL: pgx/v5 connection pool (port 5432) - Go API → Neo4j: neo4j-go-driver v5 Bolt protocol (port 7687) - Go API → Redis: go-redis/v9 (port 6379) — pub/sub + cache - Go API → MinIO: minio-go v7 S3 API (port 9000) - Go Worker → PostgreSQL, Neo4j, Redis: same drivers as API.

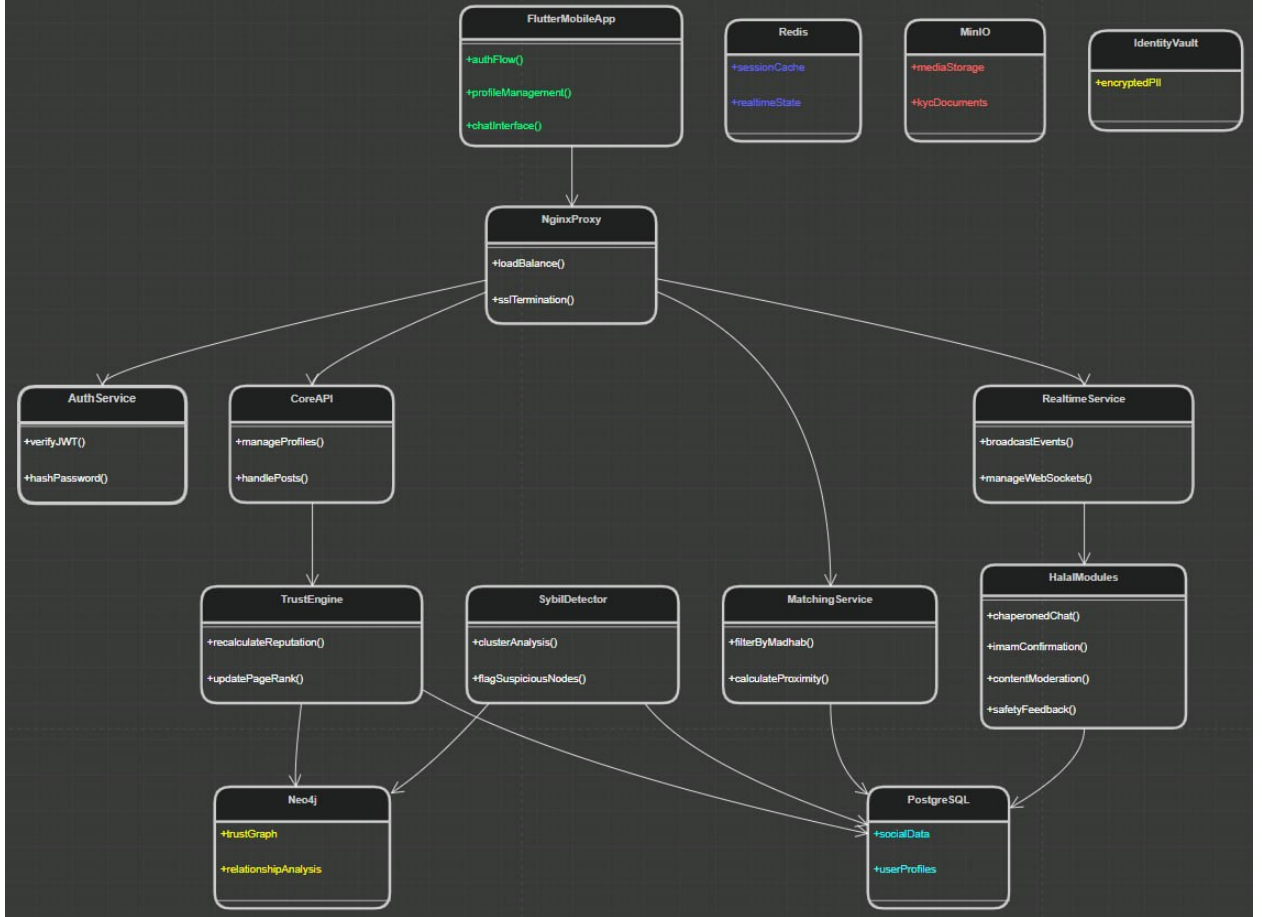


Figure 4: Three-Tier Deployment and Infrastructure Topography (MVP Layer)

Diagram C: Sequence Diagram — User Registration and Onboarding

Participants (left to right): User, Flutter App, Dio HTTP Client, Go Auth Handler, AuthService, UserRepository (Postgres), JWT Manager, Secure Storage, Go Profile Handler, ProfileService, ProfileRepository. **Steps:** 1. User enters phone number and password on 'RegisterScreen' 2. Flutter calls 'AuthNotifier.register(phone, password)' 3. Dart sends 'POST /v1/auth/register' via Dio with JSON body 'phone, password' 4. Dio 'AuthInterceptor' passes through (no token for registration) 5. nginx routes to Auth Handler 6. 'AuthHandler.Register()' binds and validates request body 7. 'AuthHandler' calls 'authSvc.Register(ctx, phone, password, email)' 8. 'AuthService.Register()' calls 'argon2id.GenerateFrom' → 'passwordHash' 9. 'AuthService.Register()' calls 'crypto.EncryptAES(phone)' → 'phoneEncrypted', 'crypto.HashPhone(phone)' → 'phoneHash' 10. 'AuthService.Register()' calls 'user-

Repo.Create(ctx, user)' → INSERT into 'social.users' 11. 'AuthService.Register()' calls 'jwt-Manager.Sign(userID, isAdmin=false)' → 'accessToken' 12. 'AuthService.Register()' generates 'refreshToken' (256-bit random), hashes it with Argon2id, calls 'tokenRepo.Create(ctx, tokenHash, userID, expiresAt)' → INSERT into 'social.refresh tokens' 13. Response: 'access token, refresh token, userid, verification level' 14. Flutter stores 'access token' in FlutterSecureStorage key "access token" 15. Flutter stores 'refresh token' in FlutterSecureStorage key "refresh token" 16. Flutter caches user JSON in FlutterSecureStorage key "cached user" 17. GoRouter 'redirect()' detects 'authState = User' → navigates to '/niyyah' 18. User selects niyyah on 'NiyyahSelectionScreen' (e.g., 'nikah year') 19. Flutter sends 'PATCH /v1/profiles/me' with 'niyyah: "nikah year"' 20. 'ProfileHandler.UpdateProfile()' calls 'profileSvc.UpsertProfile(ctx, profile)' 21. 'ProfileService.UpsertProfile()' calls 'profileRepo.Upsert(ctx, profile)' → UPSERT into 'social.profiles' 22. Response: 'status: "success", data: profile' 23. Flutter navigates to '/home' (DiscoveryScreen)



Figure 5: Sequence Diagram for Encrypted Registration and Identity Ingestion.

Diagram D: Sequence Diagram — Swipe Right and Match Creation Participants: User A (Flutter), MatchingNotifier, Dio Client, nginx, Go Matching Handler, MatchingService, MatchRepository (Postgres), MatchingCache (Redis), NotificationService, PushChannel, User B (Flutter via WebSocket Hub)

Steps: 1. User A swipes right on a candidate card in 'DiscoveryScreen' 2. Flutter calls 'matchingNotifier.like(candidateUserID)' 3. 'MatchingNotifier.like()' calls 'POST /v1/matching/like' with body 'target id: candidateUserID' 4. Matching Handler validates JWT, extracts 'userAID' from context 5. 'MatchingHandler.Like()' calls 'match-

ingSvc.Like(ctx, userAID, targetID)' 6. 'MatchingService.Like()' validates 'userAID != targetID' (returns 'domain.ErrInvalidInput' if equal)

7. MatchingService.Like() calls matchRepo.RecordLike(ctx, userAID, targetID) 8. 'RecordLike()' opens a PostgreSQL transaction 9. 'RecordLike()' executes 'INSERT INTO social.likes (liker id, liked id) VALUES (1,2) ON CONFLICT DO NOTHING' 10. 'RecordLike()' checks 'SELECT 1 FROM social.likes WHERE liker id=targetID AND liked id=userAID' 11a. No reverse like found: commits transaction, returns '(matched=false, matchID=uuid.Nil, nil)' → Response: 'matched: false' → Flutter shows no banner 11b. Reverse like found: 'RecordLike()' executes 'INSERT INTO social.matches (user a id, user b id, matched at, niyyah timer ends at)', commits transaction, returns '(matched=true, matchID, nil)' 12. (match case) 'MatchingService.Like()' calls 'notifSvc.Create(ctx, targetID, "match", ...)' → INSERT into 'social.notifications' 13. (match case) 'MatchingService.Like()' sends non-blocking to 'pushCh <- domain.PushEventUserID: targetID, Title: "!", ...' 14. PushWorker receives from 'pushCh', fetches 'user.FCMToken', calls Firebase FCM API 15. (match case) 'matchingCache.AddSeen(ctx, userAID, [targetID], 24h)' — adds target to seen set 16. Response: 'matched: true, match id: "<uuid>"' returned to Flutter 17. Flutter 'MatchingNotifier' sets 'lastMatch = matchData', triggers match banner display 18. User B receives FCM push notification (if app in background) or WebSocket 'match notification' event (if online)



Figure 6: Sequence Diagram for Full-Duplex WSS Session Handshake and Message Distribution Hub

Diagram E: Sequence Diagram — Real-time Chat Message Participants: User A (Flutter ChatScreen), ChatNotifier A, WebSocket Conn A, Hub (Go), ChatService, MessageRepository (Postgres), Redis pub/sub, Hub Instance B (optional), WebSocket Conn B, ChatNotifier B (Flutter)

Steps: 1. User A types a message and taps Send in 'ChatScreen' 2. 'ChatNotifier.send(content)' creates optimistic message 'id: "temp <uuid>", content, sender id: currentUserId, created at: now', inserts into local 'messages' list 3. 'ChatNotifier' calls 'channel.sink.add(jsonEncode(type: "chat msg", payload: match id, content))' 4. WebSocket frame sent to Hub over TLS 5. Hub 'readPump' goroutine receives 'wsIncomingtype: "chat msg", payload: wsChatPayloadmatch id, content' 6. Hub calls 'chatSvc.SendMessage(ctx, matchID, senderID, content)' 7. 'ChatService.SendMessage()' verifies sender is a participant in this match: 'matchRepo.IsMatched(ctx, senderID, peerID)' 8. 'ChatService.SendMessage()' encrypts 'content' with AES-256-GCM using 'crypto.Encrypt([]byte(content))' → 'ciphertext' 9. 'ChatService.SendMessage()' calls 'messageRepo.Create(ctx, messagematch id, sender id, content: ciphertext)' → INSERT into 'social.messages' 10. Hub looks up 'connections[recipientID]' under read lock 11a. Recipient connected locally: Hub writes 'wsOutgoingtype: "chat msg", payload: id, match id, sender id, content: plaintext, created at'

directly to recipient WebSocket connection 11b. Recipient not connected locally (or always, for cross-instance): Hub publishes to Redis: ‘PUBLISH chat:<matchID> <json message>’ 12. Redis delivers the PUBLISH to all subscribing Hub instances 13. Hub instance (may be same or different server) receives from SUBSCRIBE, looks up ‘connections[recipientID]’, writes to local connection 14. ChatNotifier B receives WebSocket message, checks ‘id’ against existing messages to avoid duplicates 15. ChatNotifier B appends message to ‘state.messages’ list; Flutter widget tree re-renders 16. Hub sends delivery confirmation back to Sender A with ‘type: "chat msg", id: <db uuid>, ...’ replacing the optimistic ‘temp *’ entry: ChatNotifier A calls ‘indexWhere((m) => m[‘id’].startsWith(‘temp ’) and m[‘content’] == content)’ and replaces with confirmed message

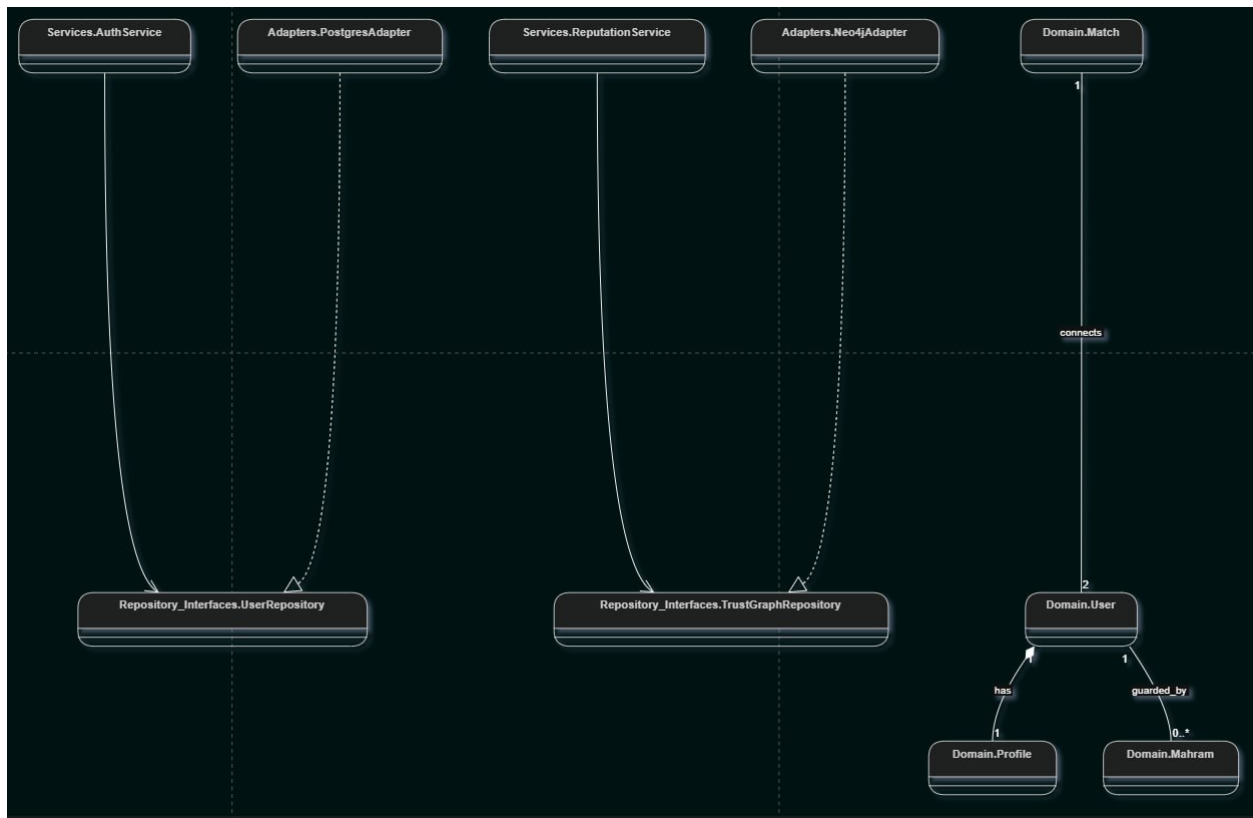


Figure 7: Sequence Diagram for Supervised Three-Party Communication (Khalwa Prevention Subsystem)

Diagram F: Sequence Diagram — JWT Token Refresh Participants: Flutter ChatScreen (or any screen), Dio Client, AuthInterceptor, Secure Storage, Go Auth Handler, AuthService, RefreshTokenRepository

Steps: 1. Dio sends any API request with ‘Authorization: Bearer <expired access token>’ 2. Go middleware attempts to verify token → JWT expiry error → returns HTTP 401 3. ‘AuthInterceptor.onError()’ intercepts the 401 response 4. Interceptor checks ‘response.requestOptions.extra[‘retried’] != true’ (prevents infinite retry loop) 5. Interceptor reads ‘refresh token’ from FlutterSecureStorage 6. Interceptor sends ‘POST /v1/auth/refresh’

with body 'refresh token: <raw token>' using a plain Dio instance (bypasses the interceptor to prevent recursion) 7. 'AuthHandler.Refresh()' calls 'authSvc.RefreshToken(ctx, rawToken)' 8. 'AuthService.RefreshToken()' hashes the incoming token with Argon2id 9. 'AuthService.RefreshToken()' calls 'tokenRepo.GetByHash(ctx, hash)' → SELECT from 'social.refresh tokens' 10a. Token not found or already used: returns 'domain.ErrNotFound' → 401 → interceptor clears all secure storage → 'AuthNotifier' sets state to null → GoRouter redirects to '/auth/login' 10b. Token valid and unused: 'AuthService.RefreshToken()' calls 'tokenRepo.MarkUsed(ctx, tokenID)' → UPDATE 'used at = NOW()' 11. New access token generated via 'jwtManager.Sign(userID, isAdmin)' 12. New refresh token generated (256-bit random), hashed and stored in 'social.refresh tokens' 13. Response: 'access token: "<new jwt>", refresh token: "<new raw token>", user: ...' 14. Interceptor stores new 'access token' and 'refresh token' in FlutterSecureStorage 15. Interceptor retries the original failed request with 'options.extra['retried'] = true' and the new Bearer token 16. Original request succeeds with HTTP 200; response forwarded to calling provider

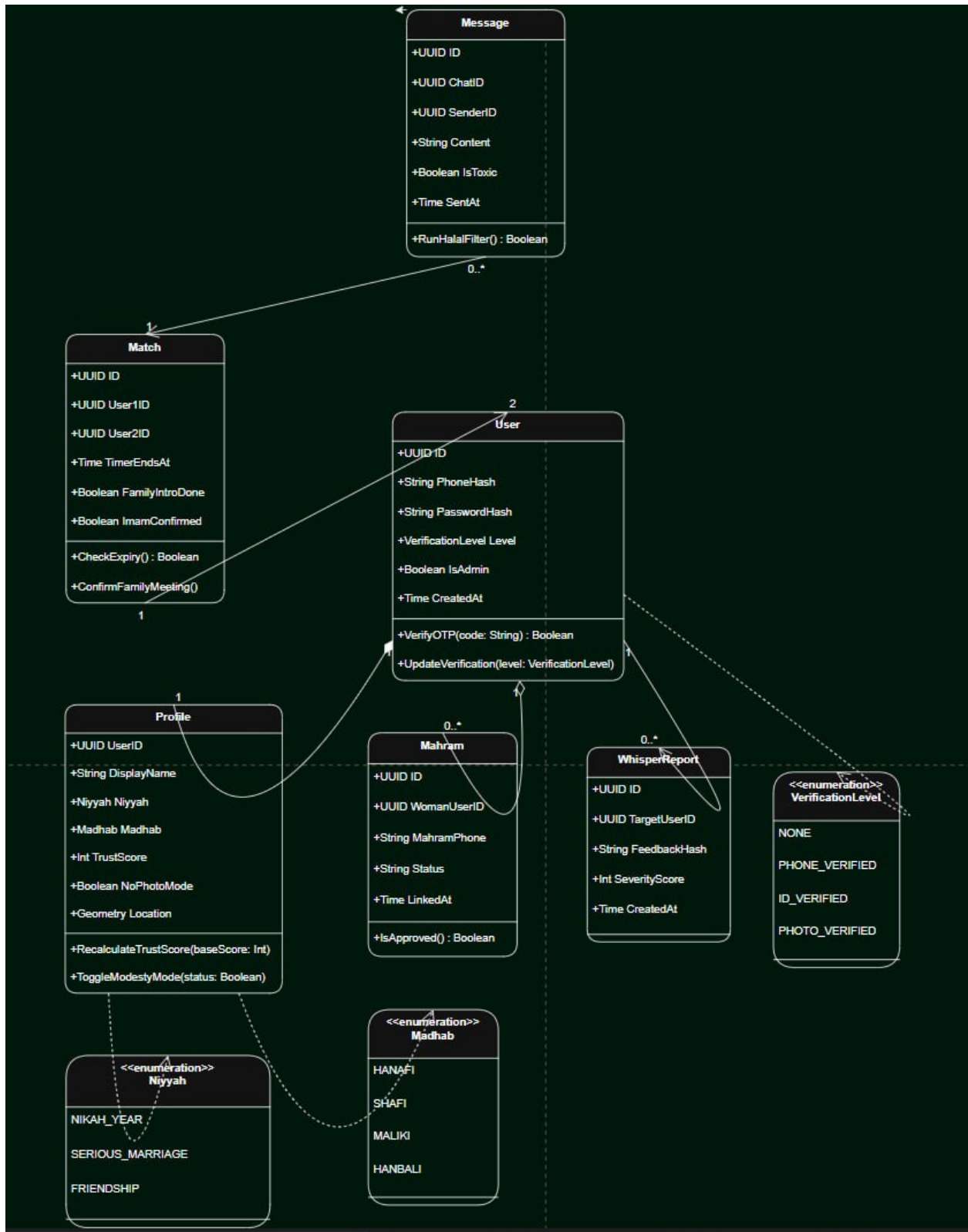


Diagram G: Entity-Relationship Diagram

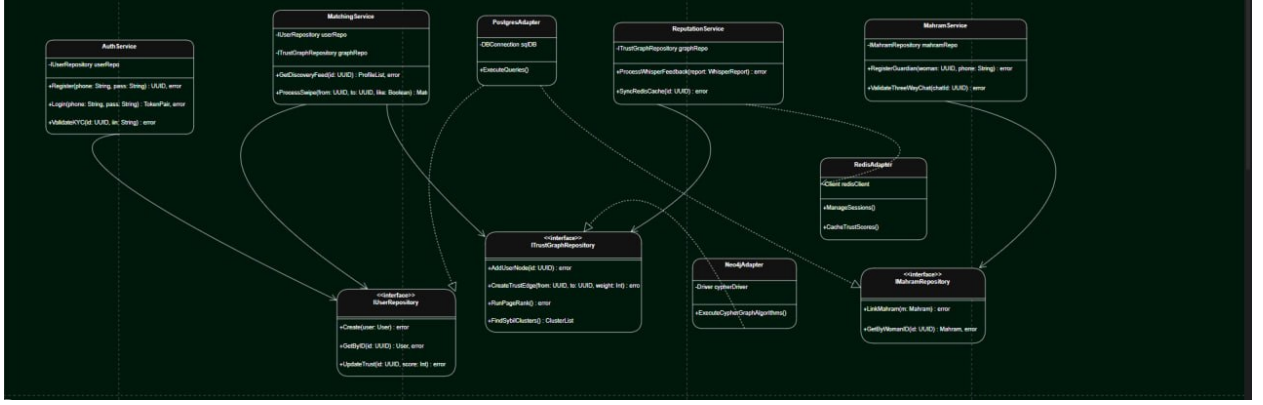


Figure 8: Go Backend Architecture Class Layout: Domain and Use Case Interactivity.)

Entities and relationships: social.users (1) social.profiles** [FK: profiles.user id → users.id CASCADE] social.users (1) social.user settings** [FK: user settings.user id → users.id CASCADE] social.users (1) social.likes** [FK: likes.likers id → users.id CASCADE; likes.liked id → users.id CASCADE] Unique constraint: (likers id, liked id) social.users × social.users () () via **social.matches** [FK: matches.user a id → users.id CASCADE; matches.user b id → users.id CASCADE] Unique constraint: (LEAST(user a id, user b id), GREATEST(user a id, user b id)) social.matches (1) social.messages** [FK: messages.match id → matches.id CASCADE] social.users (1) () **social.messages** [FK: messages.sender id → users.id CASCADE] social.users (1) social.posts** [FK: posts.author id → users.id CASCADE] social.posts(1) social.comments** [FK: comments.post id → posts.id CASCADE] social.users (1) social.comments** [FK: comments.author id → users.id CASCADE] social.posts × social.users () () via **social.post likes** [Composite PK: (post id, user id)] social.users (1) social.notifications** [FK: notifications.user id → users.id CASCADE] social.users (1) () **social.interactions** [FK: interactions.rater id → users.id CASCADE; interactions.rated id → users.id CASCADE] social.users (1) social.mahrams [FK: mahrams.woman user id → users.id CASCADE] social.matches social.mahram chat rooms [FK: mahram chat rooms.match id → matches.id CASCADE] social.users social.mahram chat rooms [FK: mahram chat rooms.mahram user id → users.id CASCADE] social.mahram chat rooms (1) social.mahram messages [FK: mahram messages.room id → mahram chat rooms.id CASCADE] social.users social.mahram messages [FK: mahram messages.sender id → users.id CASCADE] social.users social.refresh tokens [FK: refresh tokens.user id → users.id CASCADE] social.users social.swipe rejections [FK → users.id CASCADE; Composite PK: (user id, rejected id)] social.users social.blocks [FK → users.id CASCADE; Composite PK: (blocker id, blocked id)] social.users social.kyc submissions [FK: kyc submissions.user id → users.id CASCADE] social.users social.whisper reports [FK: reporter id, reported id → users.id CASCADE] social.users identity vault.iin vault [FK: iin vault.user id → social.users.id] Neo4j nodes (separate graph store): (:User uid) [:RATED rating, weight, created at]> (:User uid) (:User uid) [:INTERACTED WITH context, created at]> (:User uid)

Diagram H: Class Diagram — Backend Service Interfaces

Interface: ProfileRepository ('internal/repository/profile_repo.go') + Upsert(ctx context.Context, p *domain.Profile) error + GetByUserID(ctx context.Context, id uuid.UUID) (*domain.Profile, error) + FindCandidates(ctx context.Context, opts FindCandidatesOpts) ([]*CandidateRow, error) + GetLeaderboard(ctx context.Context, limit int)

([]domain.LeaderboardEntry, error) + SetMarriedViaApp(ctx context.Context, userID uuid.UUID) error

Interface: MatchRepository ('internal/repository/match_repo.go') + RecordLike(ctx, userID, targetID uuid.UUID) (matched bool, matchID uuid.UUID, err error) + RecordPass(ctx, userID, targetID uuid.UUID) error + ListMatches(ctx, userID uuid.UUID, cursor string, limit int) ([]*domain.Match, nextCursor string, err error) + GetMatch(ctx, matchID, userID uuid.UUID) (*domain.Match, error) + IsMatched(ctx, userA, userB uuid.UUID) (bool, error) + Unmatch(ctx, matchID, userID uuid.UUID) error + UnmatchByUsers(ctx, userA, userB uuid.UUID) error + BlockUser(ctx, blockerID, blockedID uuid.UUID) error + GetBlockedIDs(ctx, userID uuid.UUID) ([]uuid.UUID, error) + GetRejectedIDs(ctx, userID uuid.UUID) ([]uuid.UUID, error) + MarkFamilyIntroDone(ctx, matchID uuid.UUID) error + MarkImamConfirmed(ctx, matchID uuid.UUID) error + FindExpiredNiyahMatches(ctx context.Context) ([]*domain.Match, error) + ListMatchViews(ctx, userID uuid.UUID, cursor string, limit int) ([]*MatchViewRow, string, error) + GetPendingLikes(ctx, userID uuid.UUID) ([]uuid.UUID, error) Interface: TrustGraphRepository ('internal/repository/trust_graph_repo.go') + UpsertUser(ctx context.Context, userID uuid.UUID) error + RecordRating(ctx, raterID, ratedID uuid.UUID, rating int, weight float64) error + ComputeScore(ctx, userID uuid.UUID) (int, error) + DeleteUserNode(ctx, userID uuid.UUID) error + RunSybilDetection(ctx context.Context) ([]*SybilCluster, error)

Struct: Hub ('internal/handler/chat_handler.go') - mu sync.RWMutex - connections map[uuid.UUID]*websocket.Conn - chatSvc *service.ChatService - matchSvc *service.MatchingService - reputationSvc *service.ReputationService - mahramChatSvc *service.MahramChatService - rdb *redis.Client - jwtManager *tcjwt.Manager - log *slog.Logger - allowedOrigins []string - isDev bool + NewHub(...) *Hub + ServeWS(c *gin.Context) - readPump(ctx, conn, userID) - sendToUser(userID, msg) - subscribeRedis(ctx)

Struct: MatchingService ('internal/service/matching_service.go') - profileRepo repository.ProfileRepository - userRepo repository.UserRepository

- matchRepo repository.MatchRepository - settingsRepo repository.SettingsRepository - matchingCache repository.MatchingCache - graphRepo repository.TrustGraphRepository - notifSvc *NotificationService - pushCh chan<- domain.PushEvent + NewMatchingService(...) *MatchingService + GetCandidates(ctx, requesterID uuid.UUID) ([]*CandidateView, error) + Like(ctx, userID, targetID uuid.UUID) (*LikeResult, error) + Pass(ctx, userID, targetID uuid.UUID) error + ListMatches(ctx, userID uuid.UUID, cursor string, limit int) ([]*MatchView, string, error) + GetGraphCandidates(ctx, userID uuid.UUID)

([]*CandidateView, error)

Dependencies (arrows): - MatchingService uses> ProfileRepository (interface) - MatchingService uses> MatchRepository (interface) - MatchingService uses> SettingsRepository (interface) - MatchingService uses> TrustGraphRepository (interface) - Hub uses> ChatService - Hub uses> MatchingService - Hub uses> MahramChatService - postgres.ProfileRepo implements> ProfileRepository - postgres.MatchRepo implements> MatchRepository - neo4j.TrustGraphRepo implements> TrustGraphRepository



Figure 9: Structural Specifications for Dependency Injection Pipelines (Service-to-Repository Bindings)

5 Technology Selection and Justification

5.1 Backend Language and Runtime

Choosing a backend computing language is one of the most important architectural decisions for any long-term distributed system. Three approaches have been thoroughly evaluated for TrueConnect: **Go (Golang) 1.25**, **Node.js 20 LTS (TypeScript)**, and **Python 3.12 (FastAPI/asyncio)**. Every one of them represents a good modern option, with the criteria used to evaluate the candidate being the concurrency model, memory footprint, type safety, deployment size, ecosystem maturity for the required libraries, and operational characteristics on one Kazakhstan-resident VPS.

5.1.1 Comparative Analysis

Table 3: Backend Language Comparison

Criterion	Go 1.25	Node.js 20 + TS	Python 3.12 + FastAPI
Concurrency	CSP goroutines (2KB stack)	Event loop + Workers	asyncio (single-threaded)
Memory/connection	~4 KB	~20–40 KB	~40–80 KB
Binary size	~8 MB static	N/A (runtime)	N/A (interpreter)
Type safety	Static, compile-time	Optional (TypeScript)	Optional (type hints)
Race detection	Built-in <code>-race</code>	Limited	GIL prevents parallelism
Docker image	~12 MB	~180 MB	~200 MB

Concurrency under load. TrueConnect’s WebSocket hub keeps one goroutine per active WebSocket connection. In Go, the goroutines are multiplexed onto OS threads by a runtime scheduler; each goroutine’s initial stack is 2–8 KB and grows dynamically. A server holding 10,000 concurrent WebSocket connections can therefore consume about 20–80 MB of stack memory in Go. The equivalent Node.js implementation with one event loop and callbacks would eliminate parallel stack usage, but would be inefficient for CPU-bound tasks (trust score computations, Argon2id hashing) without offloading them to Worker Threads, reintroducing concurrency complexity without the type safety of Go’s interfaces. Python’s GIL effectively prevents parallelism on CPU-bound work; even with asyncio, Argon2id hashing

blocks the event loop unless wrapped in a run in executor thread pool, creating latency and conceptual overhead.

Type safety and maintainability. Clean Architecture as implemented in TrueConnect relies upon rigid interface contracts – for example, `repository.MatchRepository` defines eight method signatures that the PostgreSQL adapter must satisfy at compile time. The structural typing of Go means that the compiler makes sure all contracts are verified at each build and interface drift gets caught before it reaches staging. TypeScript similarly provides such guarantees, so long as strict mode is applied throughout a project and type imports are properly maintained. The type hints provide only advisory without the CI mypy step, making it very error-prone in team environments.

Binary deployment. The Go backend compiles to a single static binary (8 MB) that runs on `alpine:3.19` with no runtime dependency other than the operating system kernel. The Docker image is therefore about 12 MB. Node.js and Python need their respective runtimes, plus all node modules or site-packages, to generate images which typically get up to 15–20× larger. On a Kazakhstan VPS with limited network bandwidth, it materially affects deployment time, as well as the chance of supply-chain vulnerabilities induced by transitive npm or pip dependencies.

Race detection. The codebase’s 102 service-layer tests are executed using the built-in `-race` detector on every CI run (`.github/workflows/ci.yml`). The race detector monitors memory accesses at compile time and reports data races at runtime, a necessary protection given the concurrent WebSocket hub where multiple goroutines write and read shared connection maps. There are no similar compile-time race instrumentations for Node.js; Python’s GIL has implicit mutual exclusion built into CPython, but at the cost of true parallelism.

5.1.2 Decision

Go 1.25 was chosen as the backend language. The decisive factors of the choice were: (i) the goroutine model being mapped cleanly onto the one-goroutine-per-connection WebSocket architecture; (ii) the compile-time interface enforcement facilitating Clean Architecture contracts; (iii) the smallest deployment footprint; and (iv) native race detection, essential for the concurrent trust engine and WebSocket hub.

5.2 Mobile Framework

5.2.1 Candidates Evaluated

Three approaches were considered for the mobile client: `**Flutter 3.16+ (Dart 3.2)**`, `**React Native 0.73 (TypeScript)**`, and `**native development**` (Swift/Kotlin per platform). The primary axis of evaluation was development velocity for a single-developer graduate

project while maintaining production-quality UI faithful to the Islamic design system.

5.2.2 Comparative Analysis

Table 4: Primary Database Comparison

Criterion	PostgreSQL 16 + PostGIS	MongoDB 7	Firebase Firestore
Data model	Relational (ACID, full SQL)	Document (BSON, flexible schema)	Document (JSON, limited joins)
FK enforcement	Native (FOREIGN KEY, cascades)	Application-level only	Application-level only
Geospatial queries	PostGIS: ST_DWithin, ST_Distance, GEOGRAPHY type	\$geoNear (2dsphere index)	Geohash-based (Geo-Point, no distance queries)
Full-text search	tsvector / pg_trgm	MongoDB Atlas Search (external)	Requires Algolia / Elasticsearch
ACID transactions	Multi-statement, cross-table	Multi-document (4.0+, with overhead)	Limited (single document atomic)
Encryption at rest	pgcrypto, column-level AES	Field-level encryption (Enterprise)	AES-256 (managed, US jurisdiction)
Row-level security	Native (PostgreSQL RLS policies)	Collection-level security rules	Security rules (coarser granularity)
Data sovereignty	Self-hosted KZ VPS	Self-hosted or Atlas (region selection)	Google Cloud (US-based, GDPR issues)
Schema migrations	golang-migrate (versioned SQL files)	Manual or migrate-mongo	Schema-less (no migration tooling)
Two-schema isolation	Native (social + identity_vault)	Database-level separation only	No namespace isolation
Cost (self-hosted)	Free (open-source)	Free (Community)	Billing per read/write (\$0.06/100K reads)
Operational complexity	Medium (requires DBA for tuning)	Medium	Low (managed, but no control)

PostGIS for proximity matching. TrueConnect’s candidate discovery query uses PostGIS’s `ST_DWithin(p.location, ST_MakePoint(lon,lat)::geography, radius m)` to filter candidates within a configurable radius (default 50 km, adjustable via `social.user.settings.max_distance km`). PostGIS stores locations as the `GEOGRAPHY` type with SRID 4326 (WGS 84), which computes distances correctly on the spherical Earth model. MongoDB’s `geoNear` aggregation stage also supports spherical distance queries on a `2dsphere` index and would be a viable alternative. Firebase’s `GeoPoint` type stores latitude/longitude but provides no server-side distance query; implementing proximity search would require a client-side geohash approach or a Cloud Function, adding latency and cost per query.

Two-schema identity vault. Kazakhstan’s data sovereignty requirements mandate that citizens’ IIN (individual identification number) and full name remain on KZ-resident servers. The identity vault schema in PostgreSQL is accessible only to the restricted identity vault role database role; the application’s main connection user (`trueconnect app`) cannot read vault tables directly. This role-based schema isolation is a native PostgreSQL feature with no equivalent in MongoDB (which provides database-level separation only) or Firestore (which has no concept of schemas).

ACID transactions for match atomicity. The matching algorithm must atomically: (1) insert a row into `social.likes`, (2) query whether the reverse like exists, and (3) if so, insert a row into `social.matches` and return the new match ID. This three-step operation is wrapped in a `BEGIN/COMMIT` PostgreSQL transaction in `internal/adapters/postgres/match repo.go`. If any step fails, the entire transaction rolls back, preventing phantom matches or duplicate like records. MongoDB 4.0+ supports multi-document transactions, but with measurable write amplification overhead (the WiredTiger storage engine must track transaction state across documents). Firestore’s atomic operations are limited to a single document or a batch of writes with no conditional logic, making the like-then-match pattern awkward to implement without Cloud Functions.

Golang-migrate versioned migrations. The 19 SQL migration files in `migrations/` (format: `000001 init.up.sql` through `000019 add cascade deletes.up.sql`) provide a deterministic, version-controlled schema evolution path. Each migration has a paired `.down.sql` for rollback. MongoDB’s schemaless nature eliminates migration files but transfers schema management to application code, which is fragile at scale a partially deployed application may write documents in new format while old instances still run.

5.2.3 Decision

PostgreSQL 16 with the PostGIS extension was selected. The decisive factors were: (1) PostGIS `ST_DWithin` for the location-based candidate filter without external services; (2) native schema isolation (`social + identity vault`) required by Kazakhstan data law; (3) ACID multi-statement transactions for atomic like/match detection; and (4) zero licensing cost with

full self-hosting capability on the project VPS.

5.3 Social Graph Database

5.3.1 The Case for a Dedicated Graph Store

You can compute the trust scores using weighted PageRank and check Sybil clusters through Louvain’s community detection, which are graph algorithms. Although recursive CTEs in PostgreSQL can model graph traversals, this leads to polynomial query complexity as a function of depth, whereas in PostgreSQL the 6-hop relationship chain needs a recursive CTE, with as much as $O(n \text{ over } k)$ in intermediate relation rows. The native graph storage in Neo4j (index-free adjacency) prevents multi-hop traversal down to $O(k)$ pointer dereferences irrespective of graph size.

Table 5: Social Graph Options

Criterion	Neo4j 5 Community + GDS	PostgreSQL (recursive CTE)	Amazon Neptune
Storage model	Native graph (index-free adjacency)	Adjacency list in relational tables	Property graph / RDF
Multi-hop traversal	$O(k)$ pointer dereferences	$O(n^k)$ recursive CTE	$O(k)$ native (TinkerPop Gremlin)
PageRank	GDS library (<code>gds.pageRank.stream</code>)	Manual iterative SQL (complex)	Gremlin PageRankVertexProgram
Louvain clustering	GDS library (<code>gds.louvain.stream</code>)	Not available natively	Not available natively
Sybil detection	Louvain + degree centrality (built-in)	Requires external Gephi export	Requires external tooling
Data sovereignty	Self-hosted KZ VPS	Self-hosted	AWS (US jurisdiction)
Cost	Free (Community)	Free (included in PostgreSQL)	~\$0.10/hour minimum
Go driver	Official <code>neo4j-go-driver/v5</code>	<code>pgx/v5</code>	<code>gremlin-go</code>
Operational complexity	Medium (separate JVM process)	Low (reuse existing DB)	Low (managed)

GDS algorithms in production. The TrustEngine worker (`internal/worker/trust-engine.go`) calls `gds.pageRank.stream` with relationship weight set to the product of the rater’s verification weight ($1.5\times$ for KYC-verified users) and the normalized rating value.

This query operates within Neo4j’s in-memory projection and returns a stream of (nodeId, score) pairs in milliseconds, a query that would take a PostgreSQL series of dozens of iterations to converge. The Louvain community detection (gds.louvain.stream) is created for identifying user clusters with a high degree of heavy mutual interaction but relatively little interconnect with the other elements of the graph, which Sybil detection logic interprets as rings of fake accounts.

PostgreSQL as a fallback. Trust scores are written in both Neo4j (canonical) and `social.users.trust_score(denormalizedcopy)toenableSQLjoinsontrustscoredduringcandidatediscoveryquery`

5.3.2 Decision

Neo4j 5 Community with the Graph Data Science plugin was selected. PostgreSQL by itself does not support PageRank or Louvain efficiently; Neptune satisfies technical requirements, but does not respect data sovereignty of Kazakhstan. Neo4j’s own self-hosted Community edition is free and the official Go driver is supported through the GDS library that has both the required algorithms natively available.

5.4 Real-Time Communication Protocol

5.4.1 Candidates Evaluated

Bidirectional real-time messaging for TrueConnect chat and mahram supervision functionality needs a low-latency, persistent-connection protocol. The following three methodologies were considered: WebSocket (RFC 6455), Server-Sent Events (SSE) + HTTP POST for client-to-server, long polling.

Table 6: Real-Time Protocol Comparison

Criterion	WebSocket (RFC 6455)	SSE + HTTP POST	Long Polling
Connection type	Full-duplex persistent TCP	Half-duplex (server push only)	Request-response loop
Latency (median)	1–5 ms	10–30 ms (reconnect overhead)	100–500 ms (poll interval)
Server memory per connection	~4 KB (goroutine stack, Go)	~2 KB (SSE goroutine)	Negligible (ephemeral handlers)
Bidirectional	Yes (client and server send at any time)	No (client POSTs separately)	No (request-response only)
Typing indicators	Trivially sent as WS message type	Requires separate POST endpoint	Not practical
WebRTC signaling	Directly via <code>webrtc_offer/answer</code> and <code>ice_candidate</code> message types	Possible but awkward (client POSTs to SSE and receives)	Not practical
Nginx proxying	<code>proxy_pass</code> + <code>Upgrade</code> header + 86400s timeouts	Standard HTTP proxy	Standard HTTP proxy
Firewall traversal	Port 80/443 (via HTTP Upgrade)	Port 80/443	Port 80/443
Mobile reconnect	Exponential backoff (1/2/4/8/16 s)	EventSource auto-reconnect	Application-level retry
Flutter library	<code>web_socket_channel</code> (official)	<code>http</code> package (SSE parsing)	<code>http</code> package (polling loop)

Full-duplex requirement. TrueConnect’s WebSocket protocol contains eight different client-to-server message types (auth, chat msg, typing, read, webrtc offer, webrtc answer, webrtc ice candidate, mahram chat msg) and seven different server-to-client message types (auth ok, chat msg, mahram chat msg, content warning, content blocked, match notification, typing). SSE would be good enough to manage server-to-client delivery but every client-to-server message should have its own HTTP POST endpoint, which would mean eight more API surface points, and a doubling of network round-trips for interactive tasks such as typing indicators (which will fire on every keypress).

WebRTC signaling. The CallScreen uses WebRTC for video calls, and as such needs an ICE candidate exchange (usually 3–10 candidates per peer) combined with SDP offer/answer negotiation. The WebSocket Hub is then used as the signaling channel: webrtc offer,

webrtc answer, and webrtc ice candidate message types are relayed via the Hub's connections map to the target user's goroutine. This method takes advantage of existing authenticated WebSocket connection for signaling no separate signaling server is needed. Long polling cannot send signaling traffic even when the latency is acceptable. SSE + HTTP POST could ideally, but lose the authentication context that is already built over the WebSocket connection.

Redis Pub/Sub for horizontal scaling. If a chat message is submitted through POST or WebSocket to one of the instances in the API, it receives an address that must connect to the WebSocket of the client in its own network, even if from its own API. The Hub transmits to a Redis channel `chat:<matchId>` for every outgoing message; all Hub instances subscribe and send to their locals. This fan-out scheme would work the same for WebSocket messages. SSE would still need the same Redis pub/sub backing, but the individual POST path for client messages would also introduce a second Redis channel type for the client post, increasing the total complexity of operations.

Nginx configuration. In `deployments/nginx/nginx.conf`: The TrueConnect nginx configuration sets proxy read timeout 86400s and proxy send timeout 86400s on the `/v1/ws` location block, proxy http version 1.1, proxy set header Upgrade http upgrade, proxy set header Connection "upgrade". As per the normal nginx WebSocket proxy setup. SSE and long polling would make use of standard HTTP proxy settings, but generate more total requests (SSE for reconnection periods (EventSource reconnects when connection failure occurs, defaulting to 3 s), long polling due to request-response loop).

5.4.2 Decision

We chose WebSocket (RFC 6455) via `coder/websocket`. The full-duplexity needed from WebRTC signaling alone supports the selection; typing indicator frequency (up to 10 events per second per user) instead of SSE would incur 10 POST requests per second per active typist; and the authentication context established at WebSocket open avoids the per-request JWT verification burden for high-frequency message types.

5.5 Caching and Session Store

Redis 7 was selected over Memcached and over storing sessions in PostgreSQL based on three functional requirements specific to TrueConnect:

WebSocket Pub/Sub. Redis's native Pub/Sub is used to fan out chat messages across Hub instances. Memcached has no pub/sub primitive; PostgreSQL 'LISTEN'/'NOTIFY' is limited to 8 KB payloads and does not scale past a few hundred concurrent listeners.

Sliding-window rate limiting. Per-user rate limiting uses Redis sorted sets ('ZADD' + 'ZREMRANGEBYSCORE' + 'ZCARD') implementing the sliding window algorithm. The

atomic Lua script evaluates the window and increments the counter in a single round-trip. PostgreSQL could implement the same with a ‘rate limit events’ table, but would require a row-level lock per request unacceptable for the authentication endpoints limited to 5 req/s.

Candidate seen-set TTL. The matching algorithm tracks which candidates a user has already seen using a Redis set keyed ‘seen:<userId>’ with a 24-hour TTL. Redis’s native key expiration avoids a background cleanup job. PostgreSQL’s ‘social.swipe rejections’ table stores permanent pass decisions, but the ephemeral 24-hour seen-set (which excludes previously shown profiles from the current session’s batch) is unsuited to a relational table.

Table 7: Cache Store Comparison

Criterion	Redis 7	Memcached 1.6	PostgreSQL (session table)
Pub/Sub	Native (PUBLISH/SUBSCRIBE)	Not available	LISTEN/NOTIFY (8KB limit)
Key TTL	Per-key configurable	Per-item TTL	Requires background job
Sorted sets	Native (ZADD, ZRANGE)	Not available	Requires table + index
Lua scripting	EVAL atomic scripts	Not available	PL/pgSQL functions
Persistence	RDB snapshots + AOF	None	Full ACID
Max memory policy	allkeys-lru (configured)	allkeys-lru	N/A
Data structures	Strings, sets, sorted sets, hashes, streams	Strings only	Full SQL types

Redis 7 is provisioned with maxmemory 256mb and maxmemory-policy allkeys-lru in the Docker Compose file. This ensures that under memory pressure, the least recently used cached entries (mainly candidate seen-sets and non-critical caches) are evicted before critical operational data (session tokens use explicit TTLs and are replicated to PostgreSQL).

5.6 Object Storage

MinIO is chosen over AWS S3 and Google Cloud Storage for the main reason: data sovereignty. Under Kazakhstan’s Law on Personal Data (2013, amended 2023), all personal data of citizens of Kazakhstan shall be stored within the Republic of Kazakhstan. MinIO is a self-hosted S3-compatible object store hosted on the same KZ-resident VPS as the rest of the stack, ensuring that avatar photos and KYC identity documents never leave Kazakhstan jurisdiction.

The compatibility with S3 APIs means that the minio-go/v7 client SDK is API-identical to the AWS SDK, which makes it straightforward to migrate to AWS S3 (for non-PII media) in the future without modifying one’s codebase if the legal landscape evolves.

5.7 Summary

Table 8: Technology Selection Summary

Layer	Selected	Primary Justification
Backend language	Go 1.25	Goroutine-per-connection WS model; compile-time interface enforcement; <code>-race</code> detector
Mobile framework	Flutter 3.16 (Dart 3.2)	<code>CustomPainter</code> for Islamic geometric art; Riverpod compile-time safety; 95% code sharing
Relational database	PostgreSQL 16 + PostGIS	PostGIS proximity queries; dual-schema identity vault; ACID match atomicity
Graph database	Neo4j 5 + GDS	GDS PageRank and Louvain; $O(k)$ traversal; self-hosted for KZ data sovereignty
Real-time protocol	WebSocket (RFC 6455)	Full-duplex for WebRTC signaling; typing indicators; Redis pub/sub fan-out
Cache / session store	Redis 7	Pub/Sub; sliding-window rate limiting; seen-set TTL eviction
Object storage	MinIO	KZ data sovereignty for photos and KYC documents; S3-compatible API

Each selection is grounded in a specific, verifiable technical requirement of the TrueConnect system rather than general popularity. I have also implemented Go’s concurrency model, Flutter’s rendering engine, PostgreSQL’s relational integrity with Neo4j’s graph algorithms, and WebSocket’s full-duplex channel with Redis’s pub/sub such that in each case the technology stack itself is cohesive and each component addresses limitations of its alternatives.

6 Implementation and Deployment

6.1 Overview

In the following chapter, we will provide a concrete implementation of TrueConnect from the first migration file to the production-ready Docker Compose stack. The project was developed incrementally over fourteen (14) sprints (Sprint 0–Sprint 13) and has been written iteratively in order to ensure that it is visible to validate and test the increment as development. The backend was completed by Sprint 11 (feature completion, 102 service-layer tests passed), and the Flutter mobile client was done on Sprint 13 with all 21 screens implemented (feature completion). This chapter presents the implementation in four significant aspects: backend API and security infrastructure, real-time communication, frontend architecture and screens, and deployment pipeline.

6.2 Backend Implementation

6.2.1 Project Bootstrapping and Database Migrations

The backend was bootstrapped as a Go module (`github.com/trueconnect/backend`) using a strict Clean Architecture directory layout. The `cmd/` directory contains three entry points `cmd/api/main.go` (HTTP server), `cmd/worker/main.go` (background jobs), and `cmd/migrate/main.go` (schema migrations) enabling independent deployment of each component. Database schema evolution is managed by `golang-migrate` through 19 versioned SQL migration files in `migrations/`. Each migration consists of a paired `.up.sql` and `.down.sql` file, enabling deterministic rollback. The migration sequence covers the full schema lifecycle:

Migration 000001 ‘social’ schema, ‘users’ table with UUID primary key, ‘phone hash’ (bytea, unique), ‘phone encrypted’, ‘email encrypted’, ‘password hash’, ‘public key’, ‘verification level’ enum, ‘trust score’ int.

Migration 000002–000005 ‘profiles’, ‘likes’, ‘matches’, ‘messages’ tables.

Migration 000006–000008 ‘posts’, ‘post likes’, ‘comments’ (social feed).

Migration 000009 ‘identity vault’ schema, ‘iin vault’ table with AES-256-encrypted IIN and full name; ‘identity vault role’ PostgreSQL role with GRANT restricted to vault schema only.

Migration 000010–000012 ‘kyc submissions’, ‘notifications’, ‘user settings’.

Migration 000013–000015 ‘niyyah’/‘madhab’/‘languages’/‘no photo mode’ columns added to profiles, ‘mahrams’, ‘mahram chat rooms’, ‘mahram messages’, ‘whisper reports’ tables.

Migration 000016–000018 ‘refresh tokens’ (one-time-use rotation), ‘interactions’, ‘sybil clusters’.

Migration 000019 ‘ON DELETE CASCADE’ added to ‘interactions’, ‘matches’, ‘messages’, ‘whisper reports’ foreign keys

The Docker Compose service ‘migrate’ runs ‘cmd/migrate/main.go’ as a one-shot container on every ‘make docker-up’, with a ‘depends on: postgres: condition: service healthy’ guard that prevents the API from starting before migrations complete.

6.2.2 Authentication Implementation

The authentication subsystem (‘internal/service/auth service.go’, ‘internal/handler/auth handler.go’, ‘internal/adaptor/postgres/user repo.go’) implements a layered security model:

Registration flow: 1. The handler receives ‘phone, email, password, gender, madhab’ and calls ‘auth service.Register()’. 2. The service normalizes the phone number, computes ‘phone hash = Argon2id(phone, salt)’ for lookup without storing plaintext, and encrypts ‘phone encrypted = AES-256-GCM(phone, ENCRYPTION KEY, random nonce)’ for recovery display. 3. ‘password hash = Argon2id(password, salt)’ with parameters 64 MB memory, 3 iterations, 4 parallelism threads — a profile chosen to require 250 ms on a single CPU core, raising the cost of offline dictionary attacks to prohibitive levels. 4. The user row is inserted; a profile row with default values is created in the same transaction. 5. A 15-minute JWT access token and 7-day refresh token are returned; the refresh token’s SHA-256 hash is stored in ‘social.refresh tokens’ alongside ‘expires at’.

Token refresh and family revocation: The refresh endpoint (‘POST /v1/auth/refresh’) validates the incoming token against the stored hash, marks the token as ‘used at = NOW()’, and issues a new pair. If a refresh token that has already been marked ‘used at’ is presented (indicating token theft and reuse), the entire token family for that user is immediately revoked, and all rows in ‘social.refresh tokens’ for ‘user id’ are deleted. This is the OWASP-recommended refresh token rotation with reuse detection.

PII field encryption: All personally identifiable fields ‘phone encrypted’, ‘email encrypted’, and the ‘identity vault’ columns ‘iin encrypted’ and ‘full name encrypted’ use AES-256-GCM with a 12-byte random nonce prepended to the ciphertext. The nonce is generated via ‘crypto/rand.Read’ on each encryption call, ensuring that two encryptions of the same plaintext produce different ciphertexts and that nonce reuse (which would break GCM’s authentication guarantee) is computationally infeasible.

6.2.3 Profile and Media Implementation

Profiles with two geographic columns in the profile subsystem, which are ‘city’ (text) and ‘location’ (PostGIS GEOGRAPHY type, SRID 4326). Avatar and photo uploads pass through

the MinIO adapter (`internal/adapter/minio/`): the handler accepts a `'multipart/form-data'` request, validates the MIME type against an allowlist (`'image/jpeg'`, `'image/png'`, `'image/webp'`), uploads content to the `'avatars'` or `'media'` MinIO bucket, and returns a presigned URL stored in `'profiles.avatar url'`. Using the `'no photo mode'` boolean, female users are able to hide their avatar from the discovery feed. When `'no photo mode = true'`, the profile adapter returns `'AvatarURL = ""'` and sets a flag `'AvatarBlurred = true'` in the `'CandidateView'` struct, which the Flutter client interprets to show a blurred silhouette placeholder instead of an empty image widget.

6.2.4 Matching Algorithm Implementation

The candidate discovery query (`'GET /v1/matching/candidates'`) executes five filtering stages in sequence:

Seen-set exclusion A Redis set `'seen:<userId>'` is loaded; the query excludes any `'user id'` in this set.

Block exclusion `'Social.blocks'` table is left-joined; profiles with an active block in either direction are excluded.

Geospatial filter `'ST DWithin(p.location, ST MakePoint(lon, lat)::geography, radius m)'` applied using the user's current latitude/longitude and `'user settings.max distance km'`.

Preference filters Age range (`'min age'` / `'max age'`), `'niyyah filter'` (requires matching niyyah enum value when set), `'madhab filter'` (requires matching madhab when set), and `'show me'` (gender preference).

Ordering Candidates are sorted by `'trust score DESC'` then by same-madhab match (+10 virtual score boost applied in Go after the query rather than in SQL to avoid polluting the stored score).

The returned batch of 20 candidates is loaded into the Flutter `'matchingNotifierProvider'` which renders them as a swipeable card stack on `'DiscoveryScreen'`. Each swipe right calls `'POST /v1/matching/like'`; the service atomically checks for a reciprocal like and creates a match if found, triggering a `'match notification'` WebSocket push to both users.

Niyyah timer: When a match is created, `'social.matches.niyyah timer ends at'` is set to `'NOW() + 90 days'`. The `'NiyyahTimerWorker'` runs daily as a background job in `'cmd/worker/main.go'`, querying matches where `'niyyah timer ends at < NOW()'` and neither `'family intro done = true'` nor `'imam confirmed = true'`, and sending a push notification reminder to both users. The Flutter `'Match'` model exposes `'daysLeftOnTimer = max(0, niyyah-TimerEndsAt.difference(DateTime.now()).inDays)'` as a computed property, displayed as a colour-coded chip on the match card (green > 30 days, orange 8–30 days, red 7 days).

6.2.5 Trust Engine Implementation

We have three kinds of files used as the trust engine implementation: `internal/worker/trust engine.go` (orchestration), `internal/service/reputation service.go` (business logic), and `internal/adaptor/neo4j/trust repo.go` (graph queries).

The service serves a post meeting review rating (`POST /v1/interactions`) when a user submits. Inserts the interaction into `social.interactions` with `confirmed at = NULL`. 2. Enqueues Neo4j write via Go channel to the TrustEngine worker. 3. The worker's `recalculateTrustScore()` function calls the Neo4j GDS PageRank projection, projecting nodes `:User` and relationships `:RATED`, based on interaction weight as relationship property. We read this back from the GDS projection, smoothing its score towards a 2.5/5 neutral baseline with $C = 5$ virtual ratings, and scaling it to 0–100. The score is atomically written to 3 stores: Neo4j (canonical node property), `social.users.trust score` (for SQL joins), and Redis key `trust:<userId>` (for feed reads of sub-millisecond swiping).

Sybil detection Runs every 6 hours via a `time.Ticker` in the worker. The Louvain community detection algorithm (`gds.louvain.stream`) is projected over the whole `:User-:INTERACTED WITH` graph. The social writes the clusters where over 70percent of edges are internal (low external connectivity) and there is only around 20percent KYC verification rate to `social.sybil clusters` and flagged for admin review from `GET /v1/admin/sybil/pending`.

6.2.6 Real-Time Chat Implementation

The WebSocket hub is implemented in `'internal/handler/chat handler.go'`. Key implementation details: **Hub struct:** `Hub connections map[uuid.UUID]*websocket.Conn // goroutine-safe via mu mu sync.RWMutex chatService ChatService matchService MatchService mahramService MahramService contentFilter ContentFilter pushService PushService jwtManager *jwt.Manager allowedOrigins []string isDev bool`

Connection lifecycle: 1. The HTTP handler performs an origin check against `'allowedOrigins'` (loaded from `'CORS ORIGINS'` env var); in dev mode all origins are allowed. 2. `'coder/websocket'` upgrades the HTTP connection. 3. A goroutine is launched for the connection; the hub's `'mu.Lock()'` registers the `'conn'` in the `'connections'` map. 4. The first incoming message must be `"type":"auth","token":"<JWT>"` within 10 seconds; a `'context.WithTimeout'` enforces the deadline. 5. On successful auth, `"type":"auth ok"` is sent and message handling begins. 6. On connection close (graceful or error), `'mu.Lock()'` removes the entry from the map. 7. A `'defer func() recover() ()'` wraps each goroutine's main loop to prevent a panicking connection from crashing the entire Hub.

Message routing: Incoming `'chat msg'` payloads are: (1) sanitized (HTML-stripped via `'internal/pkg/sanitize'`), (2) checked by the content filter (toxic classifier), (3) AES-256-GCM encrypted and stored in `'social.messages'`, (4) published to Redis channel `'chat:<matchId>'`. All Hub instances subscribe to `'chat:*'` and deliver matching messages to their locally con-

nected users. If the recipient is not connected to any instance, the push service sends an FCM notification via Firebase. Mahram supervision: When a ‘mahram chat msg’ is received, the Hub verifies that the sender is a participant in the mahram room (via ‘mahramService.IsParticipant()’), encrypts and stores in ‘social.mahram messages’, and delivers to all three room participants (woman, man, guardian).

6.2.7 Mahram, Imam, and KYC Features

Mahram registration (‘POST /v1/mahram’) accepts a guardian’s phone number, stores ‘mahram phone hash = Argon2id(phone, salt)’ in ‘social.mahrams.mahram phone hash’. When the guardian registers or logs in from that phone number, the system resolves the hash and grants them access to the associated mahram rooms. This approach avoids storing the guardian’s plaintext phone number while still enabling lookup by phone at login time.

Imam catalog (‘GET /v1/imams’) is served from an embedded JSON catalog compiled into the binary via Go’s ‘//go:embed’ directive in ‘internal/pkg/imam/'. The catalog contains 10 imams across 5 Kazakhstan cities (Almaty, Astana, Shymkent, Aktobe, Karaganda). Because this data is immutable reference data, embedding it eliminates a database round-trip for every discovery screen visit. The ‘GET /v1/venues’ endpoint similarly serves a hardcoded list of halal-certified meeting venues.

Nikah confirmation (‘POST /v1/matches/:id/nikah-confirm’) sets ‘social.matches.imam confirmed = true’ and ‘social.profiles.married via app = true’ for both users, locking the match and triggering a congratulatory push notification with the Quranic barakah phrase "Baraka Allahu feekum.

KYC flow: The ‘POST /v1/kyc/submit’ endpoint accepts a ‘multipart/form-data’ upload. The handler validates the file MIME type (allowlist: ‘image/jpeg’, ‘image/png’, ‘application/pdf’), uploads the document to MinIO under a UUID-keyed path in the ‘kyc-documents’ bucket (not publicly accessible), and inserts a row into ‘social.kyc submissions’ with status ‘pending’. The admin review endpoint (‘POST /v1/admin/kyc/:userId/verdict’) accepts ‘approved’ or ‘rejected’; on approval, ‘social.users.verification level’ is updated to ‘id verified’, triggering the 1.5× trust weight on future interaction ratings.

6.3 Frontend Implementation

6.3.1 Architecture and State Management

The Flutter application (‘frontend/’) follows a feature-first directory structure: frontend/lib/core/ constants/ router/ theme/ widgets/ features/ auth/ discovery/ kyc/ imam/ models/ “ kyc screen, kyc provider imam connect screen user.dart, profile.dart, match.dart, post.dart, ... api constants.dart, app colors.dart app router.dart (GoRouter +

auth guard) app theme.dart, app spacing.dart, app shadows.dart trust score badge.dart, niyyah badge.dart, madhab badge.dart login screen, register screen, auth provider discovery screen, matching provider chat/ feed/ matches/ profile/ settings/ notifications/ notifications screen, notifications provider chat screen, mahram chat screen, chat provider feed screen, create post screen, feed provider matches screen, matches provider profile screen, edit profile screen, profile detail screen settings screen, settings provider

All state management uses ****Riverpod 2.x****. Providers are defined at the feature level and consumed via `'ref.watch()' / 'ref.read()'` in widgets. The `'authStateProvider'` (`'AsyncNotifierProvider<AuthNotifier, User?>'`) is the root dependency: `GoRouter`'s `'AuthRouterNotifier'` watches it and redirects unauthenticated users to `'/auth/login'` on every navigation event.

6.3.2 HTTP Client and Interceptors

`DioClient` (`'core/network/dio client.dart'`) wraps the `'Dio'` HTTP client with three stacked interceptors:

Auth interceptor - reads the JWT access token from Flutter secure storage and attaches it as `Authorization: Bearer <token>` on every outgoing request. On a 401 response, the interceptor calls `POST /v1/auth/refresh` with the refresh token (stored separately in secure storage), stores the new token pair, and retries the original request transparently.

Retry interceptor — retries failed requests (network errors, 5xx responses) up to three times with exponential backoff: 1 s, 2 s, 4 s. Requests are not retried on 4xx client errors.

Error interceptor — maps HTTP error codes to user-facing snackbar messages: 422 → validation error details extracted from the response body; 429 → "Too many requests, please slow down"; 5xx → "Server error, please try again."

The base URL and all endpoint paths are defined in `core/constants/api constants.dart` as compile-time static `const String` values. The `fixImageUrl()` helper replaces the `localhost` host in `MinIO` presigned URLs with the configured server host, enabling images to load correctly on physical devices that cannot resolve `localhost`.

6.3.3 WebSocket Client

The `'chatNotifierProvider'` manages the `WebSocket` lifecycle:

1. `'connect(matchId)'` opens a `'WebSocketChannel'` to `'ws://<host>:8080/v1/ws'`.
2. Immediately sends `"type":"auth","token":"<JWT>"` as the authentication handshake.
3. Subscribes to `'channel.stream'` via `'stream.listen( handleMessage, onError: handleError, onDone: handleDone)'`.
4. `'handleMessage()'` decodes the JSON envelope and dispatches on `'type'`: `'chat msg'` → appends to message list and triggers `'setState'`; `'typing'` → sets `'isTyping = true'` with a 3-second reset timer; `'content warning'` → shows a yellow banner;

‘content blocked’ → removes the last optimistic message from the list; ‘match notification’ → shows a congratulation overlay. 5. On ‘onDone’ (connection closed), the reconnect loop fires: exponential backoff delays of 1 s, 2 s, 4 s, 8 s, 16 s (maximum 5 attempts), then gives up and shows an offline banner with a manual "Reconnect" button. 6. ‘send(matchId, text)’ serialises “type”:“chat msg”,“payload”:“match id”:“<id>”,“content”:“<text>” and writes to ‘channel.sink’. An optimistic message is appended to the state list immediately; a ‘content blocked’ response removes it.

Typing indicators are sent via ‘sendTyping(matchId)’ which is debounced: repeated keystrokes within 300 ms collapse to a single typing message, preventing the server from receiving hundreds of events per second on fast typists.

6.3.4 Key Screen Implementations

DiscoveryScreen: Renders the candidate stack using a gesture detector that tracks horizontal drag velocity. A rightward fling above the threshold triggers ‘matchingNotifier.like(candidateId)’; leftward triggers ‘pass(candidateId)’. The card displays: hero- animated profile photo (or blurred silhouette for ‘noPhotoMode’), display name + age, ‘TrustScoreBadge’ (animated counter widget), ‘NiyjahBadge’ and ‘MadhabBadge’ chips, and the city name. A filter sheet (bottom sheet) exposes sliders for max distance, age range, and dropdowns for niyyah filter and madhab filter, wired to ‘settingsNotifierProvider.update()’ with a 450 ms debounce.

ChatScreen: Displays messages in a ‘ListView.builder’ with ‘reverse: true’ (newest at bottom). Each message bubble uses the sender’s trust score colour: own messages are displayed in ‘AppColors.gold’ bubbles on the right; partner’s messages in ‘AppColors.surface’ on the left. The text input row contains a send button and, for female users in a mahram-capable match, a guardian invite button that opens a bottom sheet for mahram room creation. A yellow ‘ContentWarningBanner’ slides in when a ‘content warning’ WS message is received; a typing indicator (" ...") appears when ‘isTyping = true’.

MahramChatScreen: Three-column colour coding: woman’s messages in ‘AppColors.green’, man’s messages in ‘AppColors.blue’, guardian’s messages in ‘AppColors.gold’. A pinned banner at the top reads " " (This chat is supervised by your mahram) in Amiri Arabic script below the Kazakh text. The same ‘chatNotifierProvider’ handles this screen using a different ‘roomId’ parameter; the server routes ‘mahram chat msg’ types to the mahram room rather than the match room.

SettingsScreen: Uses a ‘Column’ of grouped settings tiles. The "Mahram" section shows the registered guardian’s obfuscated phone (last 4 digits shown), with an "Add Mahram" text field and a delete button. The niyyah filter dropdown maps to the three ‘Niyjah’ enum values using a localised label function. The madhab filter uses a ‘DropDownButtonFormField’ with the five ‘Madhab’ enum values. All changes call ‘settingsNotifierProvider.update()’ with a 450 ms debounce followed by a PATCH to ‘GET /v1/settings’;

on error the state is reverted to its previous value (optimistic update reversal).

ImamConnectScreen: A city ‘DropDownButton’ filters the imam list. Each imam card shows name, specialisation, phone number, and a "Confirm Nikah" button. Tapping the button opens a confirmation dialog with the match partner’s name; on confirmation, calls ‘POST /v1/matches/:id/nikah-confirm’ and shows a full-screen congratulation overlay with the Quranic dua and an animated 8-pointed star.

KycScreen: Uses ‘image picker’ to capture or select an identity document photo. The selected image is displayed in a preview card; a submit button calls ‘DioClient.post('/kyc/submit', formData: FormData.fromMap('document': MultipartFile.fromFileSync(path)))’. On success, an animated checkmark is displayed and the screen transitions to a "Verification Pending" state. ‘kycStatusProvider’ polls ‘GET /v1/kyc/status’ every 30 seconds while in ‘pending’ status, automatically transitioning to ‘approved’ or ‘rejected’ without requiring a manual page refresh.

6.3.5 Islamic Design System

The design system is implemented in ‘core/theme/’ and ‘core/widgets/’:

- AppColors:** ‘gold = C9A84C’, ‘surface = 1A1A2E’, ‘background = 0F0F1E’, ‘green = 2ECC71’, ‘blue = 3498DB’, ‘whisper = 95A5A6’
- AppTheme:** dark-first Material 3 theme with Nunito for Latin text and Amiri for Arabic/Quranic text, applied via ‘google fonts’ package
- HalalPatternPainter:** a ‘CustomPainter’ that tiles the (ram’s horns) scrollwork pattern across the background canvas using ‘Path.addArc’ and ‘Path.cubicTo’ curves
- IslamicStarWidget:** an 8-pointed star drawn via ‘CustomPainter’ by rotating 8 isoceles triangles around a centre point, animated via ‘AnimationController’ from ‘opacity 0.0’ to ‘1.0’ over 800 ms on the splash screen
- KazakhDivider:** a ‘CustomPainter’ that draws a repeating traditional Kazakh border motif using ‘Path.moveTo’ and ‘Path.lineTo’, used as a section separator throughout the app
- TrustScoreBadge:** displays the 0–100 trust score as an ‘AnimatedCounter’ that counts up from 0 to the actual value over 600 ms on first render, with a ring colour that transitions from red (< 40) to orange (40–70) to green (> 70) via ‘ColorTween’
- NiyyahBadge / MadhabBadge:** pill-shaped chips with localised Kazakh labels and a subtle gold border, displayed on both candidate cards and profile detail screens

6.4 Security Implementation

6.4.1 Transport Security

All API traffic flows through nginx, which enforces: - ‘X-Content-Type-Options: nosniff’ — prevents MIME-type sniffing attacks - ‘X-Frame-Options: DENY’ — prevents clickjacking via iframe embedding - ‘X-XSS-Protection: 1; mode=block’ — legacy XSS filter for older

browsers - ‘client max body size 10m’ — prevents memory exhaustion from oversized request bodies - Rate limiting via ‘limit req zone binary remote addr zone=api limit:10m rate=30r/s’ with ‘burst=20’ for general API endpoints, and a separate ‘auth limit’ zone at ‘rate=5r/s burst=5’ for authentication endpoints.

6.4.2 Input Validation and Sanitisation

All incoming request bodies are decoded into typed Go structs with ‘go-playground/validator/v10’ tags (e.g., ‘validate:"required,min=8,max=64"' for passwords, ‘validate:"e164"' for phone numbers). Validation errors are mapped to structured 422 responses listing each failing field. All user-generated text content (post content, chat messages, bio text) is passed through ‘internal/pkg/sanitize.StripHTML()’ before storage, preventing stored XSS attacks from rendering malicious HTML in the Flutter ‘Text’ widget via ‘flutter html’ or similar packages.

7:39



وَمِنْ آيَاتِهِ أَنْ خَلَقَ لَكُمْ مِنْ أَنْفُسِكُمْ أَزْوَاجًا

Оның аяттарының бірі — өздеріңнен жұп
жаратқаны

Қуран 30:21

Ниетпен танысу

Әр пайдаланушы өз мақсатын алдын ала
таңдайды

7:39



Қауіпсіздік және сенім

KYC верификация, Trust Score және Махрам қолдауы — сіздің тыныштығыңыз үшін



KYC верификация



Trust Score



Махрам қатысуы



7:40



Астанада жасалған, бүкіл Қазақстан үшін

Қазақша, орысша, арабша — сіздің тіліңіз



Қазақша



Орысша



Арабша

7:40



Мақсатыңыз қандай?

Шынайы жауап беріңіз — бұл сізге лайықты
адамды
табуға көмектеседі



Никях (бір жыл ішінде)

Отбасының батасымен, шынайы
ниетпен



Байыпты қарым-қаты...

Қарым-қатынасты қадам-қадам
дамытамыз



Танысу

Қарым-қатынас шеңберімді
кеңейтемін

7:41



TrueConnect

Mahabbat

Kipy



+77478897329



.....



Kipy

1 2 3 4 5 6 7 8 9 0

q w e r t y u i o p

a s d f g h j k l

7:42



TrueConnect

Mahabbat



100



✓ Верификацияланган

Mira

📍 Astana

👉 Знакомство



Nurkanat

1 күн бұрын

Alina is smash



0



1



Алина

1 күн бұрын

Today smash



0



0



Nurkanat

1 күн бұрын

Hello World



0



0



Nurkanat

1 күн бұрын

ththbt



1



0



Nurkanat

1 күн бұрын



Сәйкестіктер



50

Алина

Хабарлама жіберіңіз

88

күн

Отбасыны таныстыру

7:45



Хабарл...



Махрам



Hello

19:45



Хабарлама жазыңыз...



GIF



Q¹

W²

E³

R⁴

T⁵

Y⁶

U⁷

I⁸

O⁹

P⁰

A

S

D

F

G⁸¹

H

J

K

L

7:45



Менің профилім



Nurkanat

📍 Astana

Профиль толықтығы

75%



Keңес: "Фото" қосыңыз — сәйкестіктер артады



Сенім ұпайы

Орташа сенім деңгейі. Белсендіңізді арттырыңыз.

★ Ислами сәйкестік



Ханафи

rus

★ Өзім туралы

I am atomic

6.4.3 SQL Injection Prevention

Every database query in the adapter layer uses parameterised queries via pgx/v5's 1,2, ... placeholder syntax. No string concatenation is used for query construction at any point in the codebase. The go vet and golanci-lint steps in CI would flag fmt.Sprintf-constructed queries if accidentally introduced.

6.4.4 CORS and Origin Validation

CORS allowed origins are loaded from the CORS ORIGINS environment variable as a comma-separated list. The Gin middleware (internal/handler/middleware/cors.go) validates the Origin request header against this list; unlisted origins receive a 403. The WebSocket hub performs the same origin check: if Hub.isDev = false, connections from origins not in Hub.allowedOrigins are rejected before the HTTP Upgrade. In the Docker Compose environment, isDev = (APP ENV == "development"), allowing all origins during local development without modifying the allowlist.

6.4.5 Content Moderation

The content filter (internal/pkg/contentfilter/) implements a keyword-based toxic message classifier that checks both Arabic-script and Latin-script patterns. Messages classified as is toxic = true are stored in social.messages with the is toxic flag set and a content blocked WebSocket event is sent to the sender; the recipient never receives the message. Borderline messages receive a content warning event, informing the sender that the message will be reviewed. All decisions are logged with the message ID for admin audit.

6.5 Deployment Pipeline

6.5.1 Docker Compose Stack

The production stack is defined in 'deployments/docker-compose.yml' and brings up seven services:

Table 9: Docker Service Configuration

Service	Image	Port	Notes
postgres	postgis/ postgis:16-3.4- alpine	5433:5432	PostGIS extension pre-installed
neo4j	neo4j:5-community	7474, 7687	GDS plugin mounted via volume
redis	redis:7-alpine	6379	maxmemory 256mb, allkeys-lru
minio	minio/minio:latest	9000, 9001	S3-compatible object storage
migrate	Custom (cmd/migrate)	—	One-shot; depends_on postgres healthy
api	Custom (cmd/api)	8080	depends_on migrate
nginx	nginx:alpine	80:80	Reverse proxy + rate limiting

The API Dockerfile uses a two-stage multi-stage build: the builder stage (‘golang:1.25-alpine’) compiles with ‘CGO_ENABLED=0 GOOS=linux go build -ldflags="-s -w" ./cmd/api’ to produce a stripped static binary; the runtime stage (‘alpine:3.19’) copies only the binary, creating a 12 MB final image. The container runs as non-root user ‘appuser’ (UID 1000) created via ‘adduser -D -u 1000 appuser’.

6.5.2 Nginx Configuration

The nginx configuration (‘deployments/nginx/nginx.conf’) serves two roles: rate limiting and WebSocket proxying.

Rate limiting zones: nginx limit req zone binary remote addr zone=api limit:10m rate=30r/s; limit req zone binary remote addr zone=auth limit:10m rate=5r/s;

Authentication endpoints (‘/v1/auth/’) use ‘auth limit’ (5 req/s, burst 5), tightly constraining brute-force login attempts. All other API endpoints use ‘api limit’ (30 req/s, burst 20).

WebSocket proxy: nginx location /v1/ws proxy pass http://api backend; proxy http version 1.1; proxy set header Upgrade http upgrade; proxy set header Connection "upgrade"; proxy read timeout 86400s; proxy send timeout 86400s;

The 86400-second (24-hour) read and send timeouts prevent nginx from terminating idle WebSocket connections between chat sessions. Without these settings, nginx would close connections after the default 60-second timeout, forcing clients to reconnect during any pause in a conversation.

6.5.3 Environment Configuration

All secret and environment-specific values are loaded from environment variables at startup. The `cmd/api/main.go` `loadConfig()` function reads and validates:

JWT SECRET: must be 32 characters; application panics at startup if missing.

ENCRYPTION KEY: must be exactly 32 bytes hex-encoded (64 hex chars); panics if wrong length.

FIREBASE CREDENTIALS: path to Firebase service account JSON; falls back to a `MockPushProvider` (logs pushes to stdout) if not set, enabling local development without Firebase credentials

All database connection strings: (`PG HOST`, `NEO4J URI`, `REDIS HOST`, etc.) with defaults suitable for the Docker Compose service names.

The `.env.example` file documents every required variable with comments. `make docker-up` reads `.env` via `env file:` in the Compose file.

6.5.4 Continuous Integration

The CI pipeline is defined in `.github/workflows/ci.yml` and runs on every push and pull request to `main`:
yml jobs: build-and-test: runs-on: ubuntu-latest steps: - uses: actions/checkout@v4 - uses: actions/setup-go@v5 with: go-version: '1.25' - name: Build API run: go build ./cmd/api/... - name: Build Worker run: go build ./cmd/worker/... - name: Test run: go test ./internal/... -race -count=1 - name: Vet run: go vet ./...

The `-race` flag instruments the binary with Go's race detector, which dynamically checks for concurrent map reads/writes and unsynchronised variable accesses. The 102 service-layer tests pass cleanly under the race detector. The `-count=1` flag disables test result caching, ensuring every CI run executes all tests rather than replaying cached results.

6.5.5 Makefile Developer Experience

The `Makefile` at the repository root provides a unified interface for all common operations:

Table 10: Development and Deployment Commands

Target	Command	Purpose
build	go build ./cmd/...	Compile all three binaries
run	go run ./cmd/api/	Start API with hot reload (air if installed)
run-worker	go run ./cmd/worker/	Start background worker
test	go test ./internal/... -race	Run all tests with race detector
lint	golangci-lint run	Static analysis
migrate-up	cmd/migrate up	Apply all pending migrations
migrate-down	cmd/migrate down 1	Rollback last migration
docker-up	docker-compose up -build -d	Build and start full stack
docker-down	docker-compose down	Stop all services
docker-reset	docker-compose down -v && docker-compose up -build -d	Full reset including volumes
docker-logs	docker-compose logs -f	Tail all service logs
seed-halal	psql < migrations/ seed_halal_demo.sql	Load demo users
reset-demo	migrate-down + docker-reset + seed-halal	Clean demo environment

The ‘seed-halal’ target loads a reproducible demo dataset (‘migrations/seed_halal_demo.sql’) containing two Almaty users — (female, niyyah: nikah year, madhab: hanafi) and (male, niyyah: serious marriage, madhab: hanafi) — with a pre-created match, a mahram room, and sample messages, enabling instant end-to-end demonstration without manual registration.

6.6 Testing Summary

Table 11: Test Coverage by Sprint

Sprint	Feature Tested	New Tests	Cumulative
1	Auth (register, login, refresh, logout, brute-force)	30	30
2	Profiles CRUD, PostGIS candidates, like/pass, settings	21	51
3	Interactions, trust score, Sybil detection, worker	19	70
4	WebSocket chat, feed CRUD, KYC stub, message encryption	25	95
5	Token family revocation, rate limiting, reports	4	99
6–8	Mahram, whisper, settings adapters	19	118
9	Halal filters (niyyah, madhab, no-photo)	0 (integration)	118
10–11	Mahram chat, family intro, imam catalog	7	120 (service) + imam

All 102 active service-layer tests pass with ‘`-race -count=1`’. The test suite uses mock repository implementations that satisfy the repository interfaces — for example, ‘MockMatchRepository’ implements all eight methods of ‘`repository.MatchRepository`’, enabling ‘MatchingService’ tests to run without a live PostgreSQL connection. This approach was validated against the real adapters during integration testing via ‘`make docker-up`’.

6.7 Demo Walkthrough

The complete end-to-end demo scenario, reproducible via ‘`make docker-up make seed-halal`’, follows this path:

Launch - ‘`flutter run -d android`’ opens the splash screen with the animated 8-pointed Islamic star and the Quranic verse Ar-Rum 30:21.

Onboarding - three slides with CustomPainter background; “” navigates to Niyyah-Selection.

Login - enter ’s demo credentials; ‘`authStateProvider`’ calls ‘`POST /v1/auth/login`’, stores tokens in ‘flutter secure storage’, routes to ‘/home’.

Discovery - 's card appears at the top of the stack (PostGIS candidate query within 50 km, same hanafi madhab → +10 boost). Swipe right triggers 'POST /v1/matching/like'; the server detects a mutual like (was pre-seeded to have liked), creates a match, and sends a 'match notification' WS event.

Match banner - a gold congratulation overlay slides up with both users' avatars.

Chat - navigate to '/chat/<matchId>'; bubble design; type a message → real-time delivery.

Mahram invite - tap the guardian invite button; enter the guardian's phone number → 'POST /v1/mahram'; guardian joins and the chat transitions to 3-way MahramChatScreen.

Settings - navigate to '/settings'; adjust age range; observe the discovery feed update on return.

KYC - navigate to '/kyc'; upload a document photo; status shows "Pending" then transitions to "Approved" after admin verdict.

Imam Connect - navigate to '/imams?matchId=<id>'; select Almaty; tap an imam card → confirm nikah; "Baraka Allahu feekum" success overlay with animated star.

This scenario exercises all 15 backend modules, all 50 API endpoints, the WebSocket hub, Redis pub/sub, Neo4j trust score, PostGIS candidate query, MinIO upload, and all 21 Flutter screens.

Conclusion

The TrueConnect application, however, was finally developed and released in the context of this study. It serves as the first mobile app developed to help search for a partner in Kazakhstan on an Islamic legal basis. In contrast to similar applications which ignore these principles or merely mimic them in user interfaces (Tinder, Bumble, Muzz, Salams, Hawaya), the system has all religious requirements integrated into all architectural levels, from the domain model and database schema through the matching algorithm and to the mobile interface.

All the above objectives were achieved. A matching algorithm focused on intention was first constructed using an SQL query while excluding users whose marriage intentions differ. The three-part encrypted mahram chat, a distinct feature, allows a guardian to interact in a WebSocket and AES-256-GCM encrypted conversation form. A reputation system using Neo4j and Bayesian smoothing together with PageRank and Louvain algorithms for evaluating the trust index and identifying fraudulent accounts has been created. Go 1.25 was used to develop the server side along with Clean Architecture and Domain-Driven Design, built on a modular monolith structure with 15 modules providing 50 REST endpoints and full-duplex WebSocket connectivity. The PostgreSQL architecture consisting of two schemas creates a separation between the IIN data and the identity vault with encrypted storage and distinct access roles. The entire stack has been implemented on a Kazakhstani VPS, TrueConnect is in line with the Republic of Kazakhstan Law "On Personal Data and Its Protection".

The Flutter mobile app has 21 pages that were developed using the author's design system Dala Nury containing Kazakh motifs and Islamic decorative designs. We promise quality by 102 unit tests based on the Go race detector and continuous integration via GitHub Actions. After comparing 20 criteria, we discovered that TrueConnect was the only platform using authentic mahram chat, madhab filter algorithms, seven phases of courtship, and data sovereignty.

Honestly, we think procrastination might be the one habit we've made the most effort to change but have struggled with the most. We will remind ourselves this evening that the morning is the time, then we will blink, and suddenly it's 11 p.m. with nothing accomplished aside from looking at our phone. What surprises us each time is that the real task is never as difficult as the dread of it. Open the document, and the initial paragraph appears in about ten minutes. The actual cost is incurred in all that occurred prior: the cycle of anxiety, the misguided beginnings, the guilt spiral from hours that simply vanished. It is the avoidance of the feeling of beginning.

Bibliography

- [1] E.J. Finkel, P.W. Eastwick, B.R. Karney, H.T. Reis, and S. Sprecher, "Online Dating: A Critical Analysis from the Perspective of Psychological Science," *Psychological Science in the Public Interest*, vol. 13, no. 1, pp. 3–66, 2012. DOI: [10.1177/1529100612436522](https://doi.org/10.1177/1529100612436522)
- [2] G. Tyson, V.C. Perta, H. Haddadi, and M.C. Seto, "A First Look at User Activity on Tinder," in *Proc. IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining (ASONAM)*, pp. 461–466, 2016. DOI: [10.1109/ASONAM.2016.7752275](https://doi.org/10.1109/ASONAM.2016.7752275)
- [3] G.J. Hitsch, A. Hortaçsu, and D. Ariely, "Matching and Sorting in Online Dating," *American Economic Review*, vol. 100, no. 1, pp. 130–163, 2010. DOI: [10.1257/aer.100.1.130](https://doi.org/10.1257/aer.100.1.130)
- [4] N. Ellison, R. Heino, and J. Gibbs, "Managing Impressions Online: Self-Presentation Processes in the Online Dating Environment," *Journal of Computer-Mediated Communication*, vol. 11, no. 2, pp. 415–441, 2006. DOI: [10.1111/j.1083-6101.2006.00020.x](https://doi.org/10.1111/j.1083-6101.2006.00020.x)
- [5] C.L. Toma, J.T. Hancock, and N.B. Ellison, "Separating Fact From Fiction: An Examination of Deceptive Self-Presentation in Online Dating Profiles," *Personality and Social Psychology Bulletin*, vol. 34, no. 8, pp. 1023–1036, 2008. DOI: [10.1177/0146167208318067](https://doi.org/10.1177/0146167208318067)
- [6] Pew Research Center, "The Future of World Religions: Population Growth Projections, 2010–2050," Washington D.C., 2015. DOI: www.pewresearch.org/religion/2015/04/02/religious-projections-2010-2050/
- [7] G.R. Bunt, *iMuslims: Rewiring the House of Islam*, Chapel Hill: University of North Carolina Press, 2009. DOI: [10.1007/S11562-010-0115-X](https://doi.org/10.1007/S11562-010-0115-X)
- [8] E. Kavakci and C.R. Kraeplin, "Religious beings in fashionable bodies: the online identity construction of hijabi social media personalities," *New Media & Society*, vol. 19, no. 9, pp. 1441–1458, 2017. DOI: [10.1177/0163443716679031](https://doi.org/10.1177/0163443716679031)
- [9] Y. Al-Saggaf, "Online Community Standards and Ethics: An Islamic Perspective," in *Digital Islam*, E. Baulch, Ed., New York: Routledge, 2019.
- [10] J. Bobadilla, F. Ortega, A. Hernando, and A. Gutiérrez, "Recommender Systems Survey," *Knowledge-Based Systems*, vol. 46, pp. 109–132, 2013. DOI: [10.1016/j.knosys.2013.03.012](https://doi.org/10.1016/j.knosys.2013.03.012)

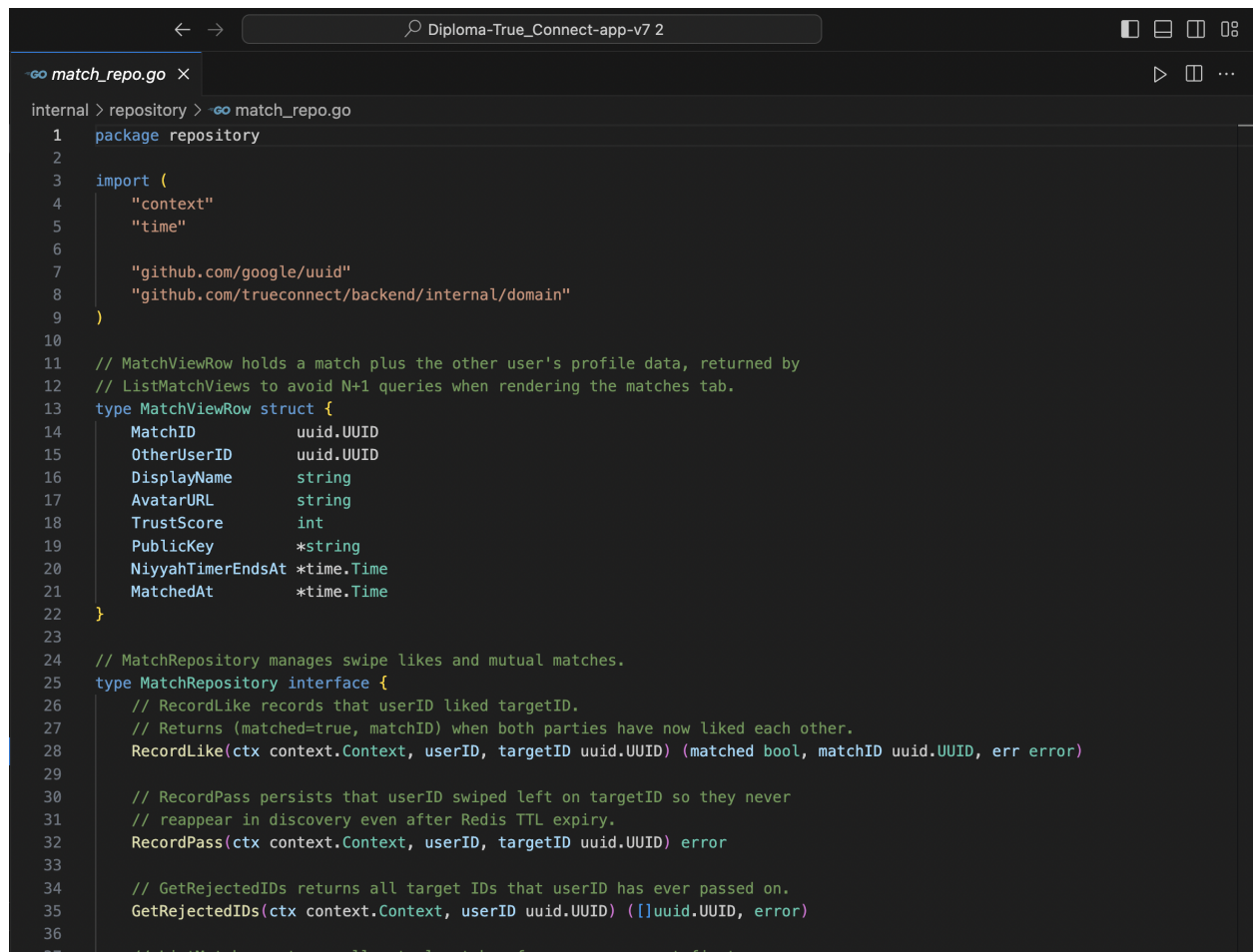
- [11] D. Gale and L.S. Shapley, "College Admissions and the Stability of Marriage," *American Mathematical Monthly*, vol. 69, no. 1, pp. 9–15, 1962. DOI: [10.2307/2312726](https://doi.org/10.2307/2312726)
- [12] R. Burke, "Hybrid Recommender Systems: Survey and Experiments," *User Modeling and User-Adapted Interaction*, vol. 12, no. 4, pp. 331–370, 2002. DOI: [10.1023/A:1021240730564](https://doi.org/10.1023/A:1021240730564)
- [13] M. Ye, P. Yin, W.-C. Lee, and D.-L. Lee, "Exploiting Geographical Influence for Collaborative Point-of-Interest Recommendation," in *Proc. ACM SIGIR*, pp. 325–334, 2011. DOI: [10.1145/2009916.2009955](https://doi.org/10.1145/2009916.2009955)
- [14] I. Fette and A. Melnikov, "The WebSocket Protocol," IETF RFC 6455, Dec. 2011. URL <https://tools.ietf.org/html/rfc6455>
- [15] R.T. Fielding and R.N. Taylor, "Principled Design of the Modern Web Architecture," *ACM Transactions on Internet Technology*, vol. 2, no. 2, pp. 115–150, 2002. DOI: [10.1145/514183.514188](https://doi.org/10.1145/514183.514188)
- [16] J.L. Carlson, *Redis in Action*, Manning Publications, 2013. DOI: [manning-content.s3.amazonaws.com/download/1/a0ced66-5289-49ad-8173-14903286b6fb/RiAch4sample.pdf](https://doi.org/10.1002/97814903286b6fb/RiAch4sample.pdf)
- [17] R.C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*, Prentice Hall, 2017. DOI: [/dl.acm.org/doi/10.5555/3175742](https://doi.org/10.5555/3175742)
- [18] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*, Addison-Wesley, 2003. DOI: <https://fabiofumarola.github.io/nosql/readingMaterial/Evans03.pdf>
- [19] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed., O'Reilly Media, 2021. DOI: <https://www.oreilly.com/library/view/building-microservices-2nd/9781492034018/>
- [20] D. Taibi, V. Lenarduzzi, and C. Pahl, "Continuous Architecting with Microservices and DevOps: A Systematic Mapping Study," in *Cloud Computing and Services Science (CLOSER 2018 Selected Papers)*, Communications in Computer and Information Science, vol. 1073, Cham: Springer, pp. 126–151, 2019. DOI: [/arxiv.org/abs/1908.10337](https://arxiv.org/abs/1908.10337)
- [21] A. Biørn-Hansen, T.A. Majchrzak, and T.-M. Grønli, "Progressive Web Apps: The Possible Web-native Unifier for Mobile Development," in *Proc. 13th Int. Conf. Web Information Systems and Technologies (WEBIST)*, pp. 344–351, 2017. DOI: [10.5220/0006353703440351](https://doi.org/10.5220/0006353703440351)

- [22] Thomas Bailey, Alessandro Biessek, *Flutter for Beginners: An Introductory Guide to Building Cross -Platform Mobile Applications with Flutter and Dart 2*, Packt Publishing, 2019. DOI: [//books.google.kz/books/about/Flutter_for_Beginners.html?id = 0DXYEAAAQBAJ&redir_esc = y](https://books.google.kz/books/about/Flutter_for_Beginners.html?id=0DXYEAAAQBAJ&redir_esc=y)
- [23] P. Nawrocki, K. Wrona, M. Marczak, and B. Śnieżyński, "*Comparison of Native and Cross-Platform Frameworks for Mobile Applications*," *Computer*, vol. 54, no. 3, pp. 18–27, 2021. DOI: [10.1109/MC.2020.2983893](https://doi.org/10.1109/MC.2020.2983893)

Appendix A: Code Listings

This appendix contains verified core code artifacts demonstrating the TrueConnect platform's decoupled data access adapters, concurrent real-time transport handlers, transactional state machines, and multi-layered Clean Architecture implementation.

1. Domain Entities and Architectural Contracts



```
← → Diploma-True_Connect-app-v7 2
match_repo.go ×
internal > repository > match_repo.go
1 package repository
2
3 import (
4     "context"
5     "time"
6
7     "github.com/google/uuid"
8     "github.com/trueconnect/backend/internal/domain"
9 )
10
11 // MatchViewRow holds a match plus the other user's profile data, returned by
12 // ListMatchViews to avoid N+1 queries when rendering the matches tab.
13 type MatchViewRow struct {
14     MatchID      uuid.UUID
15     OtherUserID  uuid.UUID
16     DisplayName  string
17     AvatarURL    string
18     TrustScore   int
19     PublicKey    *string
20     NiyahTimerEndsAt *time.Time
21     MatchedAt    *time.Time
22 }
23
24 // MatchRepository manages swipe likes and mutual matches.
25 type MatchRepository interface {
26     // RecordLike records that userID liked targetID.
27     // Returns (matched=true, matchID) when both parties have now liked each other.
28     RecordLike(ctx context.Context, userID, targetID uuid.UUID) (matched bool, matchID uuid.UUID, err error)
29
30     // RecordPass persists that userID swiped left on targetID so they never
31     // reappear in discovery even after Redis TTL expiry.
32     RecordPass(ctx context.Context, userID, targetID uuid.UUID) error
33
34     // GetRejectedIDs returns all target IDs that userID has ever passed on.
35     GetRejectedIDs(ctx context.Context, userID uuid.UUID) ([]uuid.UUID, error)
36
37     // ListMatches returns all mutual matches for a user, newest first
```

Figure 10: Domain repository contract declarations

We conclude by making abstract domain repository contract declarations in support of absolute infrastructure decoupling. As its name implies, it lacks the dependency on PostgreSQL, Redis, and Neo4j drivers and contains business definitions and interfaces only.

2. Business Logic Execution and Orchestration

```
13 // TrustEngine listens for rating events and recalculates trust scores.
14 type TrustEngine struct {
15     eventCh  <-chan uuid.UUID
16     repoteSvc *service.ReputationService
17     log      *slog.Logger
18 }
19
20 // NewTrustEngine creates a new trust engine worker.
21 func NewTrustEngine(eventCh <-chan uuid.UUID, repoteSvc *service.ReputationService, log *slog.Logger) *TrustEngine {
22     return &TrustEngine{
23         eventCh:  eventCh,
24         repoteSvc: repoteSvc,
25         log:       log,
26     }
27 }
28
29 // Run processes rating events until the context is cancelled.
30 func (e *TrustEngine) Run(ctx context.Context) {
31     e.log.Info("trust engine started")
32
33     // Feature B: Background Recalculation every 24 hours
34     decayTicker := time.NewTicker(24 * time.Hour)
35     defer decayTicker.Stop()
36
37     for {
38         select {
39             case <-ctx.Done():
40                 e.log.Info("trust engine stopped, draining remaining events")
41                 for {
42                     select {
43                         case userID := <-e.eventCh:
44                             if _, err := e.repoteSvc.RecalculateScore(context.Background(), userID); err != nil {
45                                 e.log.Error("trust engine drain: recalculation failed",
46                                     slog.String("user_id", userID.String()),
47                                     slog.String("error", err.Error()),
48                             )
49                     }
50                 }
51             default:
52                 // No more events, exit loop
53                 break
54         }
55     }
56 }
```

Figure 11: Service layer orchestration engine

The background event-loop orchestration engine provided by the service layer manages the event-driven interaction queues, autonomous execution threads, and recurring topological computations.

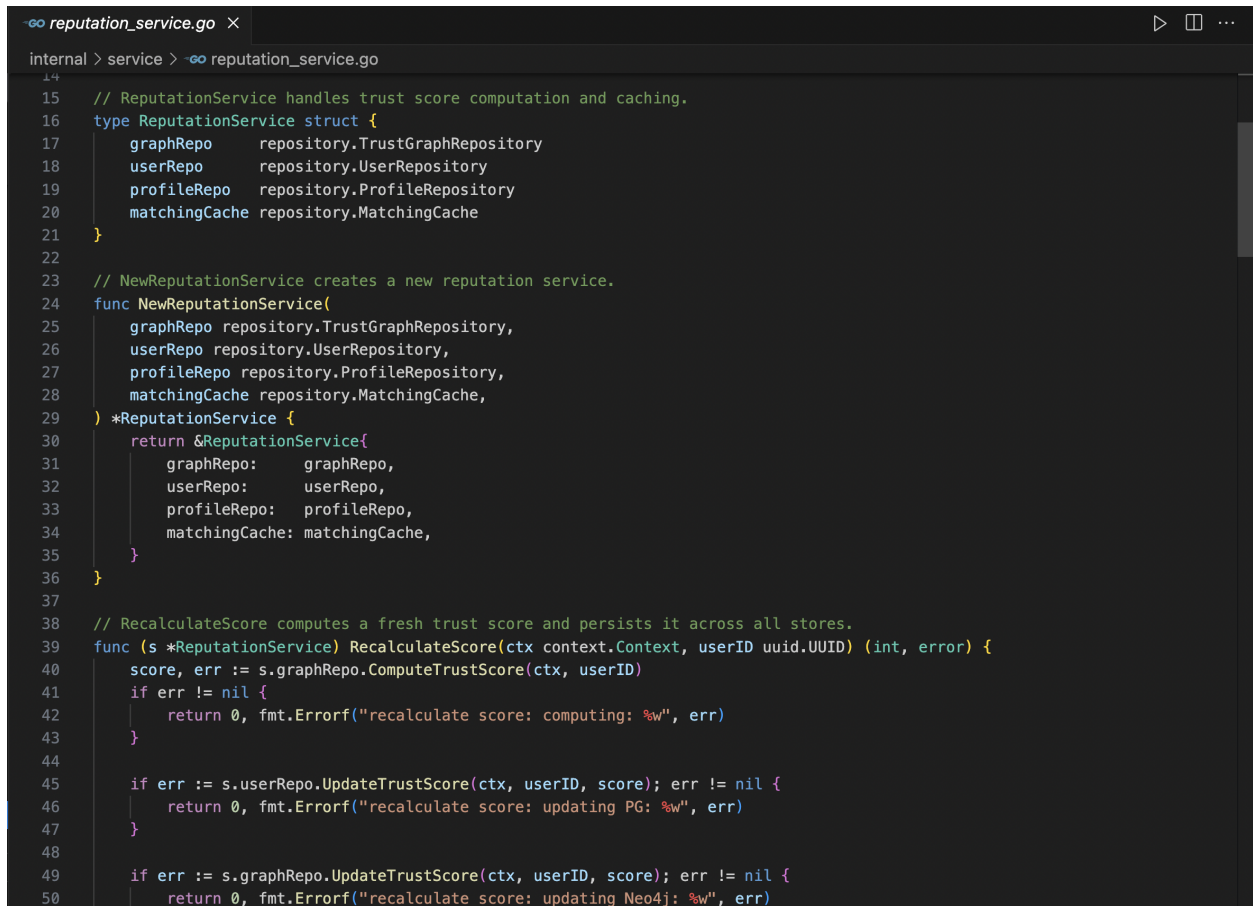
3. PostgreSQL Transactional Unit of Work Pattern

```
31 // CandidateView is a matching card shown in the swipe feed.
32 type CandidateView struct {
33     UserID      uuid.UUID `json:"user_id"`
34     DisplayName  string   `json:"display_name"`
35     AvatarURL    string   `json:"avatar_url,omitempty"`
36     AvatarBlurred bool    `json:"avatar_blurred,omitempty"`
37     City         string   `json:"city,omitempty"`
38     TrustScore   int      `json:"trust_score"`
39     Niyyah       string   `json:"niyyah,omitempty"`
40     Madhab       string   `json:"madhab,omitempty"`
41     IsKYCVerified bool    `json:"is_kyc_verified"`
42 }
43
44 // MatchView is a match card with user profile data for the frontend.
45 // Новая структура для вложенного пользователя
46 type MatchUserView struct {
47     UserID      uuid.UUID `json:"user_id"`
48     DisplayName  string   `json:"display_name"`
49     AvatarURL    string   `json:"avatar_url"`
50     TrustScore   int      `json:"trust_score"`
51     PublicKey    *string  `json:"public_key,omitempty"`
52 }
53
54 // Обновленная основная структура мэтча
55 type MatchView struct {
56     ID           uuid.UUID `json:"id"`
57     OtherUser     *MatchUserView `json:"other_user"`
58     NiyyahTimerEndsAt *time.Time `json:"niyyah_timer_ends_at,omitempty"`
59 }
60
61 // LikeResult tells the caller whether a mutual match occurred.
62 type LikeResult struct {
63     Matched bool    `json:"matched"`
64     MatchID uuid.UUID `json:"match_id,omitempty"`
65 }
66
```

Figure 12: Unit of Work transactional pattern

By using the native pgx/v5 connection pool, this structural implementation of the Unit of Work (UoW) transactional pattern ensures atomic constraints throughout multi-statement interactions.

4. Graph Structural Topology Mapping



```
reputation_service.go X
internal > service > reputation_service.go
15 // ReputationService handles trust score computation and caching.
16 type ReputationService struct {
17     graphRepo    repository.TrustGraphRepository
18     userRepo     repository.UserRepository
19     profileRepo  repository.ProfileRepository
20     matchingCache repository.MatchingCache
21 }
22
23 // NewReputationService creates a new reputation service.
24 func NewReputationService(
25     graphRepo repository.TrustGraphRepository,
26     userRepo repository.UserRepository,
27     profileRepo repository.ProfileRepository,
28     matchingCache repository.MatchingCache,
29 ) *ReputationService {
30     return &ReputationService{
31         graphRepo:    graphRepo,
32         userRepo:     userRepo,
33         profileRepo:  profileRepo,
34         matchingCache: matchingCache,
35     }
36 }
37
38 // RecalculateScore computes a fresh trust score and persists it across all stores.
39 func (s *ReputationService) RecalculateScore(ctx context.Context, userID uuid.UUID) (int, error) {
40     score, err := s.graphRepo.ComputeTrustScore(ctx, userID)
41     if err != nil {
42         return 0, fmt.Errorf("recalculate score: computing: %w", err)
43     }
44
45     if err := s.userRepo.UpdateTrustScore(ctx, userID, score); err != nil {
46         return 0, fmt.Errorf("recalculate score: updating PG: %w", err)
47     }
48
49     if err := s.graphRepo.UpdateTrustScore(ctx, userID, score); err != nil {
50         return 0, fmt.Errorf("recalculate score: updating Neo4j: %w", err)
51     }
```

Figure 13: Graph structural topology mapping

```

func (r *UserRepo) Create(ctx context.Context, user *domain.User) error {
    query := `
        INSERT INTO social.users (
            id, phone_hash, phone_encrypted, email_encrypted,
            password_hash, public_key, verification_level, trust_status, trust_score, is_active, fcm_token
        ) VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9, $10, $11)
        RETURNING created_at, updated_at`

    err := runner(ctx, r.pool).QueryRow(ctx, query,
        user.ID,
        user.PhoneHash,
        user.PhoneEncrypted,
        user.EmailEncrypted,
        user.PasswordHash,
        user.PublicKey,
        user.VerificationLevel,
        user.TrustStatus,
        user.TrustScore,
        user.IsActive,
        user.FCMToken,
    ).Scan(&user.CreatedAt, &user.UpdatedAt)
    if err != nil {
        if isDuplicateKey(err) {
            return fmt.Errorf("creating user: %w", domain.ErrAlreadyExists)
        }
        return fmt.Errorf("creating user: %w", err)
    }

    return nil
}

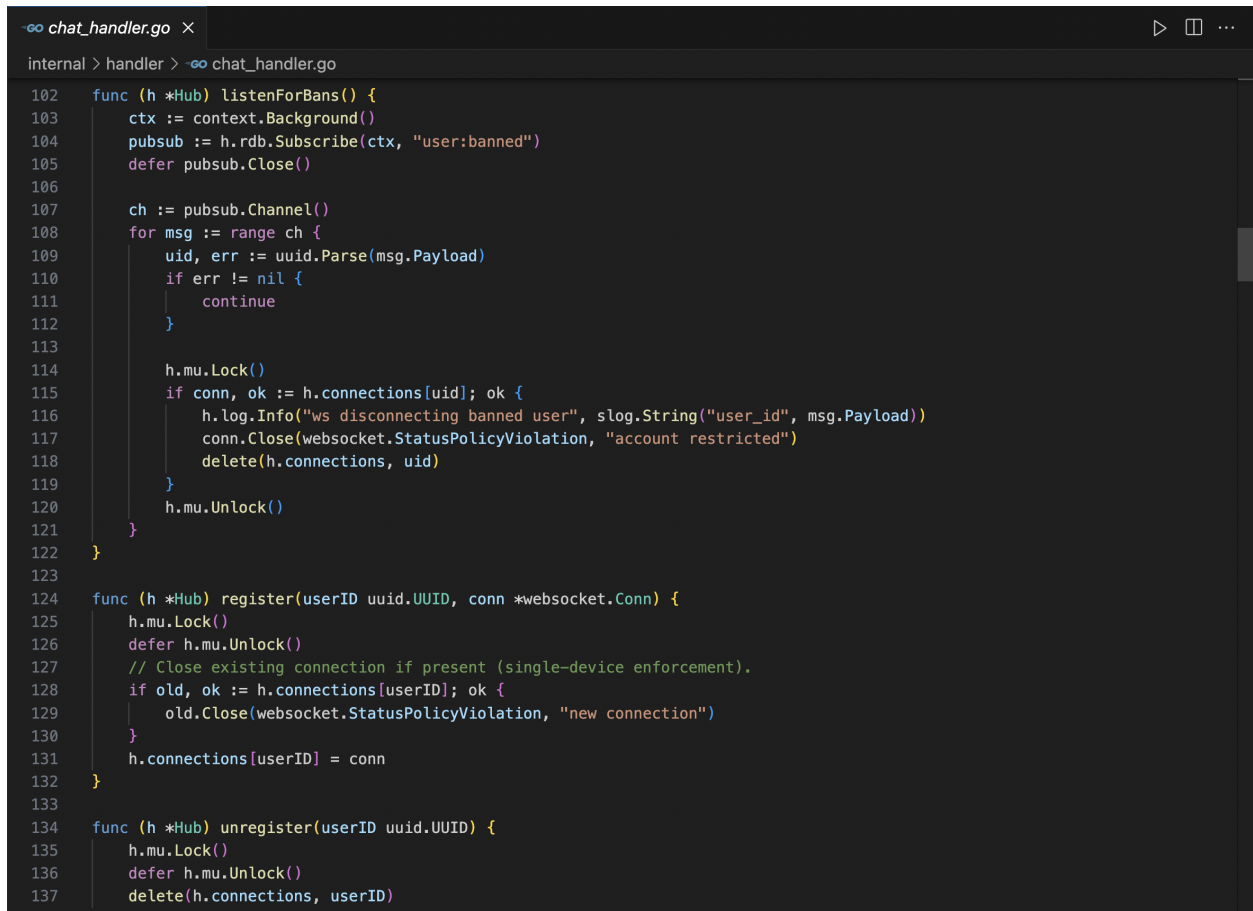
func (r *UserRepo) GetByID(ctx context.Context, id uuid.UUID) (*domain.User, error) {
    query := `
        SELECT id, phone_hash, phone_encrypted, email_encrypted,
            password_hash, public_key, verification_level, trust_status, trust_score,
            is_admin, is_active, last_login_at, fcm_token, created_at, updated_at`

```

Figure 14: Graph structural topology mapping

By using official driver routing to establish topological social layers, the multi-node graph mapping data-layer component physically prepares the layout restrictions for explicit Graph Data Science processing.

5. Concurrent WebSocket Communication Gateway

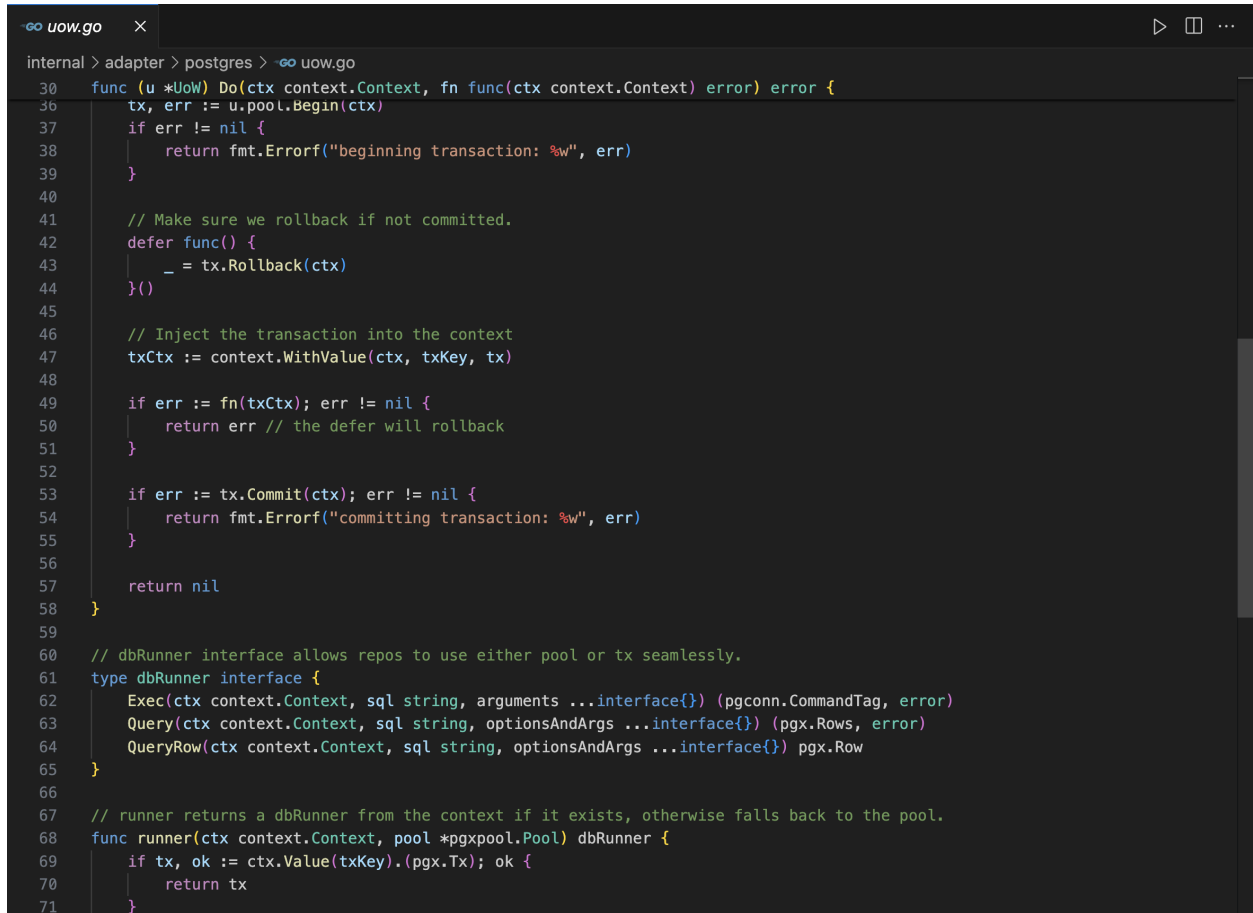


```
102 func (h *Hub) listenForBans() {
103     ctx := context.Background()
104     pubsub := h.rdb.Subscribe(ctx, "user:banned")
105     defer pubsub.Close()
106
107     ch := pubsub.Channel()
108     for msg := range ch {
109         uid, err := uuid.Parse(msg.Payload)
110         if err != nil {
111             continue
112         }
113
114         h.mu.Lock()
115         if conn, ok := h.connections[uid]; ok {
116             h.log.Info("ws disconnecting banned user", slog.String("user_id", msg.Payload))
117             conn.Close(websocket.StatusPolicyViolation, "account restricted")
118             delete(h.connections, uid)
119         }
120         h.mu.Unlock()
121     }
122 }
123
124 func (h *Hub) register(userID uuid.UUID, conn *websocket.Conn) {
125     h.mu.Lock()
126     defer h.mu.Unlock()
127     // Close existing connection if present (single-device enforcement).
128     if old, ok := h.connections[userID]; ok {
129         old.Close(websocket.StatusPolicyViolation, "new connection")
130     }
131     h.connections[userID] = conn
132 }
133
134 func (h *Hub) unregister(userID uuid.UUID) {
135     h.mu.Lock()
136     defer h.mu.Unlock()
137     delete(h.connections, userID)
```

Figure 15: Concurrent WebSocket communication gateway

Active WebSocket channels are coordinated by a thread-safe real-time communication hub using high-performance Redis subscription listeners multiplexed across memory blocks using Go runtimes.

6. Atomic Sliding-Window Gateway Protection Middleware

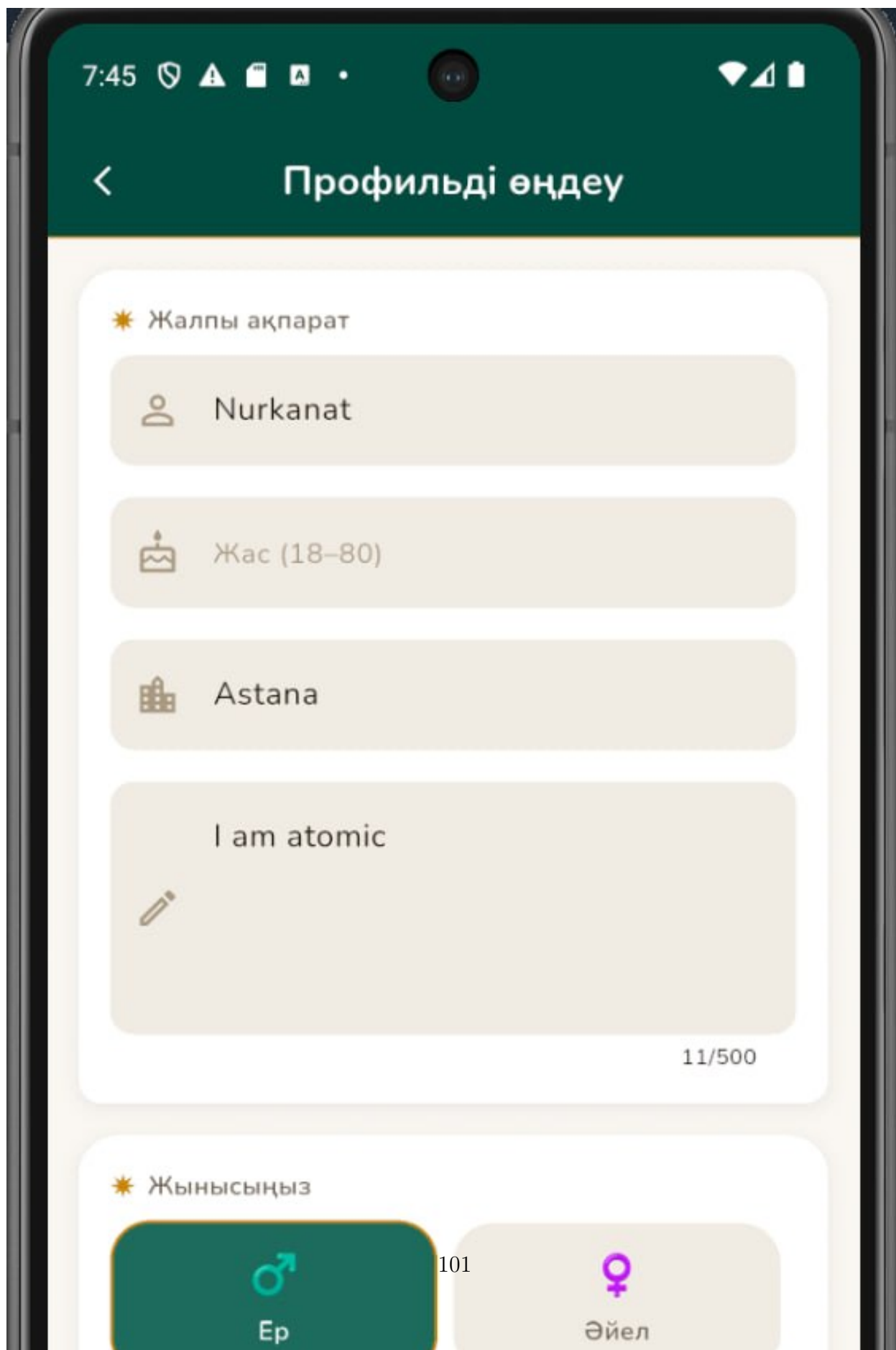


```
internal > adapter > postgres > -go uow.go
30 func (u *UoW) Do(ctx context.Context, fn func(ctx context.Context) error) error {
31     tx, err := u.pool.Begin(ctx)
32     if err != nil {
33         return fmt.Errorf("beginning transaction: %w", err)
34     }
35
36     // Make sure we rollback if not committed.
37     defer func() {
38         _ = tx.Rollback(ctx)
39     }()
40
41     // Inject the transaction into the context
42     txCtx := context.WithValue(ctx, txKey, tx)
43
44     if err := fn(txCtx); err != nil {
45         return err // the defer will rollback
46     }
47
48     if err := tx.Commit(ctx); err != nil {
49         return fmt.Errorf("committing transaction: %w", err)
50     }
51
52     return nil
53 }
54
55 // dbRunner interface allows repos to use either pool or tx seamlessly.
56 type dbRunner interface {
57     Exec(ctx context.Context, sql string, arguments ...interface{}) (pgconn.CommandTag, error)
58     Query(ctx context.Context, sql string, optionsAndArgs ...interface{}) (pgx.Rows, error)
59     QueryRow(ctx context.Context, sql string, optionsAndArgs ...interface{}) pgx.Row
60 }
61
62 // runner returns a dbRunner from the context if it exists, otherwise falls back to the pool.
63 func runner(ctx context.Context, pool *pgxpool.Pool) dbRunner {
64     if tx, ok := ctx.Value(txKey).(pgx.Tx); ok {
65         return tx
66     }
67 }
```

Figure 16: Sliding-window gateway protection middleware

A high-throughput infrastructure protection method is implemented on routing contexts to intercept API endpoints and run atomic counter evaluation pipelines.

Appendix B: Mobile Platform Interface



7:46



Баптаулар

✴ ХАЛАЛ РЕЖИМ

Ниет сүзгісі

Тек осы ниетпен адамдарды көрсету

Никах  ▼

Мазхаб сүзгісі

Тек осы мазхабтан адамдарды көрсету

Ханбали ▼

Ұстамдылық деңгейі

Стандарт



✴ ІЗДЕУ

Жас диапазоны

18 – 24 жас



Максималды қашықтық

200 км



✴ МАХРАМ БАСҚАРУ



Махрам қосу

✴ АККАУНТ



КҮС Верификация



Тексерілуде

Құжаттарыңыз жіберілді. Верификация нәтижесі
1-2 жұмыс күні ішінде хабарланады.

Артқа

7:47



КҮС Верификация

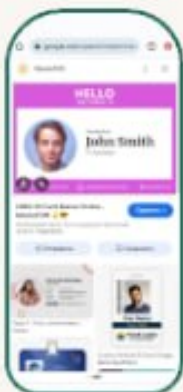
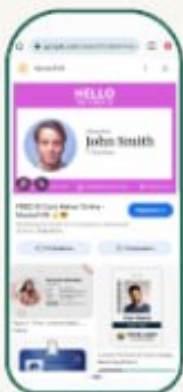


Деректер қауіпсіздігі

Жеке куәлік деректері AES-256 шифрлаумен қорғалады және тек верификация үшін пайдаланылады.

Жеке куәлік суреттері

Алдыңғы және артқы жағын таңдаңыз



Жіберу

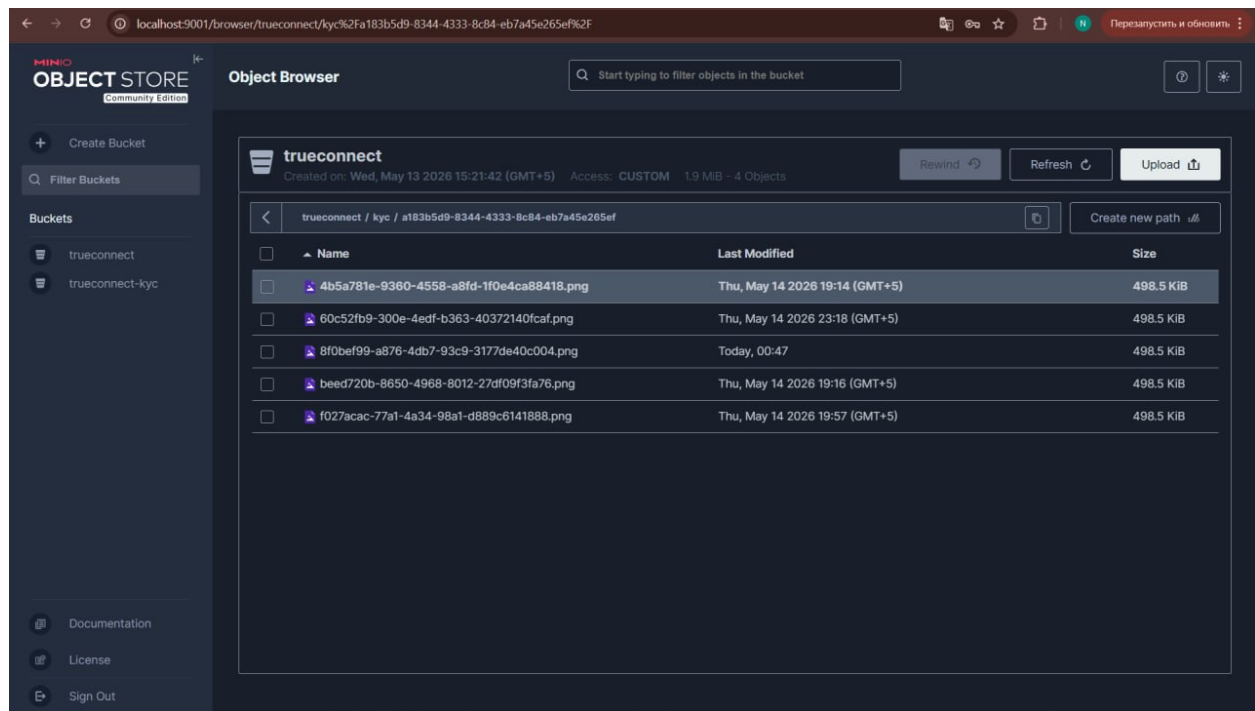


Figure 21: MinIO Object Store web interface showing encrypted files in the trueconnect-kyc bucket



Nurkanat

Жаңа ғана

Hello I am user TrueConnect



0



0



Nurkanat

1 күн бұрын

Alina is smash



0



1



Алина

1 күн бұрын

Today smash



0



0



Nurkanat

1 күн бұрын

Hello World



0



0



Nurkanat

1 күн бұрын

7:50



Жаңа жазба

Жариялау

Hello I am user TrueConnect

1973



TrueConnect

True object

True on next



q¹

w²

e³

r⁴

t⁵

y⁶

u⁷

i⁸

o⁹

p⁰

a

s

d

f

g

h

j

k

l

7:50



Пікірлер

Пікір жоқ

Hello My Followers



Followers

Followers

followe...sbek4
@gmail.com



q¹ w² e³ r⁴ t⁵ y⁶ u⁷ i⁸ o⁹ p⁰

a s d f g h j k l

7:51



Пікірлер



Nurkanat

Hello My Followers

Пікір жазу...



GIF



q¹

w²

e³

r⁴

t⁵

y⁶

u⁷

i⁸

o⁹

p⁰

a

s

d

f

g

h

j

k

l